BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGHIÊN CỨU PHƯƠNG PHÁP TÍCH HỢP VMAMBA VÀO MÔ HÌNH YOLO26-SEG TRONG PHÂN ĐOẠN POLYP TỪ ẢNH NỘI SOI ĐẠI TRỰC TRÀNG

Ngành: Công nghệ thông tin

SINH VIÊN THỰC HIỆN:

1. 2001230490 – Lê Đức Lương – 14DHTH09
2. 2001230312 – Phùng Tuấn Huy – 14DHTH13
3. 2001230830 – Trần Mạnh Toàn – 14DHTH09

TP. HỒ CHÍ MINH, tháng 11 năm 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGHIÊN CỨU PHƯƠNG PHÁP TÍCH HỢP VMAMBA VÀO MÔ HÌNH YOLO26-SEG TRONG PHÂN ĐOẠN POLYP TỪ ẢNH NỘI SOI ĐẠI TRỰC TRÀNG

Ngành: Công nghệ thông tin

GIẢNG VIÊN HƯỚNG DẪN:

Phùng Thế Bảo

SINH VIÊN THỰC HIỆN:

1. 2001230490 – Lê Đức Lương – 14DHTH09
2. 2001230312 – Phùng Tuấn Huy – 14DHTH13
3. 2001230830 – Trần Mạnh Toàn – 14DHTH09

TP. HỒ CHÍ MINH, tháng 11 năm 2026

LỜI CẢM ƠN

Nhóm thực hiện xin gửi lời cảm ơn chân thành đến thầy **Phùng Thế Bảo**, giảng viên hướng dẫn. Trong suốt mười hai tuần thực hiện đề tài, thầy đã định hướng nội dung nghiên cứu, góp ý cho từng giai đoạn và nhắc nhóm luôn kiểm chứng kết quả một cách cẩn thận.

Nhóm xin cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công Thương TP. Hồ Chí Minh. Những kiến thức thầy cô truyền đạt trong những năm học là nền tảng để nhóm thực hiện khóa luận này.

Nhóm cũng cảm ơn các nhóm tác giả đã công bố miễn phí các bộ dữ liệu Kvasir-SEG, Kvasir, CVC-ClinicDB, CVC-ColonDB, ETIS-Larib và PolypGen cùng các thư viện mã nguồn mở được sử dụng trong đề tài. Nhờ đó nhóm có thể thực nghiệm và đối chiếu kết quả. Nhóm cảm ơn nền tảng Kaggle đã cung cấp tài nguyên GPU cho quá trình huấn luyện.

Cuối cùng, nhóm xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ trong suốt thời gian học tập và thực hiện đề tài.

Dù đã cố gắng, khóa luận khó tránh khỏi thiếu sót. Nhóm mong nhận được góp ý của quý thầy cô để hoàn thiện hơn.

TP. Hồ Chí Minh, tháng 11 năm 2026 Nhóm sinh viên thực hiện

MỤC LỤC

LỜI CẢM ƠN i
MỤC LỤC ii
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT vii
Chữ viết tắt vii
Ký hiệu ix
DANH MỤC HÌNH x
DANH MỤC BẢNG xii
MỞ ĐẦU 1
1. Lý do chọn đề tài 1
2. Mục tiêu nghiên cứu 2
3. Đối tượng và phạm vi nghiên cứu 2
4. Phương pháp nghiên cứu 3
5. Đóng góp của đề tài 4
6. Kế hoạch thực hiện và phân công công việc 4
7. Bố cục báo cáo 6
CHƯƠNG 1: TỔNG QUAN 7
1.1. Bối cảnh và bài toán phân đoạn polyp 7
1.2. Các bộ dữ liệu công khai 8
1.3. Các hướng nghiên cứu phân đoạn polyp 8
1.3.1. Mạng tích chập dạng mã hóa – giải mã 8
1.3.2. Transformer và mô hình lai 9
1.3.3. Phát hiện và phân đoạn một giai đoạn: họ YOLO 9
1.3.4. Mô hình không gian trạng thái, Mamba và VMamba 9
1.3.5. Mamba trong họ YOLO 10
1.4. Tổng hợp và khoảng trống nghiên cứu 10
1.5. Tình hình nghiên cứu trong nước 12
1.6. Kết chương 12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 13
2.1. Các bài toán thị giác máy tính 13
2.2. Mạng nơ-ron tích chập 14
2.3. Họ mô hình YOLO 14

2.3.1. Ý tưởng phát hiện một giai đoạn 14
2.3.2. Gán nhãn, hàm mất mát và bỏ NMS 15
2.3.3. Phân đoạn thực thể bằng mặt nạ mẫu 15
2.4. Mô hình YOLO26-seg 15
2.4.1. Kiến trúc 15
2.4.2. Các bước của thuật toán YOLO26-seg 16
2.4.3. Ưu và nhược điểm 17
2.5. Cơ chế chú ý và chi phí tính toán 17
2.6. Mô hình không gian trạng thái, Mamba và VMamba 18
2.6.1. Mô hình không gian trạng thái và rời rạc hóa 18
2.6.2. Mamba: SSM có chọn lọc 18
2.6.3. VMamba và mô-đun SS2D 19
2.6.4. Ưu và nhược điểm của VMamba 19
2.7. Các độ đo đánh giá 20
2.7.1. Các đại lượng cơ bản 20
2.7.2. IoU và Dice 20
2.7.3. AP, mAP50 và mAP50–95 21
2.7.4. Chi phí tính toán 21
2.8. So sánh hai mô hình qua nhiều lần huấn luyện 21
2.8.1. Vì sao cần nhiều seed 21
2.8.2. Kiểm định t ghép cặp 21
2.8.3. Kiểm định Wilcoxon và cỡ hiệu ứng 22
2.9. Kết chương 22
CHƯƠNG 3: MÔ HÌNH ĐỀ XUẤT 23
3.1. Ý tưởng và động cơ thiết kế 23
3.2. Kiến trúc tổng thể 24
3.3. Khối C2TSVMamba 25
3.3.1. Khung tách–xử lý–ghép 25
3.3.2. Nhánh SS2D 26
3.3.3. Nhánh hình dạng 28
3.3.4. Cổng 28
3.3.5. Hợp nhất, mạng truyền thẳng và nối tắt có hệ số 29
3.3.6. Số tham số 29

3.4. Thuật toán quét kết hợp song song 29
3.5. Huấn luyện và lưu trọng số 31
3.5.1. Khởi tạo và học chuyển giao 31
3.5.2. Hàm mất mát và chiến lược huấn luyện 31
3.5.3. Lưu trọng số ở dạng FP32 32
3.6. Ưu điểm và hạn chế của mô hình đề xuất 32
3.7. Các chế độ tách thành phần 33
3.8. Kết chương 34
CHƯƠNG 4: XÂY DỰNG BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU 35
4.1. Nguồn dữ liệu 35
4.1.1. Bộ dữ liệu Kvasir-SEG 35
4.1.2. Ảnh nền normal-cecum 35
4.2. Xây dựng và phân chia bộ dữ liệu BG20 36
4.3. Chuyển mặt nạ phân đoạn thành nhãn đa giác 37
4.4. Đổi kích thước ảnh 38
4.5. Tăng cường dữ liệu 39
4.6. Chuẩn hóa, kiểm tra toàn vẹn và tổng hợp quy trình 41
CHƯƠNG 5: CÀI ĐẶT VÀ ỨNG DỤNG 43
5.1. Môi trường và công cụ 43
5.2. Cài đặt hai mô hình 44
5.2.1. Mô hình gốc YOLO26s-seg 44
5.2.2. Mô hình tích hợp VMamba 45
5.3. Quy trình huấn luyện và lưu trữ kết quả thực nghiệm 46
5.4. Ứng dụng PolypVision: yêu cầu và kiến trúc 46
5.5. Dịch vụ suy luận và tổ chức lưu trữ 48
5.6. Giao diện và chức năng 48
5.7. Triển khai trên dữ liệu thực tế 50
5.8. Hạn chế và kết chương 51
CHƯƠNG 6: THỰC NGHIỆM VÀ ĐÁNH GIÁ 52
6.1. Thiết lập thực nghiệm 52
6.2. So sánh mô hình gốc và mô hình đề xuất trên 20 seed 53
6.2.1. Kết quả tổng hợp 53
6.2.2. Độ ổn định giữa các seed 54

6.2.3. Độ nhạy theo quy tắc chọn epoch 55
6.2.4. Quá trình hội tụ 55
6.2.5. Ma trận nhầm lẫn theo đối tượng 56
6.3. Đánh giá theo pixel và so sánh với PraNet, U-Net 56
6.4. Chỉ số topology của mặt nạ 59
6.5. Chi phí tính toán 60
6.6. Thí nghiệm tách thành phần 62
6.7. Kết quả trên dữ liệu thực tế 64
6.8. Thảo luận 66
6.9. Kết chương 67
KẾT LUẬN 68
1. Kết quả đạt được 68
2. Đánh giá kết quả 68
3. Hạn chế 69
4. Hướng phát triển 69
TÀI LIỆU THAM KHẢO 71
PHỤ LỤC 78
Phụ lục A. Kết quả từng seed 78
A.1. Mô hình gốc và mô hình đề xuất (20 seed) 78
A.2. Dice theo pixel của bốn mô hình (20 seed) 78
A.3. Thí nghiệm tách thành phần (5 seed) 79
Phụ lục B. Cấu hình huấn luyện 79
Phụ lục C. Tổ chức mã nguồn và tệp kết quả 80
Phụ lục D. Hướng dẫn chạy lại 81
Phụ lục E. Phân công theo hạng mục công việc 82
Phụ lục F. Chi tiết khối C2TSVMamba 83
F.1. Truy vết kích thước tensor 83
F.2. Ví dụ phép quét song song 83
F.3. Ước tính chi phí tầng 10 84
F.4. Quy trình suy luận và huấn luyện của mô hình đề xuất 85
Phụ lục G. API và cấu trúc lưu trữ của ứng dụng PolypVision 85
Phụ lục H. Kết quả thực nghiệm bổ sung 88
H.1. So sánh hai mô hình YOLO trên tập kiểm định 88

H.2. Đánh giá theo pixel, topology và độ trễ 91
H.3. Dữ liệu ngoài 93

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

CHỮ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
BG20	Background 20%	Bộ dữ liệu của đề tài: Kvasir-SEG cộng số ảnh nền bằng 20% số ảnh polyp
BN	Batch Normalization	Chuẩn hóa theo lô
C2PSA	CSP block with Partial Self-Attention	Khối tự chú ý từng phần của YOLO26 (tầng 10 của mô hình gốc)
C2TSVMamba	CSP block with Topology–Shape-aware VMamba	Khối đề xuất thay C2PSA ở tầng 10
CI	Confidence Interval	Khoảng tin cậy
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu ảnh dùng để tiền huấn luyện YOLO
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSP	Cross Stage Partial	Cấu trúc tách–ghép kênh qua các tầng
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song trên GPU của NVIDIA
CVC	Computer Vision Center	Trung tâm Thị giác máy tính (Barcelona), nơi công bố các bộ CVC-ClinicDB, CVC-ColonDB
DFL	Distribution Focal Loss	Hàm mất mát tiêu điểm theo phân phối
EMA	Exponential Moving Average	Trung bình trượt hàm mũ
FFN	Feed-Forward Network	Mạng truyền thẳng
FLOPs	Floating-Point Operations	Số phép tính dấu phẩy động
FN	False Negative	Âm tính giả (polyp bị bỏ sót)
FP	False Positive	Dương tính giả (báo nhầm)
FP16, FP32	16-bit / 32-bit Floating Point	Số thực dấu phẩy động 16 bit / 32 bit
FPN	Feature Pyramid Network	Mạng kim tự tháp đặc trưng

Viết tắt	Tiếng Anh	Tiếng Việt
GFLOPs	Giga Floating-Point Operations	Tỉ phép tính dấu phẩy động
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
HSV	Hue, Saturation, Value	Không gian màu sắc độ – bão hòa – độ sáng
IEEE	Institute of Electrical and Electronics Engineers	Viện Kỹ sư Điện và Điện tử (kiểu trích dẫn)
IoU	Intersection over Union	Tỉ lệ giao trên hợp
MAC	Multiply–Accumulate	Phép nhân–cộng
mAP	mean Average Precision	Trung bình độ chính xác trung bình
mAP50, mAP50–95	mAP at IoU 0.5 / averaged over IoU 0.5–0.95	mAP ở ngưỡng IoU 0,5 / trung bình trên các ngưỡng IoU 0,5–0,95
NMS	Non-Maximum Suppression	Lọc trùng hộp không cực đại
P, R	Precision, Recall	Độ chính xác, độ phủ
P3, P4, P5	Pyramid levels 3–5	Các mức đặc trưng có bước 8, 16, 32 pixel
PAN	Path Aggregation Network	Mạng tổng hợp đường dẫn
PSA	Partial Self-Attention	Tự chú ý từng phần
SPPF	Spatial Pyramid Pooling – Fast	Gộp kim tự tháp không gian, bản nhanh
SS2D	2D Selective Scan	Quét chọn lọc hai chiều (bốn hướng)
SSM	State Space Model	Mô hình không gian trạng thái
STAL	Small-Target-Aware Label assignment	Gán nhãn có ưu tiên vật nhỏ
TN	True Negative	Âm tính đúng
TP	True Positive	Dương tính đúng
TSVM	Topology–Shape-aware VMamba	Tên gọi tắt của mô hình đề xuất (YOLO26s-seg + C2TSVMamba)
VMamba	Visual Mamba (Visual State Space Model)	Mô hình không gian trạng thái thị giác
YOLO	You Only Look Once	Họ mô hình phát hiện một giai đoạn
ZOH	Zero-Order Hold	Giữ bậc không (cách rời rạc hóa SSM)

KÝ HIỆU

Ký hiệu	Ý nghĩa	Xuất hiện ở
A, B, C, Δ	Ma trận trạng thái, ma trận đầu vào, ma trận đầu ra và bước thời gian của mô hình không gian trạng thái	Mục 2.6, 3.3–3.4
$\bar{A}, \bar{B}$	A, B sau rời rạc hóa	Mục 2.6.2
h_t	Trạng thái ẩn tại bước t	Mục 2.6, 3.4
F, G	Đặc trưng của nhánh SS2D và tín hiệu cổng; cổng có dạng $F \cdot (1 + G)$	Mục 3.3.4
b_0, b_1	Số mảnh và số lỗ của mặt nạ (hai số Betti đầu tiên)	Mục 6.4
Δ (ở Chương 6)	Hiệu số chỉ số giữa hai mô hình cùng seed	Chương 6
d_z	Cỡ hiệu ứng Cohen cho dữ liệu ghép cặp	Mục 2.8.3, 6.2
p	Giá trị p của kiểm định thống kê	Mục 2.8, Chương 6
SD	Độ lệch chuẩn qua các seed (ddof = 1)	Chương 6

DANH MỤC HÌNH

Hình 2.1: Bốn kiểu đầu ra trên cùng một ảnh tập kiểm định có hai polyp: (a) phân loại ảnh, (b) phát hiện đối tượng bằng hộp bao, (c) phân đoạn ngữ nghĩa (hai polyp chung một mặt nạ), (d) phân đoạn thực thể (mỗi polyp một mặt nạ riêng). Hộp bao và mặt nạ vẽ từ nhãn đa giác của ảnh; xác suất ở ô (a) là số minh họa, không phải đầu ra mô hình. 13

Hình 3.1: Kiến trúc YOLO26s-seg và vị trí khối C2TSVMamba (tầng 10). Kích thước tensor ghi theo dạng C×H×W, tính cho ảnh vào 640×640 với hệ số quy mô s. Tầng 22 (C3k2, attn=True) có PSABlock ở cả hai mô hình. 24

Hình 3.2: Cấu trúc khối C2TSVMamba. Khung nét đứt là khối TSVMamba. Đặc trưng bên trong khối có kích thước 256×20×20. Chi tiết nhánh SS2D ở Hình 3.4. 26

Hình 3.3: Bốn thứ tự duyệt của SS2D, minh họa trên lưới 4×4 (thực tế là 20×20, L = 400). Chấm tròn đánh dấu vị trí đầu chuỗi, ô vuông đánh dấu vị trí cuối chuỗi. 26

Hình 3.4: Các bước bên trong nhánh SS2D như được cài đặt. Bước ③ đến ⑤ được thực hiện riêng cho từng hướng. 26

Hình 4.1: Ví dụ ảnh có polyp, mặt nạ phân đoạn gốc và ảnh nền trong bộ dữ liệu BG20 36

Hình 4.2: Trước và sau khi chuyển mặt nạ phân đoạn thành nhãn đa giác..... 38

Hình 4.3: Trước và sau khi đổi kích thước ảnh bằng thu phóng giữ tỷ lệ và LetterBox..... 39

Hình 4.4: Ảnh huấn luyện trước và sau các phép tăng cường dữ liệu 41

Hình 4.5: Quy trình xây dựng bộ dữ liệu BG20 và tiền xử lý dữ liệu 42

Hình 5.2: Kiến trúc hệ thống PolypVision. Website (Spring Boot, cổng 8080) và ứng dụng di động (Flutter) là hai client độc lập của cùng dịch vụ suy luận FastAPI (cổng 8000). Chỉ nhánh Web có MongoDB và kho tệp ảnh. 47

Hình 6.1: Mask mAP50–95 của từng seed. Mỗi đường nối hai lượt cùng seed của hai mô hình. 54

Hình 6.2: Ma trận nhầm lẫn trung bình 20 seed. Ô nền–nền để trống vì không xác định được ở mức đối tượng. 56

Hình 6.3: Dice, IoU, Precision và Recall theo pixel của bốn mô hình. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed. 57

Hình 6.4: Ví dụ dự đoán của bốn mô hình (seed 0). Ba ảnh được chọn tự động: ảnh cả bốn mô hình đều tốt, ảnh TSVM hơn mô hình gốc nhiều nhất và ảnh khó nhất. 58

Hình 6.5: Tỉ lệ ảnh có polyp được dự đoán đúng topology (số mảnh và số lỗ khớp nhãn). 59

Hình 6.6: Mask mAP50–95 từng seed (0–4) của năm cấu hình. Đường xám nối kết quả của cùng một seed; vạch ngang đậm là trung bình, thanh dọc là ±1 độ lệch chuẩn qua 5 seed. 63

Hình 6.7: Dice theo pixel của bốn mô hình trên bốn bộ dữ liệu ngoài. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed. 65

Hình 6.8: Tỉ lệ ảnh không có polyp của PolypGen (125 ảnh) bị báo nhầm. 66

Hình PL.1: Số tham số của từng thành phần trong khối C2TSVMamba. Tiêu đề hình ghi tổng của C2TSVMamba và của C2PSA để so sánh. 83

Hình PL.2: Trung bình các chỉ số mặt nạ của hai mô hình trên 20 seed. Thanh lỗi là ±1 độ lệch chuẩn qua các seed. 88

Hình PL.3: Hiệu số mask mAP50–95 (TSVM − gốc) của từng seed, sắp xếp từ nhỏ đến lớn. 88

Hình PL.4: Phân bố mask mAP50–95 qua 20 seed của hai mô hình. 89

Hình PL.5: Mask mAP50–95 theo ba quy tắc chọn epoch. Điểm là trung bình, thanh là ±1 độ lệch chuẩn. 89

Hình PL.6: Mask mAP50–95 trên tập kiểm định theo epoch. Đường là trung bình 20 seed, miền tô là ±1 độ lệch chuẩn. Ô nhỏ phóng to đoạn epoch 70–100. 90

Hình PL.7: Hàm mất mát phân đoạn trên tập kiểm định theo epoch. Dấu ▼ đánh dấu epoch có giá trị trung bình thấp nhất. 91

Hình PL.8: Dice theo pixel của từng seed. 91

Hình PL.9: Tỉ lệ ảnh nền bị báo nhầm (có ít nhất một pixel dự đoán là polyp) trên 40 ảnh nền. 92

Hình PL.10: Tỉ lệ ảnh mắc từng kiểu lỗi topology: thừa mảnh, có lỗ và thiếu mảnh. 92

Hình PL.11: Độ trễ trên CPU của hai mô hình ở hai chế độ đo. Thanh lỗi là ±1 độ lệch chuẩn qua 100 lần đo. 93

Hình PL.12: Dice theo pixel trên từng trung tâm của PolypGen. 94

DANH MỤC BẢNG

Bảng 0.1: Kế hoạch thực hiện theo tuần 4
Bảng 0.2: Phân công nhiệm vụ trong nhóm 5
Bảng 1.1: Một số bộ dữ liệu phân đoạn polyp công khai 8
Bảng 1.2: Tổng hợp các nhóm phương pháp phân đoạn polyp 10
Bảng 1.3: So sánh đề tài với các công trình gần nhất 11
Bảng 2.1: So sánh ba bài toán thị giác cơ bản 13
Bảng 2.2: Các khối chính của YOLO26s-seg (ảnh vào 640×640)..... 15
Bảng 2.3: Ưu và nhược điểm của YOLO26-seg với bài toán phân đoạn polyp .. 17
Bảng 2.4: Ưu và nhược điểm của VMamba..... 19
Bảng 3.1: So sánh mô hình đối chứng và mô hình đề xuất..... 25
Bảng 3.2: Số tham số của tầng 10 (quy mô s)..... 29
Bảng 3.3: Các thành phần mất mát của YOLO26-seg (theo đúng nhánh mã đã chạy) 32
Bảng 3.4: Bốn chế độ của khối TSVMamba..... 34
Bảng 4.1: Phân bố ảnh và nhãn trong bộ dữ liệu BG20..... 36
Bảng 4.2: Thông số tăng cường dữ liệu trong cấu hình huấn luyện 40
Bảng 5.1: Môi trường huấn luyện và đánh giá..... 43
Bảng 5.2: Siêu tham số huấn luyện (giống nhau ở mọi lượt của cả hai mô hình)44
Bảng 5.3: Các tệp được thêm hoặc sửa trong fork..... 45
Bảng 5.4: Đối chiếu yêu cầu đề cương với chức năng đã cài đặt 47
Bảng 5.5: Kết quả kiểm thử tự động **[CẦN KIỂM TRA: chạy lại các lệnh kiểm thử trước khi nộp và cập nhật số]**..... 50
Bảng 6.1: Chỉ số mặt nạ của hai mô hình (20 seed, epoch có mask mAP50–95 lớn nhất) 53
Bảng 6.2: Mức dao động của mask mAP50–95 qua 20 seed..... 54
Bảng 6.3: Mask mAP50–95 theo ba quy tắc chọn epoch 55
Bảng 6.4: Số vùng polyp phát hiện đúng, bỏ sót và dự đoán thừa (trên 127 vùng polyp) 56
Bảng 6.5: Chỉ số theo pixel của bốn mô hình trên tập kiểm định (20 seed)..... 56
Bảng 6.6: So sánh ghép cặp TSVM với từng mô hình theo pixel (cùng seed).... 57
Bảng 6.7: Chỉ số topology của bốn mô hình (ngưỡng 20 pixel, 20 seed)..... 59
Bảng 6.8: Số tham số và số phép tính của hai mô hình 60

Bảng 6.9: Độ trễ suy luận của hai mô hình (checkpoint seed 0, lô 1 ảnh, ảnh 640 × 640, FP32) 61
Bảng 6.10: Kết quả thí nghiệm tách thành phần (mask mAP50–95, seed 0–4) .. 62
Bảng 6.11: Dice và IoU theo pixel trên bốn bộ dữ liệu ngoài (20 seed) 64
Bảng PL.1: Chỉ số mặt nạ của từng seed trên tập kiểm định 78
Bảng PL.2: Dice theo pixel trên 120 ảnh có polyp của tập kiểm định 78
Bảng PL.3: Mask mAP50–95 từng seed của năm cấu hình 79
Bảng PL.4: Siêu tham số huấn luyện (theo `args.yaml` của 40 lượt chính) 79
Bảng PL.5: Các thư mục chính của kho mã 80
Bảng PL.6: Script tạo ra số liệu trong báo cáo 81
Bảng PL.7: Phân công theo hạng mục công việc (C: chính, H: hỗ trợ) 82
Bảng PL.8: Kích thước tensor qua khối C2TSVMamba (một ảnh 640×640) 83
Bảng PL.9: Các vòng của phép quét song song trên ví dụ 84
Bảng PL.10: Ước tính chi phí phần xử lý chính của tầng 10. Không tính cv1, cv2 vì hai khối dùng chung, khoảng 210 triệu MAC 84
Bảng PL.11: Các API của dịch vụ suy luận 85
Bảng PL.12: Các trường chính trong kết quả suy luận của API `/predict` 86
Bảng PL.13: Tập `users` trong MongoDB 86
Bảng PL.14: Tập `diagnosis_history` trong MongoDB 87
Bảng PL.15: Mask mAP50–95 tại một số epoch 90
Bảng PL.16: Tỉ lệ ảnh có polyp bị bỏ sót hoàn toàn (mô hình không dự đoán pixel nào) 93
Bảng PL.17: Dice theo pixel trên từng trung tâm của PolypGen (20 seed) 93

MỞ ĐẦU

1. LÝ DO CHỌN ĐỀ TÀI

Ung thư đại trực tràng là một trong những bệnh ung thư phổ biến và gây tử vong nhiều nhất. Theo ước tính GLOBOCAN năm 2022, bệnh này chiếm 9,6% số ca ung thư mới, đứng thứ ba, và 9,3% số ca tử vong do ung thư, đứng thứ hai [1]. Phần lớn các ca bệnh phát triển từ polyp ở niêm mạc ruột, nên phát hiện và cắt bỏ polyp sớm qua nội soi là cách phòng bệnh quan trọng [2]. Tuy vậy, polyp vẫn bị bỏ sót với tỉ lệ đáng kể [3], trong khi phát hiện được nhiều u tuyến hơn gắn với nguy cơ ung thư thấp hơn [4]. Một công cụ tự động khoanh vùng polyp trên ảnh vì thế có giá trị hỗ trợ thực tế.

Bài toán đặt ra là **phân đoạn polyp**: xác định những điểm ảnh thuộc polyp, chứ không chỉ đánh dấu một hộp bao quanh. Bài toán khó vì polyp có màu và kết cấu gần giống niêm mạc, đường biên mờ, ảnh có phản quang, nếp gấp, bọt che khuất, và kích thước polyp thay đổi lớn. Các bộ dữ liệu công khai như Kvasir-SEG [5] cho phép nghiên cứu bài toán này theo một quy trình có thể kiểm tra lại.

YOLO26-seg [6] (công bố năm 2026, bản tiền ấn phẩm) phát hiện và phân đoạn từng đối tượng trong một lần suy luận, không cần bước lọc trùng NMS. Ngữ cảnh toàn cục của mô hình chủ yếu đến từ khối chú ý đặt ở mức đặc trưng sâu nhất; các khối còn lại phần lớn là tích chập, chỉ nhìn vùng lân cận nhỏ. VMamba [7] là một cách khác để lấy ngữ cảnh toàn cục: mô-đun SS2D quét bản đồ đặc trưng theo bốn hướng. Đề tài kết hợp cơ chế quét này với một nhánh trích đặc trưng hình dạng và đường biên, vốn là manh mối quan trọng với polyp. Việc thay thế không nhằm làm mô hình nhẹ hơn: khối mới làm tăng số tham số và thời gian suy luận, và mức tăng đó là một câu hỏi cần đo.

Từ đó, đề tài đặt câu hỏi: **thay khối chú ý ở tầng sâu của YOLO26-seg bằng một khối VMamba có thêm nhánh nhận biết hình dạng thì chất lượng phân đoạn polyp thay đổi thế nào, và phải trả thêm bao nhiêu chi phí tính toán?** Để kết luận không dựa vào một lần huấn luyện may mắn, mỗi mô hình được huấn luyện lặp lại với 20 seed (seed 0–19), rồi so sánh ghép cặp có kiểm định thống kê.

2. MỤC TIÊU NGHIÊN CỨU

Mục tiêu tổng quát: nghiên cứu và đánh giá phương pháp tích hợp khối VMamba vào YOLO26-seg cho bài toán phân đoạn polyp trên ảnh nội soi đại trực tràng, đồng thời xây dựng ứng dụng minh họa.

Mục tiêu cụ thể (theo đề cương đã duyệt):

1. Khảo sát cơ sở lý thuyết và các công trình liên quan đến phân đoạn polyp, YOLO26-seg, Mamba và VMamba.
2. Xây dựng và tiền xử lý bộ dữ liệu ảnh nội soi có mặt nạ phân đoạn từ các bộ dữ liệu công khai, dùng một cách chia dữ liệu thống nhất cho mọi thực nghiệm.
3. Đề xuất và cài đặt khối C2TSVMamba thay khối C2PSA ở tầng 10 của YOLO26-seg.
4. So sánh mô hình đề xuất với YOLO26-seg nguyên bản về chất lượng phân đoạn và chi phí tính toán; xây dựng ứng dụng minh họa.

Câu hỏi nghiên cứu:

- **CH1.** Trên cùng dữ liệu và cấu hình huấn luyện, mô hình tích hợp có mask mAP50–95 trung bình cao hơn mô hình gốc không, và chênh lệch có ý nghĩa thống kê không? Dao động giữa các seed (khoảng 0,01) lớn hơn mức chênh lệch có thể kỳ vọng (vài phần nghìn), nên khả năng phát hiện một chênh lệch nhỏ vẫn hạn chế dù có 20 seed.
- **CH2.** Mỗi thành phần của khối đề xuất (nhánh SS2D, nhánh hình dạng, cổng điều khiển) đóng góp bao nhiêu? Câu hỏi này được trả lời bằng thí nghiệm tách thành phần (ablation).
- **CH3.** Khối đề xuất làm tăng số tham số, chi phí tính toán và thời gian suy luận bao nhiêu?
- **CH4.** Mô hình đã huấn luyện có chạy đúng trong một ứng dụng thực tế (Web, Mobile) không, và hoạt động ra sao trên ảnh ngoài tập huấn luyện?

3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu: ảnh nội soi đại trực tràng dạng ảnh tĩnh; mô hình YOLO26s-seg (cỡ *small*) và mô hình YOLO26s-seg tích hợp khối C2TSVMamba; mô-đun quét chọn lọc hai chiều SS2D của VMamba.

Phạm vi:

- **Dữ liệu:** bộ BG20 gồm 1.200 ảnh, là 1.000 ảnh polyp của Kvasir-SEG [5] và 200 ảnh manh tràng bình thường của Kvasir v2 [8] làm ảnh nền; chia 1.040 ảnh huấn luyện và 160 ảnh kiểm định, phần polyp dùng đúng danh sách chia 880/120 của nhóm tác giả bộ dữ liệu. Không có tập kiểm tra riêng: chỉ số được đo và epoch tốt nhất được chọn trên tập kiểm định, nên số tuyệt đối có thể lạc quan hơn thực tế, nhưng hạn chế này như nhau cho cả hai mô hình.
- **Dữ liệu thực tế:** bốn bộ dữ liệu công khai khác nguồn (CVC-ClinicDB 612 ảnh, CVC-ColonDB 380 ảnh, ETIS-Larib 196 ảnh, phần ảnh đơn của PolypGen 1.537 ảnh từ 6 trung tâm), dùng nguyên checkpoint, không huấn luyện lại. Đây không phải dữ liệu thu tại bệnh viện.
- **Mô hình và thực nghiệm:** một lớp đối tượng (polyp); hai mô hình chỉ khác nhau ở tầng 10, mọi siêu tham số khác giống nhau; 100 epoch, ảnh vào 640×640, 20 seed. PraNet [9] và U-Net [10] được huấn luyện trên cùng dữ liệu để đối chiếu chỉ số theo điểm ảnh.
- **Ứng dụng:** PolypVision gồm dịch vụ suy luận (FastAPI), website (Spring Boot) và ứng dụng di động (Flutter), xử lý ảnh tĩnh, chưa xử lý video trực tiếp. Ứng dụng phục vụ nghiên cứu, **không phải thiết bị y tế**.

4. PHƯƠNG PHÁP NGHIÊN CỨU

- **Nghiên cứu lý thuyết:** tổng hợp tài liệu về phân đoạn polyp, họ YOLO, cơ chế chú ý, mô hình không gian trạng thái, Mamba và VMamba (Chương 1, Chương 2).
- **Thực nghiệm có đối chứng:** mô hình gốc là nhóm đối chứng, mô hình tích hợp là nhóm thử nghiệm; hai mô hình dùng chung dữ liệu, tiền xử lý, siêu tham số và danh sách seed, nên khác biệt có hệ thống chỉ đến từ tầng 10. Kết quả ghép cặp theo seed, kiểm định bằng t ghép cặp và Wilcoxon, kèm khoảng tin cậy 95% và cỡ hiệu ứng. Thí nghiệm tách thành phần tắt lần lượt từng nhánh để đo đóng góp.
- **Độ đo** (mục 2.7): mask mAP50–95 (độ đo chính), mask mAP50, Precision, Recall, F1; Dice, IoU theo điểm ảnh; số tham số, FLOPs, thời gian suy luận trên CPU và GPU.
- **Nghiên cứu ứng dụng:** xây dựng PolypVision, tích hợp hai checkpoint, cho phép tải ảnh, xem mặt nạ, độ tin cậy và so sánh hai mô hình trên cùng ảnh.

5. ĐÓNG GÓP CỦA ĐỀ TÀI

– **Về mô hình:** thiết kế và cài đặt khối C2TSVMamba thay khối C2PSA ở tầng 10 của YOLO26s-seg, kết hợp nhánh SS2D quét bốn hướng với nhánh đặc trưng hình dạng đa tỉ lệ có hướng qua một cổng điều khiển. Bản cài đặt chỉ dùng PyTorch, không cần biên dịch kernel CUDA riêng.

– **Về kỹ thuật:** phát hiện lỗi tràn số khi lưu checkpoint FP16 (một thống kê của lớp chuẩn hóa lô trong khối mới vượt giới hạn FP16) và chuyển sang lưu FP32.

– **Về đánh giá:** so sánh trên 20 seed ghép cặp có kiểm định, khoảng tin cậy và cỡ hiệu ứng; thí nghiệm tách thành phần; đánh giá trên bốn bộ dữ liệu ngoài; mọi con số truy được về tệp kết quả gốc và script tính lại.

– **Về ứng dụng:** PolypVision có dịch vụ suy luận dùng chung cho Web và Mobile, so sánh được hai mô hình trên cùng ảnh.

Đề tài không khẳng định mô hình đề xuất tốt hơn mô hình gốc khi chưa có bằng chứng thống kê; kết quả và mức độ tin cậy được trình bày ở Chương 6.

6. KẾ HOẠCH THỰC HIỆN VÀ PHÂN CÔNG CÔNG VIỆC

Đề tài thực hiện trong 12 tuần, từ 17/08/2026 đến 08/11/2026, nội dung từng tuần theo đề cương đã duyệt.

Bảng 0.1: Kế hoạch thực hiện theo tuần

Tuần	Thời gian	Nội dung công việc	Người phụ trách	Sản phẩm	Trạng thái
1	17/08 – 23/08	Xác định lý do chọn đề tài, khảo sát các công trình liên quan. Lập kế hoạch và phân công	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
2	24/08 – 30/08	Thu thập, phân tích bộ dữ liệu ảnh polyp đại tràng (Kaggle và nguồn công khai)	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
3	31/08 – 06/09	Làm sạch, chuyển nhãn, tiền xử lý dữ liệu cho 1.000 ảnh polyp và ảnh nền	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
4	07/09 – 13/09	Mô tả bài toán, thiết lập môi trường huấn luyện, chọn ngôn ngữ và framework	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

Tuần	Thời gian	Nội dung công việc	Người phụ trách	Sản phẩm	Trạng thái
5	14/09 – 20/09	Cài đặt và huấn luyện mô hình gốc YOLO26-seg	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
6	21/09 – 27/09	Cài đặt và huấn luyện các mô hình so sánh	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
7	28/09 – 04/10	Cài đặt thành phần cải tiến: tích hợp VMamba vào YOLO26-seg	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
8	05/10 – 11/10	Thực nghiệm và đánh giá bằng Dice, IoU, Precision, Recall, F1, mAP50, mAP50–95, tốc độ suy luận, số tham số, chi phí tính toán	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
9	12/10 – 18/10	Đánh giá trên dữ liệu thực tế. So sánh và chọn mô hình	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
10	19/10 – 25/10	Xây dựng backend, tổ chức lưu trữ kết quả dự đoán	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
11	26/10 – 01/11	Cài đặt chức năng, thiết kế giao diện Web/App: tải ảnh, hiển thị mặt nạ polyp và độ tin cậy	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
12	02/11 – 08/11	Hoàn chỉnh báo cáo, thiết kế slide, nộp báo cáo	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

Bảng 0.2: Phân công nhiệm vụ trong nhóm

Thành viên	MSSV	Lớp	Vai trò	Nhiệm vụ chính	Chương phụ trách viết	Mức đóng góp (%)
Lê Đức Lương	2001230490	14DHTH09	[ĐIỀN, ví dụ: Nhóm trưởng]	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Phùng Tuấn Huy	2001230312	14DHTH13	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Trần Mạnh Toàn	2001230830	14DHTH09	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

Phân công chi tiết theo hạng mục công việc ở Phụ lục E. Nhóm gặp giảng viên hướng dẫn ít nhất một lần mỗi tuần; mã nguồn, cấu hình và kết quả huấn luyện

được quản lý bằng Git **[ĐIỀN: tên kho/nền tảng nếu muốn nêu]**; các lượt huấn luyện được chia cho nhiều tài khoản Kaggle để chạy song song. [ĐIỀN: công cụ trao đổi trong nhóm, tần suất họp nhóm.]

7. BỐ CỤC BÁO CÁO

Ngoài Mở đầu và Kết luận, báo cáo gồm sáu chương: Chương 1 tổng quan bài toán, dữ liệu công khai và các hướng nghiên cứu; Chương 2 cơ sở lý thuyết về YOLO26-seg, Mamba, VMamba, các độ đo và kiểm định thống kê; Chương 3 mô hình đề xuất C2TSVMamba; Chương 4 dữ liệu và tiền xử lý; Chương 5 cài đặt và ứng dụng PolypVision; Chương 6 thực nghiệm và đánh giá. Phụ lục gồm kết quả từng seed, cấu hình huấn luyện, tổ chức mã nguồn, hướng dẫn chạy lại và các chi tiết được chuyển khỏi phần thân.

CHƯƠNG 1: TỔNG QUAN

Chương này trình bày bối cảnh bài toán phân đoạn polyp, các bộ dữ liệu công khai và các hướng nghiên cứu chính, từ mạng tích chập, Transformer đến mô hình không gian trạng thái Mamba, rồi chỉ ra khoảng trống mà khóa luận hướng tới.

1.1. BỐI CẢNH VÀ BÀI TOÁN PHÂN ĐOẠN POLYP

Ung thư đại trực tràng đứng thứ ba về số ca mắc mới và thứ hai về số ca tử vong trong các bệnh ung thư năm 2022 [1]. Phần lớn các ca phát triển qua nhiều năm từ polyp; u tuyến (adenoma) là loại có nguy cơ hóa ác cao, và cắt bỏ polyp sớm khi nội soi giúp giảm nguy cơ ung thư [2]. Tuy vậy, kết quả nội soi phụ thuộc vào kinh nghiệm và sự tập trung của bác sĩ:

- khi nội soi hai lần liên tiếp, lần đầu bỏ sót khoảng 26% u tuyến, với u tuyến phẳng khoảng 34% [3];
- tỉ lệ phát hiện u tuyến của bác sĩ tăng 1% thì nguy cơ ung thư xuất hiện giữa hai lần nội soi giảm khoảng 3% [4].

Hệ thống máy tính hỗ trợ khoanh vùng polyp vì thế có ích: không thay bác sĩ ra quyết định, mà nhắc chú ý những vùng nghi ngờ có thể bị bỏ qua.

Phát biểu bài toán. Đầu vào là một ảnh nội soi màu; đầu ra là mặt nạ nhị phân cùng kích thước, điểm ảnh thuộc polyp mang giá trị 1. Đề tài giải bài toán ở dạng **phân đoạn thực thể**: mỗi polyp có hộp bao, độ tin cậy và mặt nạ riêng; các mặt nạ có thể gộp lại để so sánh với phương pháp phân đoạn ngữ nghĩa (mục 2.1). So với hộp bao, mặt nạ cho biết hình dạng và đường biên tổn thương.

Khó khăn đặc thù [11]: polyp có tương phản thấp với niêm mạc, nhất là polyp phẳng; ảnh có phản quang, bọt, dịch, nếp gấp và nhòe; kích thước và hình dạng polyp rất đa dạng; thiết bị và quy trình khác nhau giữa các trung tâm làm mô hình kém đi khi gặp ảnh nơi khác [12]; dữ liệu có nhãn ít vì vẽ mặt nạ cần chuyên gia. Do đó mô hình cần cùng lúc nhìn chi tiết cục bộ để bám đường biên và nắm ngữ cảnh toàn ảnh để phân biệt polyp với vùng trông giống polyp.

1.2. CÁC BỘ DỮ LIỆU CÔNG KHAI

Bảng 1.1: Một số bộ dữ liệu phân đoạn polyp công khai

Bộ dữ liệu	**Số ảnh**	**Đặc điểm**	**Vai trò trong đề tài**
Kvasir-SEG [5]	1.000	Ảnh polyp kèm mặt nạ do bác sĩ vẽ và chuyên gia duyệt; độ phân giải khác nhau	Nguồn ảnh polyp chính
Kvasir [8]	Bản v2: 8 lớp, mỗi lớp 1.000 ảnh	Ảnh nội soi tiêu hóa phân loại theo mốc giải phẫu, tổn thương, thủ thuật	Lấy 200 ảnh lớp *normal-cecum* làm ảnh nền
CVC-ClinicDB [13]	612	Khung hình cắt từ video nội soi, kèm mặt nạ	Dữ liệu ngoài, kiểm tra trên dữ liệu thực tế
CVC-ColonDB [14]	380 (bản dùng trong các benchmark phân đoạn từ PraNet [9]; bài gốc mô tả 300 ảnh)	Khung hình cắt từ 15 đoạn video nội soi	Dữ liệu ngoài
ETIS-Larib [15]	196	Nhiều polyp nhỏ, khó phát hiện	Dữ liệu ngoài
PolypGen [12]	1.537 ảnh đơn từ 6 trung tâm (1.412 có polyp, 125 không polyp)	Thu ở nhiều bệnh viện và thiết bị	Dữ liệu ngoài, đánh giá theo từng trung tâm

Kvasir-SEG được chọn làm dữ liệu huấn luyện vì có mặt nạ chất lượng, được dùng phổ biến và có sẵn danh sách chia huấn luyện/kiểm định của nhóm tác giả. Bốn bộ còn lại khác nguồn với Kvasir-SEG, dùng để kiểm tra mô hình trên dữ liệu chưa thấy (mục 6.7).

1.3. CÁC HƯỚNG NGHIÊN CỨU PHÂN ĐOẠN POLYP

1.3.1. Mạng tích chập dạng mã hóa – giải mã

U-Net [10] gồm nửa mã hóa giảm kích thước, tăng số kênh; nửa giải mã phóng to dần; và các kết nối tắt chuyển chi tiết không gian từ nửa mã hóa sang nửa giải mã, nhờ đó học được với lượng ảnh nhỏ. **PraNet** [9] dự đoán bản đồ vị trí thô rồi dùng *chú ý ngược*: xóa vùng đã chắc chắn là polyp để tập trung tinh chỉnh vùng biên còn mơ hồ. Nhóm này bám tốt chi tiết cục bộ, nhưng mỗi lớp tích chập chỉ

nhìn vùng nhỏ; muốn có ngữ cảnh toàn ảnh phải xếp chồng nhiều lớp và giảm kích thước nhiều lần.

1.3.2. Transformer và mô hình lai

Cơ chế tự chú ý cho mỗi vị trí so sánh trực tiếp với mọi vị trí khác, nên có ngữ cảnh toàn cục ngay trong một lớp. **Polyp-PVT** [16] dùng Pyramid Vision Transformer làm bộ mã hóa, thêm mô-đun hợp nhất đặc trưng; **ColonFormer** [17] (nhóm tác giả Việt Nam) dùng bộ mã hóa Transformer phân cấp và mô-đun tinh chỉnh biên. Hạn chế chung là chi phí: với N vị trí, attention phải tính N^2 cặp tương quan (mục 2.5).

1.3.3. Phát hiện và phân đoạn một giai đoạn: họ YOLO

YOLO [18] xem phát hiện vật thể là một phép hồi quy: ảnh đi qua mạng một lần và cho ra trực tiếp hộp bao và lớp, nên nhanh hơn nhiều so với mô hình hai giai đoạn. Các phiên bản sau thêm nhánh mặt nạ theo cách của YOLACT [19] để phân đoạn thực thể; YOLOv10 [20] đưa ra cách huấn luyện hai đầu để bỏ NMS, cùng khối chú ý một phần PSA. **YOLO26** [6] (Ultralytics, 2026, bản tiền ấn phẩm) kế thừa suy luận không cần NMS và khối C2PSA ở cuối backbone, có đầu riêng cho phân đoạn (mục 2.4). Họ YOLO đã được dùng cho polyp nhưng chủ yếu ở bài toán phát hiện bằng hộp bao, ví dụ YOLOv4 cải tiến cho phát hiện polyp thời gian thực [21].

1.3.4. Mô hình không gian trạng thái, Mamba và VMamba

Mô hình không gian trạng thái (SSM) duy trì một trạng thái ẩn và cập nhật theo từng phần tử của chuỗi. **S4** [22] đưa ra cách tham số hóa giúp SSM xử lý chuỗi rất dài; **Mamba** [23] cho tham số SSM phụ thuộc đầu vào để mô hình tự chọn giữ hay quên thông tin, với chi phí gần tuyến tính theo độ dài chuỗi. Để áp dụng cho ảnh, **VMamba** [7] đề xuất mô-đun SS2D quét bản đồ đặc trưng theo bốn hướng rồi gộp kết quả, nhờ đó mỗi vị trí nhận thông tin từ cả bốn phía. Đề tài dùng SS2D làm nhánh ngữ cảnh toàn cục.

Trong ảnh y tế, **U-Mamba** [24] chèn khối lai CNN–SSM vào nnU-Net; **Swin-UMamba** [25] cho thấy bộ mã hóa Mamba khởi tạo từ trọng số tiền huấn luyện ImageNet cải thiện rõ kết quả, điều liên quan trực tiếp vì nhánh SS2D của đề tài được khởi tạo mới. Riêng với polyp, **Polyp-Mamba** [26] kết hợp Visual Mamba với mô-đun ngữ nghĩa theo tỉ lệ; **PolyMamba** [27] thêm thông tin vị trí và lọc tần số cao có cổng để làm rõ biên; **Topo-VM-UNetV2** [28] đưa bản đồ chú ý tô-pô tính bằng đồng điều bền vững vào VM-UNetV2. Hai nhận xét rút ra: hình

dạng và đường biên là manh mối quan trọng [27], [28]; và gần như tất cả là mạng dạng U cho phân đoạn ngữ nghĩa.

1.3.5. Mamba trong họ YOLO

- **Mamba YOLO** [29] thay phần lớn backbone YOLO bằng khối SSM (ODSS), thử trên COCO và PASCAL VOC cho phát hiện vật thể.
- **MambaPSA** [30] thay đúng khối C2PSA ở cuối backbone YOLO26 cỡ *nano* bằng khối Mamba nhẹ và thêm mô-đun BiViM ở neck, thử trên PASCAL VOC. Theo bài báo, toàn mô hình giảm 2,9% số tham số, giảm 12,1% FLOPs và tăng tốc độ trên CPU, mAP50:95 gần như giữ nguyên (giảm 0,1 điểm). Bài là bản tiền ấn phẩm, kết quả từ **một seed**, và tác giả lưu ý dao động giữa các seed có thể ngang mức chênh lệch được báo cáo.

1.4. TỔNG HỢP VÀ KHOẢNG TRỐNG NGHIÊN CỨU

Bảng 1.2: Tổng hợp các nhóm phương pháp phân đoạn polyp

Nhóm	**Ví dụ**	**Điểm mạnh**	**Hạn chế**
CNN dạng U	U-Net [10], PraNet [9]	Bám chi tiết cục bộ và đường biên tốt; ít dữ liệu vẫn học được	Vùng nhìn của mỗi lớp nhỏ, ngữ cảnh toàn cục hạn chế
Transformer, mô hình lai	Polyp-PVT [16], ColonFormer [17]	Có ngữ cảnh toàn cục ngay trong một lớp	Chi phí tăng theo bình phương số vị trí; cần nhiều dữ liệu hoặc tiền huấn luyện
YOLO một giai đoạn	YOLO [18], YOLO26 [6]	Nhanh; cho cả hộp bao, độ tin cậy và mặt nạ	Ít được nghiên cứu cho phân đoạn polyp; mặt nạ có độ phân giải thấp hơn ảnh
Mamba dạng U	U-Mamba [24], Polyp-Mamba [26], PolyMamba [27], Topo-VM-UNetV2 [28]	Ngữ cảnh toàn cục với chi phí thấp hơn attention; một số có thêm thông tin biên, tô-pô	Chủ yếu là phân đoạn ngữ nghĩa; quét theo thứ tự cố định

Nhóm	Ví dụ	Điểm mạnh	Hạn chế
Mamba trong YOLO	Mamba YOLO [29], MambaPSA [30]	Thêm ngữ cảnh toàn cục vào mô hình nhanh	Đến nay chỉ thử cho phát hiện vật thể trên dữ liệu tổng quát

Bảng 1.3: So sánh đề tài với các công trình gần nhất

Tiêu chí	Mamba YOLO [29]	MambaPSA [30]	Topo-VM-UNetV2 [28]	Đề tài
Mô hình nền	YOLO	YOLO26 cỡ *nano*	VM-UNetV2 (dạng U)	**YOLO26s-seg**
Bài toán	Phát hiện vật thể	Phát hiện vật thể	Phân đoạn ngữ nghĩa	**Phân đoạn thực thể**
Dữ liệu	COCO, VOC	PASCAL VOC	Ảnh polyp	**Ảnh polyp (Kvasir-SEG + ảnh nền)**
Phạm vi thay đổi	Thay phần lớn backbone	Thay C2PSA, thêm khối ở neck	Thêm bản đồ chú ý tô-pô	**Chỉ thay C2PSA ở tầng 10**
Khối Mamba	Khối SSM riêng	Mamba nhẹ, BiViM hai chiều	VMamba	**SS2D bốn hướng + nhánh hình dạng có hướng + cổng**
Cách đánh giá	Báo một kết quả, không nêu số lần chạy hay độ lệch chuẩn	Một seed (tác giả tự nêu là hạn chế)	5 seed, lấy trung bình	**20 seed ghép cặp, có kiểm định thống kê**

Khoảng trống mà đề tài hướng tới gồm ba điểm:

1. Theo khảo sát của nhóm đến tháng 10/2026, chưa thấy công trình nào đưa khối VMamba vào **YOLO26-seg** cho **phân đoạn thực thể polyp**.
2. Các công trình đưa Mamba vào YOLO chưa kết hợp thông tin **hình dạng và đường biên**.
3. Các công trình gần nhất chỉ báo cáo một lần huấn luyện [30] hoặc lấy trung bình vài seed mà không kiểm định ghép cặp [28], trong khi với dữ liệu nhỏ,

dao động giữa các lần huấn luyện có thể lớn hơn chênh lệch giữa hai mô hình [31].

Đề tài không nhắm tới vượt các mô hình tốt nhất hiện nay, mà trả lời câu hỏi có kiểm soát: **chỉ thay một tầng của YOLO26s-seg bằng khối VMamba có nhận biết hình dạng thì kết quả thay đổi thế nào, với độ tin cậy thống kê ra sao, và tốn thêm bao nhiêu chi phí**. Khác với MambaPSA, khối đề xuất **tăng** số tham số của tầng 10 (khoảng 1,81 triệu so với 0,99 triệu của C2PSA) và làm suy luận trên CPU **chậm hơn**.

1.5. TÌNH HÌNH NGHIÊN CỨU TRONG NƯỚC

Nhóm nghiên cứu của Trường Đại học Bách khoa Hà Nội phối hợp với Viện Nghiên cứu và Đào tạo Tiêu hóa, Gan mật có nhiều công trình về polyp: ColonFormer [17] (mục 1.3.2) và **NeoUNet** [32], vừa phân đoạn polyp vừa phân biệt polyp tân sinh với không tân sinh, kèm bộ dữ liệu NeoPolyp (phần công khai BKAI-IGH NeoPolyp-Small có 1.200 ảnh chụp bằng ánh sáng trắng và FICE). Đào Việt Hằng và cộng sự [33] đánh giá YOLOv8 phát hiện polyp trên 30 video nội soi thu tại Hà Nội: mô hình phát hiện được 65/68 polyp nhưng còn nhiều báo nhầm theo từng khung hình (Precision 0,48 ở ngưỡng IoU 0,5), và chỉ dừng ở phát hiện bằng hộp bao. Trong phạm vi khảo sát (arXiv, Crossref và kho tài liệu của một số trường), chưa thấy công trình trong nước nào đưa khối Mamba vào mô hình họ YOLO cho bài toán polyp.

1.6. KẾT CHƯƠNG

Chương 1 đã xác định khoảng trống: chưa có công trình đưa VMamba kèm thông tin hình dạng vào YOLO26-seg cho phân đoạn polyp, với cách đánh giá lặp lại nhiều seed. Chương 2 trình bày cơ sở lý thuyết cần thiết cho khối đề xuất và cách đánh giá.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

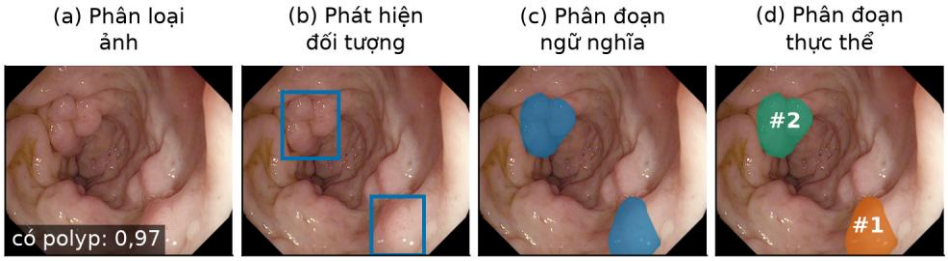
Chương này trình bày kiến thức nền cho Chương 3 và Chương 6: các bài toán thị giác, mạng tích chập, họ YOLO và YOLO26-seg, cơ chế chú ý, mô hình không gian trạng thái (Mamba, VMamba), các độ đo và phương pháp so sánh hai mô hình qua nhiều lần huấn luyện.

2.1. CÁC BÀI TOÁN THỊ GIÁC MÁY TÍNH

Bảng 2.1: So sánh ba bài toán thị giác cơ bản

Bài toán	**Câu hỏi trả lời**	**Đầu ra**	**Ví dụ với ảnh nội soi**
Phân loại ảnh	Ảnh có gì?	Một nhãn (hoặc xác suất) cho cả ảnh	"Ảnh có polyp" với xác suất 0,97
Phát hiện đối tượng	Đối tượng ở đâu?	Danh sách hộp bao, mỗi hộp có lớp và độ tin cậy	Hộp (x_1, y_1, x_2, y_2) quanh polyp, độ tin cậy 0,91
Phân đoạn ảnh	Điểm ảnh nào thuộc đối tượng?	Mặt nạ cùng kích thước ảnh	Các điểm ảnh thuộc polyp mang giá trị 1

Phân đoạn ngữ nghĩa gán mỗi điểm ảnh vào một lớp, không phân biệt các đối tượng cùng lớp: ảnh có hai polyp chỉ cho một mặt nạ chung (U-Net [10], PraNet [9]). **Phân đoạn thực thể** tách riêng từng đối tượng, mỗi polyp có hộp bao, độ tin cậy và mặt nạ riêng (YOLO26-seg). Hình 2.1 minh họa bốn kiểu đầu ra trên cùng một ảnh.



Hình 2.1: Bốn kiểu đầu ra trên cùng một ảnh tập kiểm định có hai polyp: (a) phân loại ảnh, (b) phát hiện đối tượng bằng hộp bao, (c) phân đoạn ngữ nghĩa (hai polyp chung một mặt nạ), (d) phân đoạn thực thể (mỗi polyp một mặt nạ riêng). Hộp bao và mặt nạ vẽ từ nhãn đa giác của ảnh; xác suất ở ô (a) là số minh họa, không phải đầu ra mô hình.

Nguồn: ảnh từ Kvasir-SEG [5]; hình do nhóm vẽ.

Đề tài chọn phân đoạn thực thể vì đầu ra vừa có mặt nạ để quan sát đường biên, vừa có hộp bao và độ tin cậy để hiển thị trong ứng dụng.

2.2. MẠNG NƠ-RON TÍCH CHẬP

Phép tích chập trượt một bộ lọc nhỏ trên ảnh; tại mỗi vị trí, đầu ra là tổng các tích giữa bộ lọc và vùng ảnh bên dưới:

$$y(i,j) = \sum_u \sum_v w\,(u,v)\, x(i+u,\ j+v) + b \tag{2.1}$$

Ví dụ. Với vùng ảnh $x = \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix}$ và bộ lọc biên dọc $w = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$, đầu ra tại tâm là $3 \times (-10 + 80) = 210$, báo hiệu biên dọc; ở vùng đồng nhất đầu ra bằng 0. Trong CNN, các bộ lọc được học từ dữ liệu.

Tích chập phù hợp với ảnh nhờ kết nối cục bộ, chia sẻ trọng số (lớp $k \times k$ từ C_{in} sang C_{out} kênh có $k^2 C_{\text{in}} C_{\text{out}}$ trọng số, không phụ thuộc kích thước ảnh) và đồng biến tịnh tiến. Đề tài dùng thêm **tích chập 1×1** để đổi số kênh và **tích chập theo chiều sâu** (một bộ lọc riêng cho từng kênh, rất ít tham số). Các thành phần đi kèm gồm chuẩn hóa theo lô (BN), hàm kích hoạt $\text{SiLU}(x) = x \cdot \sigma(x)$ (khối `Conv` của YOLO gồm tích chập, BN, SiLU) và kết nối tắt $y = x + F(x)$. Hạn chế chính là **trường tiếp nhận**: xếp chồng tích chập 3×3 chỉ mở rộng dần vùng nhìn, nên CNN thuần khó nắm ngữ cảnh toàn ảnh. Đây là động cơ để thêm attention hoặc SSM vào mạng.

2.3. HỌ MÔ HÌNH YOLO

2.3.1. Ý tưởng phát hiện một giai đoạn

Mô hình hai giai đoạn như Mask R-CNN [34] đề xuất vùng trước rồi phân loại từng vùng; YOLO [18] gộp làm một: ảnh đi qua mạng một lần, mỗi ô trên bản đồ đặc trưng dự đoán trực tiếp hộp bao, lớp và độ tin cậy. Một mô hình YOLO hiện đại gồm:

- **Backbone:** trích đặc trưng qua các mức P1–P5, mức Pk bằng $1/2^k$ ảnh vào (ảnh 640×640 thì P5 là 20×20).
- **Neck:** FPN [35] truyền ngữ nghĩa từ mức sâu xuống mức nông, PAN [36] truyền chi tiết vị trí theo chiều ngược lại.
- **Head:** dự đoán tại P3, P4, P5, ứng với đối tượng nhỏ, vừa, lớn.

2.3.2. Gán nhãn, hàm mất mát và bỏ NMS

YOLO26 dùng bộ gán nhãn căn chỉnh nhiệm vụ của TOOD [37]: một điểm dự đoán được chọn khi vừa có điểm lớp cao vừa có hộp trùng tốt với hộp thật. Hộp được học bằng mất mát CIoU [38], mở rộng IoU bằng hạng tử phạt khoảng cách tâm và độ lệch tỉ lệ khung. Để bỏ bước lọc trùng NMS, YOLOv10 [20] huấn luyện song song **đầu một-nhiều** (mỗi đối tượng gán cho nhiều điểm, nhiều tín hiệu học) và **đầu một-một** (mỗi đối tượng đúng một điểm, đầu ra không trùng); khi suy luận chỉ dùng đầu một-một. YOLO26 kế thừa cơ chế này.

2.3.3. Phân đoạn thực thể bằng mặt nạ mẫu

Theo YOLACT [19], nhánh Proto sinh K **mặt nạ mẫu** $P_1, \dots, P_K$ dùng chung cho cả ảnh; với mỗi đối tượng, đầu dự đoán sinh K **hệ số** $c_1, \dots, c_K$, và mặt nạ là tổ hợp tuyến tính qua sigmoid, sau đó cắt theo hộp bao:

$$\widehat{M} = \sigma\left(\sum_{k=1}^{K} c_k \, P_k\right) \tag{2.2}$$

Ví dụ với $K = 2$ trên lưới 2×2: $P_1 = \begin{bmatrix} 4 & 4 \\ -4 & -4 \end{bmatrix}$ (nửa trên), $P_2 = \begin{bmatrix} 4 & -4 \\ 4 & -4 \end{bmatrix}$ (nửa trái). Đối tượng có hệ số $(1,\ 1)$ cho tổng $\begin{bmatrix} 8 & 0 \\ 0 & -8 \end{bmatrix}$, qua sigmoid và ngưỡng "lớn hơn 0,5" chỉ ô trên–trái thuộc đối tượng; đối tượng có hệ số $(-1,\ 0)$ cho tổng $\begin{bmatrix} -4 & -4 \\ 4 & 4 \end{bmatrix}$, tức nửa dưới. Chỉ với vài hệ số, mỗi đối tượng có mặt nạ riêng. YOLO26-seg dùng $K = 32$ mặt nạ mẫu kích thước 160×160.

2.4. MÔ HÌNH YOLO26-SEG

2.4.1. Kiến trúc

YOLO26 [6] được Ultralytics công bố năm 2026 (bản tiền ấn phẩm). Đề tài dùng cỡ *small* (YOLO26s-seg), hệ số độ sâu và độ rộng 0,5.

Bảng 2.2: Các khối chính của YOLO26s-seg (ảnh vào 640×640)

Phần	Tầng	Khối	Vai trò	Đầu ra
Backbone	0–1	Conv 3×3, bước 2	Giảm kích thước nhanh	64×160×160
	2–8	C3k2 xen kẽ Conv bước 2	Trích đặc trưng ở P3, P4, P5	512×20×20
	9	SPPF	Gộp đặc trưng nhiều tỉ lệ bằng max-pooling lặp	512×20×20

Phần	Tầng	Khối	Vai trò	Đầu ra
	10	**C2PSA**	**Chú ý toàn cục trên P5. Đây là tầng mà đề tài thay thế**	512×20×20
Neck	11–22	Upsample, Concat, C3k2	FPN đi xuống rồi PAN đi lên; riêng C3k2 ở tầng 22 có thêm khối PSA	P3 128×80×80, P4 256×40×40, P5 512×20×20
Head	23	Segment26	Hộp, lớp, 32 hệ số mặt nạ, Proto 32×160×160	Tối đa 300 đối tượng

C3k2 là khối tích chập kiểu CSP (chia kênh làm hai, một phần đi qua các khối nút cổ chai rồi ghép lại); SPPF áp dụng max-pooling 5×5 ba lần nối tiếp và ghép kết quả. **C2PSA** có cùng khuôn tách–xử lý–ghép, phần xử lý là khối PSA [20] gồm attention đa đầu và mạng truyền thẳng; ở cỡ *s*, C2PSA có 990.976 tham số.

2.4.2. Các bước của thuật toán YOLO26-seg

Suy luận:

1. **Tiền xử lý:** co ảnh cho cạnh dài bằng 640, giữ tỉ lệ, thêm viền thành bội số của 32 (letterbox), chia giá trị điểm ảnh cho 255.
2. **Backbone và neck:** trích đặc trưng qua tầng 0–10, trộn ba mức P3, P4, P5 theo FPN rồi PAN.
3. **Head:** tại mỗi điểm, đầu một-một dự đoán bốn khoảng cách tới cạnh hộp, điểm lớp và 32 hệ số mặt nạ; nhánh Proto26 sinh 32 mặt nạ mẫu.
4. **Chọn đối tượng:** lấy tối đa 300 dự đoán điểm cao nhất, bỏ dự đoán dưới ngưỡng tin cậy; không có NMS.
5. **Tạo mặt nạ:** tính theo (2.2), phóng về kích thước ảnh, nhị phân hóa ở 0,5, cắt theo hộp; trả về hộp, độ tin cậy, mặt nạ của từng đối tượng.

Huấn luyện mỗi lô ảnh:

1. **Tăng cường dữ liệu:** ghép bốn ảnh (Mosaic) [39], lật, co giãn, đổi màu; Mosaic tắt ở 10 epoch cuối.
2. **Lan truyền xuôi** qua cả hai đầu; đặc trưng vào đầu một-một được tách khỏi đồ thị gradient.

3. **Gán nhãn:** đầu một-nhiều lấy 10 điểm tốt nhất cho mỗi đối tượng, đầu một-một giữ 1 điểm; YOLO26 thêm quy tắc STAL nới kích thước vật rất nhỏ khi chọn ứng viên [6].
4. **Tính mất mát** hộp (CIoU và L1), lớp, mặt nạ và ngữ nghĩa phụ cho hai đầu; trọng số đầu một-nhiều giảm tuyến tính từ 0,8 xuống 0,1.
5. **Cập nhật trọng số** bằng AdamW [40] và cập nhật bản trung bình động (EMA); cuối mỗi epoch đánh giá trên tập kiểm định và lưu trọng số tốt nhất.

YOLO26 bỏ thành phần DFL của các phiên bản trước, hồi quy trực tiếp bốn khoảng cách kèm mất mát L1 [6]. **Ví dụ giải mã hộp:** điểm ở ô (cột 10, hàng 12) của P3 (bước 8 pixel) có tâm $((10 + 0{,}5) \times 8,\ (12 + 0{,}5) \times 8) = (84,\ 100)$; nếu bốn khoảng cách tới cạnh trái, trên, phải, dưới là $(2,\ 1,\ 3,\ 4)$ ô, tức $(16,\ 8,\ 24,\ 32)$ pixel, thì hộp bao là $(68,\ 92,\ 108,\ 132)$.

2.4.3. Ưu và nhược điểm

Bảng 2.3: Ưu và nhược điểm của YOLO26-seg với bài toán phân đoạn polyp

Ưu điểm	**Nhược điểm**
Một lần suy luận cho cả hộp bao, độ tin cậy và mặt nạ	Mặt nạ mẫu chỉ có độ phân giải 160×160, có thể làm mất chi tiết biên của polyp nhỏ
Không cần NMS nên thời gian suy luận ổn định, dễ triển khai	Thiết kế cho dữ liệu tổng quát (COCO), chưa có thành phần riêng cho biên mờ
Có trọng số tiền huấn luyện trên COCO [41], giúp học nhanh với dữ liệu nhỏ	Attention chỉ có ở mức P5 (C2PSA ở tầng 10 và khối PSA trong tầng 22); các mức P3, P4 chỉ có tích chập
Thư viện Ultralytics [42] hoàn chỉnh: huấn luyện, đánh giá, xuất mô hình	Bài báo YOLO26 là bản tiền ấn phẩm, một số chi tiết chỉ có trong mã nguồn
Có nhiều cỡ mô hình (n, s, m, l, x) để cân đối tốc độ và độ chính xác	Mặt nạ bị cắt theo hộp, nên hộp sai thì mặt nạ cũng sai

2.5. CƠ CHẾ CHÚ Ý VÀ CHI PHÍ TÍNH TOÁN

Cơ chế tự chú ý của Transformer [43] tính từ chuỗi N vị trí ba ma trận truy vấn Q, khóa K, giá trị V, rồi:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V \tag{2.3}$$

Ma trận $QK^{\top}$ kích thước $N \times N$ cho biết mỗi vị trí chú ý đến vị trí khác bao nhiêu, nên chi phí tăng theo bình phương số vị trí: ở P5 (20×20) có $N^2 = 160.000$

cặp, nhưng ở P3 (80×80) đã khoảng 41 triệu cặp, gấp 256 lần. Vì vậy YOLO chỉ đặt attention ở P5. Mô hình không gian trạng thái là một hướng giữ tính toàn cục với chi phí tăng chậm hơn.

2.6. MÔ HÌNH KHÔNG GIAN TRẠNG THÁI, MAMBA VÀ VMAMBA

2.6.1. Mô hình không gian trạng thái và rời rạc hóa

Mô hình không gian trạng thái (SSM) mô tả hệ có trạng thái ẩn $h(t)$, tín hiệu vào $x(t)$ và ra $y(t)$:

$$h'(t) = A\,h(t) + B\,x(t), \qquad y(t) = C\,h(t) \tag{2.4}$$

A quyết định cách trạng thái nhớ và quên, B cách đầu vào ghi vào trạng thái, C cách đọc đầu ra. S4 [22] dùng A dạng chéo cộng hạng thấp; S4D [44] đơn giản thành ma trận chéo phần tử âm, được Mamba và VMamba dùng. Với chuỗi rời rạc và bước thời gian Δ, phép rời rạc hóa giữ nguyên trong mỗi bước (zero-order hold) cho:

$$\bar{A} = \exp(\Delta A), \qquad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\,\Delta B \approx \Delta B \tag{2.5}$$

$$h_t = \bar{A}\,h_{t-1} + \bar{B}\,x_t, \qquad y_t = C\,h_t \tag{2.6}$$

Mamba [23] và bản cài đặt của đề tài dùng xấp xỉ Euler $\bar{B} \approx \Delta B$. Vì A âm nên $0 < \bar{A} < 1$: trạng thái mới bằng trạng thái cũ đã suy giảm cộng phần đầu vào mới; Δ lớn thì quên nhanh quá khứ. Ví dụ với $A = -1$, $B = C = 1$, $\Delta = 0{,}5$, chuỗi vào $(1,\ 0,\ 2)$: $\bar{A} \approx 0{,}607$, các trạng thái lần lượt là 0,500; 0,303; 1,184, tức thông tin cũ mờ dần khi không có đầu vào mới.

2.6.2. Mamba: SSM có chọn lọc

Mamba [23] cho Δ, B, C **phụ thuộc vào đầu vào** (tính từ x_t qua phép chiếu, $\Delta_t = \text{softplus}(\cdot) > 0$), nên mô hình tự quyết định giữ hay bỏ thông tin ở từng vị trí. Tiếp ví dụ ở mục 2.6.1: nếu ở bước 3 mô hình chọn $\Delta_3 = 2$ thì $\bar{A}_3 = e^{-2} \approx 0{,}135$, phần quá khứ gần như bị xóa; theo công thức ZOH, trọng số của đầu vào mới là $1 - e^{-2} \approx 0{,}865$, lớn hơn hẳn trường hợp $\Delta = 0{,}5$ (0,393), tức mô hình đã "chọn" tập trung vào đầu vào hiện tại. Khi đó phép truy hồi (2.6) phải tính tuần tự hoặc bằng **thuật toán quét song song** [45], [46]: mỗi bước có dạng $h_t = a_t h_{t-1} + b_t$, và hai bước liên tiếp gộp được thành một bước cùng dạng

$$(a_1, b_1) \circ (a_2, b_2) = (a_2 a_1,\ a_2 b_1 + b_2) \tag{2.7}$$

Phép gộp có tính kết hợp nên gộp song song theo cây trong khoảng $\log_2 L$ vòng. Mamba gốc có kernel CUDA với chi phí tuyến tính; đề tài dùng biến thể Hillis–Steele [46] viết bằng PyTorch thuần, tổng công việc $O(L\log L)$.

2.6.3. VMamba và mô-đun SS2D

Ảnh không có thứ tự tự nhiên; trải ảnh theo một hướng thì mỗi vị trí chỉ thấy các vị trí đứng trước. VMamba [7] đề xuất **SS2D** với các bước trên bản đồ $C \times H \times W$:

1. **Chiếu vào:** tích chập 1×1 đưa C kênh lên $2EC$ kênh, tách thành nhánh chính U và nhánh cổng Z (E là hệ số mở rộng).
2. **Trộn cục bộ:** U qua tích chập theo chiều sâu 3×3 và SiLU.
3. **Trải thành bốn chuỗi** dài $L = H \times W$: theo hàng, theo hàng đảo ngược, theo cột, theo cột đảo ngược.
4. **Quét chọn lọc:** với mỗi chuỗi, tính Δ_t, B_t, C_t, rời rạc hóa theo (2.5) và chạy truy hồi (2.6).
5. **Gộp:** đưa bốn chuỗi về lưới $H \times W$ và cộng lại.
6. **Cổng và chiếu ra:** nhân với SiLU(Z), chiếu 1×1 về C kênh.

Ví dụ trên lưới 2×2 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$: bốn chuỗi là a, b, c, d; d, c, b, a; a, c, b, d; d, b, c, a. Ô b nhận thông tin từ a ở chuỗi thứ nhất, từ d, c ở chuỗi thứ hai, từ a, c ở chuỗi thứ ba và từ d ở chuỗi thứ tư; sau khi cộng, b đã nhận thông tin từ cả ba ô còn lại, điều mà một lần quét không làm được. Bản SS2D của đề tài là bản rút gọn ($E = 1$, không có hạng tử nhảy D và LayerNorm đầu ra), trình bày ở Chương 3.

2.6.4. Ưu và nhược điểm của VMamba

Bảng 2.4: Ưu và nhược điểm của VMamba

Ưu điểm	**Nhược điểm**
Mỗi vị trí nhận thông tin toàn ảnh sau một khối	Thứ tự quét cố định, không tự nhiên với ảnh; hai điểm gần nhau theo chiều dọc có thể cách xa nhau trong chuỗi quét hàng
Chi phí tăng gần tuyến tính theo số vị trí (với kernel tối ưu), rẻ hơn attention khi ảnh lớn	Ở bản đồ nhỏ như P5 ($L = 400$), lợi thế chi phí so với attention chưa rõ
Cơ chế chọn lọc cho phép giữ hay quên thông tin theo nội dung	Kernel tối ưu cần CUDA; bản PyTorch thuần chậm hơn nhiều

Ưu điểm	Nhược điểm
Có nhiều kết quả tốt trên ảnh y tế [24]–[26]	Huấn luyện từ đầu khó hơn; trọng số tiền huấn luyện ảnh hưởng lớn [25]
Không có ma trận $N \times N$ nên tiết kiệm bộ nhớ	Một số chi tiết (khởi tạo Δ, chuẩn hóa đầu ra) nhạy với độ ổn định số

2.7. CÁC ĐỘ ĐO ĐÁNH GIÁ

2.7.1. Các đại lượng cơ bản

So sánh dự đoán với nhãn thật cho TP (dự đoán polyp, đúng là polyp), FP (dự đoán polyp nhưng không phải), FN (polyp bị bỏ sót) và TN (dự đoán nền, đúng là nền):

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}} \tag{2.8}$$

Trong nội soi, Recall thấp nghĩa là bỏ sót polyp. Các đại lượng tính được ở **mức điểm ảnh** (mỗi điểm ảnh là một mẫu, dùng khi so sánh mặt nạ) hoặc **mức đối tượng** (một dự đoán là TP khi IoU với polyp thật vượt ngưỡng; ở mức này TN không xác định được).

2.7.2. IoU và Dice

Với G là tập điểm ảnh của nhãn thật và P là tập điểm ảnh dự đoán:

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{TP}{TP+FP+FN}, \qquad \text{Dice} = \frac{2|P \cap G|}{|P|+|G|} = \frac{2\,TP}{2\,TP+FP+FN} \tag{2.9}$$

Hai chỉ số liên hệ qua $\text{Dice} = 2\,\text{IoU}/(1+\text{IoU})$; ở mức điểm ảnh, Dice trùng với F1.

Ví dụ trên lưới 4×4 (1 là polyp):

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \qquad P = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Có $TP = 7$, $FP = 1$, $FN = 2$, nên Precision = 7/8 = 0,875; Recall = 7/9 = 0,778; IoU = 7/10 = 0,700; Dice = 14/17 = 0,824 (khớp với $2 \times 0{,}7/1{,}7$). Khi đánh giá cả tập có thể lấy **trung bình theo ảnh** hoặc **gộp toàn tập** (cộng TP, FP, FN rồi mới tính); Chương 6 ghi rõ cách dùng.

2.7.3. AP, mAP50 và mAP50–95

Sắp xếp các dự đoán theo độ tin cậy giảm dần, tính Precision và Recall tích lũy được đường Precision–Recall; **AP** là diện tích dưới đường này sau khi làm đơn điệu [47]. Ví dụ tập có 3 polyp, 4 dự đoán xếp theo độ tin cậy là TP, FP, TP, TP: Precision đơn điệu bằng 1,0 trên đoạn Recall 0–1/3 và 0,75 trên đoạn 1/3–1, nên $AP = 1/3 \times 1{,}0 + 2/3 \times 0{,}75 \approx 0{,}833$.

– **mAP50:** AP tại ngưỡng IoU 0,5 (đề tài một lớp nên bằng AP50).

– **mAP50–95:** trung bình AP tại 10 ngưỡng IoU từ 0,50 đến 0,95, bước 0,05 [41]. Ở ngưỡng cao chỉ mặt nạ bám sát biên mới được tính đúng, nên chỉ số này phản ánh độ chính xác đường biên tốt hơn mAP50.

Khi IoU tính giữa hai **mặt nạ**, chỉ số gọi là *mask* mAP. Đề tài lấy **mask mAP50–95** làm độ đo chính.

2.7.4. Chi phí tính toán

– **Số tham số:** tích chập $k \times k$ từ C_{in} sang C_{out} kênh có $k^2 C_{\text{in}} C_{\text{out}}$ trọng số; ví dụ tầng đầu của YOLO26s-seg (3×3, 3 sang 32 kênh) có 864 trọng số.

– **FLOPs:** số phép tính dấu phẩy động cho một ảnh, một phép nhân–cộng tính là 2 FLOPs; tích chập trên lưới $H \times W$ cần khoảng $2k^2 C_{\text{in}} C_{\text{out}} HW$ FLOPs.

– **Thời gian suy luận:** đo bằng mili giây, phụ thuộc phần cứng; phân biệt thời gian chỉ chạy mạng với thời gian toàn quy trình.

FLOPs không phản ánh đầy đủ tốc độ thực tế: một khối ít FLOPs nhưng gồm nhiều phép tính tuần tự, như phép quét SS2D, vẫn có thể chạy chậm.

2.8. SO SÁNH HAI MÔ HÌNH QUA NHIỀU LẦN HUẤN LUYỆN

2.8.1. Vì sao cần nhiều seed

Khởi tạo trọng số, thứ tự đưa ảnh và tăng cường dữ liệu đều ngẫu nhiên, được điều khiển bằng **seed**; đổi seed có thể cho kết quả khác nhau đáng kể [31]. Đề tài huấn luyện mỗi mô hình với 20 seed giống nhau và **ghép cặp** theo seed: $d_i = \text{TSVM}_i - \text{Baseline}_i$, nhờ đó loại bỏ phần dao động chung ở cùng seed.

2.8.2. Kiểm định t ghép cặp

Với n chênh lệch có trung bình $\bar{d}$ và độ lệch chuẩn s_d:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} \tag{2.10}$$

Dưới giả thuyết không (kỳ vọng của d bằng 0), t có phân phối Student với $n-1$ bậc tự do [48]; $p < 0{,}05$ nghĩa là chênh lệch có ý nghĩa thống kê ở mức 5%. Khoảng tin cậy 95% của chênh lệch trung bình là

$$\bar{d} \pm t_{0{,}975;\, n-1} \cdot \frac{s_d}{\sqrt{n}} \tag{2.11}$$

và nếu khoảng này chứa 0 thì chưa đủ cơ sở khẳng định mô hình nào tốt hơn.

2.8.3. Kiểm định Wilcoxon và cỡ hiệu ứng

Kiểm định Wilcoxon dấu–hạng [49] chỉ dùng thứ hạng của $|d_i|$ và dấu, không cần giả định phân phối chuẩn; Demšar [50] khuyến nghị dùng kiểm định này khi so sánh hai mô hình học máy. Giá trị p không cho biết chênh lệch lớn hay nhỏ, nên đề tài báo thêm **cỡ hiệu ứng** Cohen d_z cho dữ liệu ghép cặp:

$$d_z = \frac{\bar{d}}{s_d} \tag{2.12}$$

Theo quy ước của Cohen (được Lakens [51] trình bày lại), d_z khoảng 0,2 là nhỏ, 0,5 là vừa, 0,8 là lớn.

Ví dụ (số tự đặt). Năm seed cho chênh lệch $0{,}010;\ -0{,}004;\ 0{,}006;\ 0{,}002;\ 0{,}008$: $\bar{d} = 0{,}0044$, $s_d \approx 0{,}0055$, $t \approx 1{,}77$ ($p \approx 0{,}15$), Wilcoxon $W = 2$ ($p \approx 0{,}19$), khoảng tin cậy 95% là $[-0{,}0025;\ 0{,}0113]$ chứa 0, trong khi $d_z \approx 0{,}79$. Dù trung bình dương và cỡ hiệu ứng không nhỏ, **với ít cặp và dao động lớn giữa các seed, kết quả vẫn chưa có ý nghĩa thống kê**. Chương 6 dùng đúng các công cụ này.

2.9. KẾT CHƯƠNG

Chương 2 đã trình bày các thành phần của YOLO26-seg (hai đầu dự đoán bỏ NMS, mặt nạ mẫu, các bước suy luận và huấn luyện), cơ chế chú ý và chi phí bậc hai, SSM, Mamba và mô-đun SS2D của VMamba, các độ đo và phương pháp so sánh ghép cặp qua nhiều seed. Chương 3 dựa trên các kiến thức này để trình bày khối C2TSVMamba.

CHƯƠNG 3: MÔ HÌNH ĐỀ XUẤT

Mô hình đề xuất giữ nguyên gần như toàn bộ YOLO26s-seg, chỉ thay khối C2PSA ở tầng 10 bằng khối mới C2TSVMamba. Bên trong khối có nhánh SS2D quét đặc trưng theo kiểu VMamba và nhánh hình dạng dùng tích chập có hướng, điều khiển nhánh SS2D qua một cổng. Thông số trong chương đọc từ mã nguồn đã dùng để huấn luyện (fork Ultralytics 8.4.127 [42] cùng mô-đun `topology_shape_vmamba.py`) và được kiểm tra bằng cách dựng mô hình rồi đếm tham số.

3.1. Ý TƯỞNG VÀ ĐỘNG CƠ THIẾT KẾ

Để khoanh đúng polyp có biên mờ, màu gần niêm mạc và kích thước đa dạng, mô hình cần **ngữ cảnh rộng** (vùng đang xét là khối lồi lên hay chỉ là phản quang, nếp gấp) và **hình dạng của vùng** (trải theo hướng nào, đường bao ra sao). Trong YOLO26-seg, ngữ cảnh rộng ở cuối backbone do C2PSA đảm nhận bằng attention [43] trên P5 (20×20); thông tin không gian cục bộ của khối này chỉ đến từ một tích chập theo chiều sâu 3×3 làm mã hóa vị trí. Các công trình gần đây cho thấy biên và hình dạng là manh mối có ích cho polyp [27], [28], còn Mamba [23] và VMamba [7] đưa ra cách khác để thu ngữ cảnh toàn cục. Từ đó, đề tài thay C2PSA bằng một khối kết hợp quét chọn lọc kiểu VMamba với nhánh tích chập có hướng.

Vì sao chọn tầng 10:

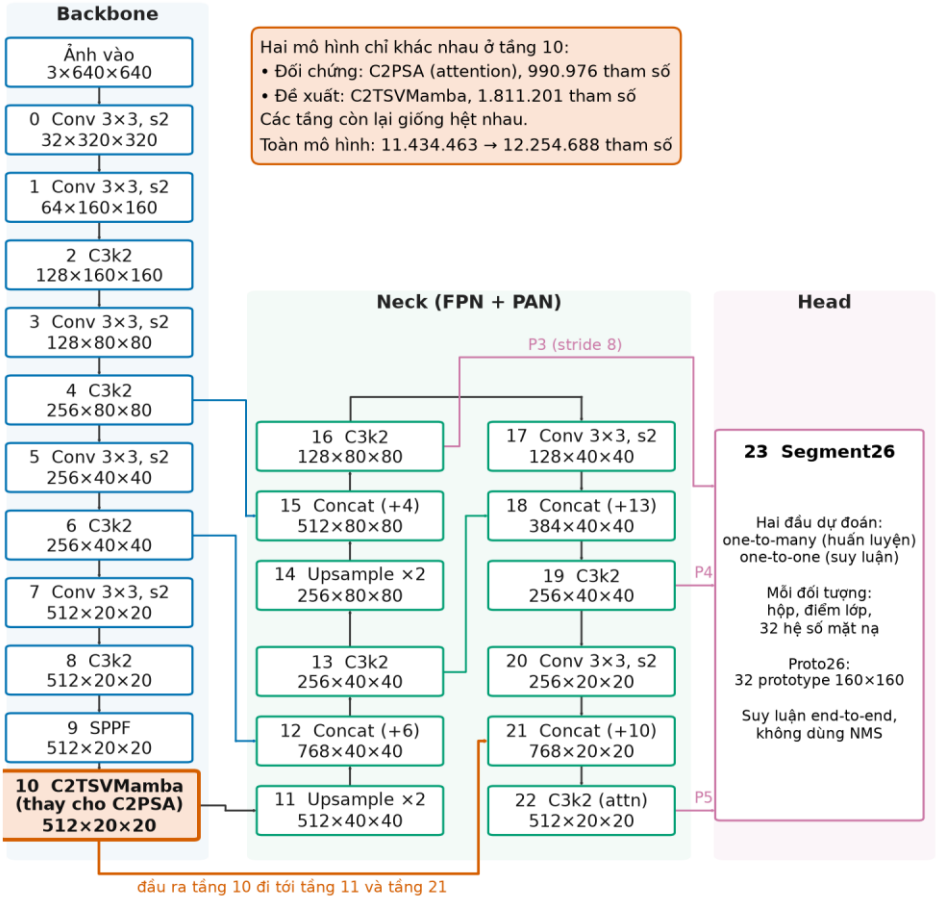
1. Đây là khối attention duy nhất trong backbone, nên hai mô hình chỉ khác nhau ở cách tổng hợp ngữ cảnh tại một vị trí.
2. Chuỗi cần quét ngắn: P5 chỉ có 20×20 = 400 vị trí, mỗi vị trí đã mang thông tin ngữ nghĩa cao.
3. Đầu ra tầng 10 đi vào tầng 11 (phóng to xuống P4, P3) và tầng 21 (nhánh P5 đi lên), nên đặc trưng của khối mới có mặt ở cả ba mức của đầu Segment26.
4. Giao diện vào/ra 512×20×20 giữ nguyên, không phải sửa neck, head hay hàm mất mát.

Phạm vi đóng góp. SS2D là ý tưởng của VMamba [7], quét chọn lọc đến từ Mamba [23], tích chập theo chiều sâu, cổng sigmoid và nối tắt có hệ số là kỹ thuật quen thuộc. Đóng góp của đề tài là thiết kế cách ghép các thành phần này (trong đó nhánh hình dạng điều biến nhánh SS2D qua cổng), cài đặt lại SS2D bằng

PyTorch thuần, tích hợp vào YOLO26s-seg cho phân đoạn polyp và đánh giá trên 20 seed. Khác với MambaPSA [30] (phát hiện vật thể, nhắm giảm chi phí, thêm BiViM ở neck) và Mamba YOLO [29] (thay nhiều khối), đề tài chỉ thay một tầng và bổ sung nhánh hình dạng.

3.2. KIẾN TRÚC TỔNG THỂ

Mô hình gồm backbone (tầng 0–10, giảm từ 640×640 xuống 20×20, tầng 9 là SPPF, tầng 10 là vị trí thay đổi), neck (tầng 11–22, FPN [35] đi xuống và PAN [36] đi lên, cho P3 80×80, P4 40×40, P5 20×20) và đầu Segment26 (tầng 23) dự đoán hộp, điểm lớp, hệ số mặt nạ và sinh 32 mặt nạ mẫu 160×160 theo YOLACT [19] (Hình 3.1).



Hình 3.1: Kiến trúc YOLO26s-seg và vị trí khối C2TSVMamba (tầng 10). Kích thước tensor ghi theo dạng C×H×W, tính cho ảnh vào 640×640 với hệ số quy mô s. Tầng 22 (C3k2, attn=True) có PSABlock ở cả hai mô hình.

Tệp cấu hình của mô hình đề xuất sao từ `yolo26-seg.yaml`, chỉ khác dòng khai báo tầng 10 (Bảng 3.1). Tầng 22 của cả hai mô hình vẫn có PSABlock, nên

mô hình đề xuất chỉ thay attention ở tầng 10. Khi suy luận, cả hai chạy end-to-end: chỉ dùng đầu một-một, lấy tối đa 300 đối tượng, không NMS [20].

Bảng 3.1: So sánh mô hình đối chứng và mô hình đề xuất

Tiêu chí	**YOLO26s-seg (đối chứng)**	**YOLO26s-seg + C2TSVMamba (đề xuất)**
Khối ở tầng 10	C2PSA (1 PSABlock, 4 đầu attention)	C2TSVMamba (1 khối TSVMamba)
Vào/ra tầng 10	512×20×20	512×20×20
Tham số tầng 10	990.976	1.811.201
Tham số toàn mô hình	11.434.463	12.254.688 (+7,17%)
Tầng 0–9 và 11–23	Giống nhau	Giống nhau
Hàm mất mát, dữ liệu, siêu tham số	Giống nhau	Giống nhau

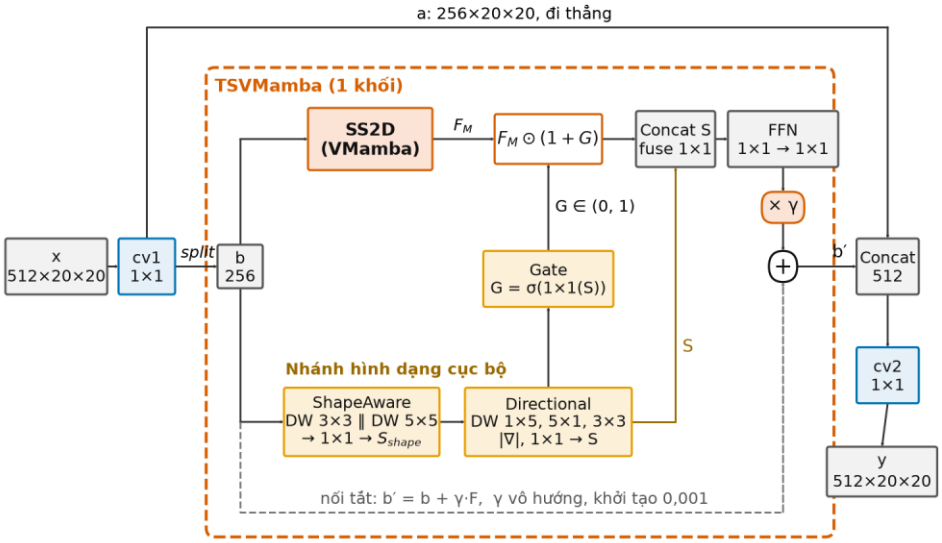
3.3. KHỐI C2TSVMAMBA

3.3.1. Khung tách–xử lý–ghép

C2TSVMamba giữ khuôn ngoài của C2PSA: đặc trưng vào $x \in \mathbb{R}^{512\times20\times20}$ qua tích chập 1×1 (kèm BN, SiLU), chia đôi theo kênh thành a và b (mỗi phần 256 kênh); a đi thẳng, b qua khối TSVMamba; hai phần ghép lại rồi qua một tích chập 1×1 nữa:

$$[a,\ b] = \text{split}(\text{Conv}_{1\times1}^{cv1}(x)), \qquad y = \text{Conv}_{1\times1}^{cv2}([\,a \parallel \text{TSVMamba}(b)\,]) \tag{3.1}$$

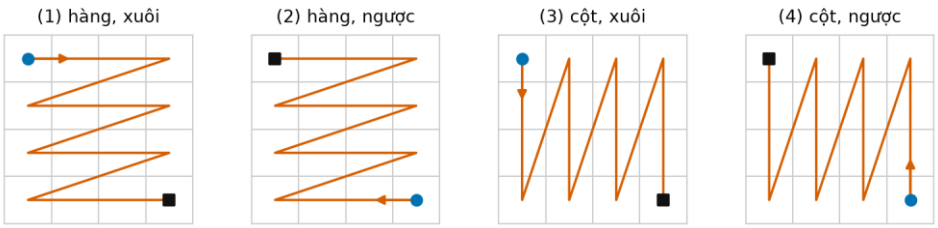
Cách chia đôi giống thiết kế CSP từ YOLOv4 [39], giúp giảm chi phí và giữ đường gradient ngắn. Với quy mô s, số khối TSVMamba thực tế là $\max(\text{round}(2 \times 0{,}5),1) = 1$. Bên trong TSVMamba (Hình 3.2), b đi đồng thời vào nhánh SS2D và nhánh hình dạng; nhánh hình dạng sinh bản đồ cổng G điều biến đầu ra SS2D; hai nhánh được hợp nhất, tinh chỉnh qua FFN, nhân hệ số γ rồi cộng trở lại vào b.



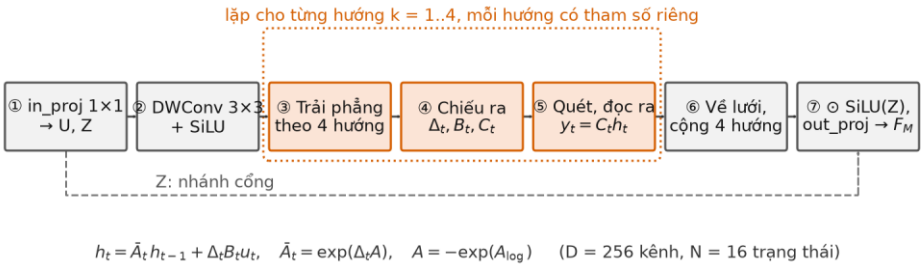
Hình 3.2: Cấu trúc khối C2TSVMamba. Khung nét đứt là khối TSVMamba. Đặc trưng bên trong khối có kích thước 256×20×20. Chi tiết nhánh SS2D ở Hình 3.4.

3.3.2. Nhánh SS2D

Nhánh này cài đặt lại SS2D của VMamba [7]: trải bản đồ đặc trưng thành bốn chuỗi theo bốn thứ tự (Hình 3.3), cho mỗi chuỗi qua SSM chọn lọc rồi gộp lại, để mỗi vị trí nhận thông tin từ mọi phía. Ký hiệu: $D = 256$ kênh, $N = 16$ chiều trạng thái, $L = H \times W = 400$.



Hình 3.3: Bốn thứ tự duyệt của SS2D, minh họa trên lưới 4×4 (thực tế là 20×20, L = 400). Chấm tròn đánh dấu vị trí đầu chuỗi, ô vuông đánh dấu vị trí cuối chuỗi.



Hình 3.4: Các bước bên trong nhánh SS2D như được cài đặt. Bước ③ đến ⑤ được thực hiện riêng cho từng hướng.

Các bước tính (Hình 3.4):

1. **Chiếu vào:** tích chập 1×1 không bias nâng b lên 512 kênh, chia thành U (nội dung sẽ quét) và Z (cổng ở bước cuối), mỗi phần 256 kênh.
2. **Trộn cục bộ:** U qua tích chập theo chiều sâu 3×3 và SiLU được $\widetilde{U}$, như tích chập ngắn trước SSM của Mamba [23].
3. **Trải phẳng bốn hướng:** theo hàng, hàng đảo ngược, cột, cột đảo ngược; gọi Π_k là phép trải thứ k và $u^{(k)} = \Pi_k(\widetilde{U})$.
4. **Sinh tham số phụ thuộc đầu vào** cho mỗi hướng, với Δ qua phép chiếu hạng thấp (hạng 16) rồi mở rộng lên 256 kênh:

$$\Delta_t^{(k)} = \mathrm{softplus}\big(W_\Delta^{(k)}\, W_{x,\Delta}^{(k)}\, u_t^{(k)} + b_\Delta^{(k)}\big) \in \mathbb{R}^D, \quad B_t^{(k)} = W_{x,B}^{(k)}\, u_t^{(k)} \in \mathbb{R}^N, \quad C_t^{(k)} = W_{x,C}^{(k)}\, u_t^{(k)} \in \mathbb{R}^N \qquad (3.2)$$

5. **Rời rạc hóa, quét và đọc ra**, với $A^{(k)} = -\exp(A_{\log}^{(k)})$ dạng chéo luôn âm theo S4D [44]:

$$\bar{A}_{t,d,n}^{(k)} = \exp\Big(\mathrm{clip}\big(\Delta_{t,d}^{(k)}\, A_{d,n}^{(k)},\, -50,\, 0\big)\Big) \qquad (3.3)$$

$$h_{t,d,n}^{(k)} = \bar{A}_{t,d,n}^{(k)}\, h_{t-1,d,n}^{(k)} + \Delta_{t,d}^{(k)}\, B_{t,n}^{(k)}\, u_{t,d}^{(k)}, \qquad h_0^{(k)} = 0 \qquad (3.4)$$

$$y_{t,d}^{(k)} = \textstyle\sum_{n=1}^{N} C_{t,n}^{(k)}\; h_{t,d,n}^{(k)} \qquad (3.5)$$

6. **Gộp bốn hướng:** đảo phép trải phẳng của từng $y^{(k)}$ về lưới 20×20 rồi cộng lại.
7. **Cổng và chiếu ra:**

$$F_M = W_{\text{out}} * \Big(\big(\textstyle\sum_{k=1}^{4} \Pi_k^{-1}\,(y^{(k)})\big) \odot \mathrm{SiLU}(Z)\Big) \in \mathbb{R}^{256\times 20\times 20} \qquad (3.6)$$

Vì $\Delta A \le 0$ nên $\bar{A} \in (0,1]$: trạng thái cũ luôn mờ dần với tốc độ do Δ quyết định. Số hạng đầu vào dùng xấp xỉ $\bar{B} \approx \Delta B$ như Mamba [23]; phép cắt clip chỉ có trong bản cài đặt này để tránh lấy exp của số âm quá lớn; (3.4) được giải bằng phép quét song song (mục 3.4). Mỗi hướng có tham số chiếu và $A_{\log}$ riêng. So với VMamba gốc, bản cài đặt không có hạng tử nhảy $D \cdot u$ (thay bằng nối tắt có hệ số γ ở ngoài khối), không có LayerNorm sau khi gộp, vẫn giữ cổng $\mathrm{SiLU}(Z)$, khởi tạo phép chiếu Δ theo mặc định PyTorch và không dùng kernel CUDA tối ưu.

3.3.3. Nhánh hình dạng

Nhánh hình dạng chỉ gồm tích chập, cung cấp thông tin hình học của vùng đặc trưng: trải theo hướng nào, rộng hẹp ra sao. Ở mức P5, một nhân 1×5 đã phủ khoảng 160 pixel ảnh gốc, nên nhánh này nắm hình dạng ở mức vùng lớn, không phải chi tiết biên từng pixel; đề tài chưa có thí nghiệm riêng kiểm chứng nhánh thực sự học được hình dạng.

ShapeAwareBranch chạy song song hai tích chập theo chiều sâu 3×3 và 5×5 (kèm BN, SiLU), ghép lại và trộn kênh bằng tích chập 1×1:

$$S_{\text{shape}} = \text{Conv}_{1\times1}([\text{DW}_{3\times3}(b) \parallel \text{DW}_{5\times5}(b)]) \quad (3.7)$$

DirectionalShapeExtractor chạy song song ba tích chập theo chiều sâu (dải ngang 1×5, dải dọc 5×1, 3×3), tính thêm độ lớn tổng hợp S_{grad}, rồi trộn hai lần bằng tích chập 1×1:

$$S_h = \text{DW}_{1\times5}(S_{\text{shape}}), \quad S_v = \text{DW}_{5\times1}(S_{\text{shape}}), \quad S_{loc} = \text{DW}_{3\times3}(S_{\text{shape}}), \quad S_{grad} = \sqrt{S_h^2 + S_v^2 + 10^{-6}} \quad (3.8)$$

$$S_{topo} = \text{Conv}_{1\times1}([S_h \parallel S_v \parallel S_{loc} \parallel S_{grad}]), \qquad S = \text{Conv}_{1\times1}([S_{\text{shape}} \parallel S_{topo}]) \quad (3.9)$$

S_{grad} có dạng giống độ lớn gradient trong xử lý ảnh [52], nhưng S_h, S_v là đáp ứng của các nhân học được, nên nó không phải gradient ảnh như bộ lọc Sobel mà là một đặc trưng ít phụ thuộc hướng. Đầu ra S có kích thước 256×20×20.

3.3.4. Cổng

Từ S, một tích chập 1×1 có bias và hàm sigmoid sinh bản đồ cổng G điều biến đầu ra SS2D:

$$G = \sigma(W_g * S + b_g) \in (0,1)^{256\times20\times20}, \qquad F'_M = F_M \odot (1 + G) \quad (3.10)$$

Hệ số $(1 + G)$ luôn trong khoảng (1, 2): cổng chỉ khuếch đại, không tắt hẳn vị trí nào, tránh mất tín hiệu SS2D lúc đầu huấn luyện. Vì sau bước hợp nhất còn một lớp BN, tác dụng thực tế là đổi tầm quan trọng tương đối giữa các vị trí và kênh. Thông tin đi một chiều: nhánh hình dạng điều khiển nhánh SS2D. Chữ "topology" trong tên `TopologyShapeGate` chỉ là ý đồ thiết kế; đề tài không dùng hàm mất mát hay ràng buộc tô-pô nào.

3.3.5. Hợp nhất, mạng truyền thẳng và nối tắt có hệ số

Ghép F'_M với S, dùng tích chập 1×1 (kèm BN, SiLU) đưa 512 về 256 kênh, qua FFN gồm hai tích chập 1×1 (256 → 512 → 256), nhân hệ số γ và cộng vào b:

$$F = \mathrm{FFN}\big(\mathrm{Conv}_{1\times1}^{fuse}([F'_M \parallel S])\big), \qquad b' = b + \gamma F, \qquad \gamma_0 = 10^{-3} \quad (3.11)$$

γ là số vô hướng khởi tạo 0,001, nên lúc đầu khối gần như không đổi đầu vào ($b' \approx b$) và đóng góp tăng dần trong huấn luyện, tránh xáo trộn các tầng đã tiền huấn luyện. Gộp lại:

$$\mathrm{TSVMamba}(b) = b + \gamma \cdot \mathrm{FFN}\Big(\mathrm{Conv}_{1\times1}^{fuse}([\,\mathrm{SS2D}(b) \odot (1 + G(b)) \parallel S(b)\,])\Big) \quad (3.12)$$

3.3.6. Số tham số

Bảng 3.2: Số tham số của tầng 10 (quy mô s)

Thành phần	**Số tham số**	**Tỉ lệ trong tầng 10**
cv1 + cv2 (khung C2)	526.336	29,1%
SS2D	281.856	15,6%
ShapeAwareBranch	141.312	7,8%
DirectionalShapeExtractor	400.640	22,1%
Cổng	65.792	3,6%
fuse	131.584	7,3%
FFN	263.680	14,6%
γ	1	~0%
Tổng C2TSVMamba	**1.811.201**	100%
C2PSA của mô hình đối chứng	990.976	

Nhánh hình dạng cộng cổng chiếm khoảng 33,5% số tham số của tầng, nhiều hơn chính SS2D; phần tăng thêm so với C2PSA (820.225 tham số) vì thế không chỉ đến từ SS2D. Bảng truy vết kích thước tensor qua từng bước và hình cơ cấu tham số ở Phụ lục F.

3.4. THUẬT TOÁN QUÉT KẾT HỢP SONG SONG

Xét một kênh d và một chiều trạng thái n, (3.4) là truy hồi tuyến tính bậc một

$$h_t = a_t\, h_{t-1} + b_t, \qquad h_0 = 0, \qquad t = 1, \dots, L \quad (3.13)$$

với $a_t = \bar{A}_t$, $b_t = \Delta_t B_t u_t$. Tính tuần tự cần khoảng 400 lượt phép toán nhỏ nối tiếp, chậm trên GPU dù $4 \times 256 \times 16 = 16.384$ chuỗi chạy song song với

nhau. Coi mỗi vị trí là phép biến đổi affine $h \mapsto a_t h + b_t$, hai phép liên tiếp hợp thành một phép cùng dạng:

$$(a_1, b_1) \circ (a_2, b_2) = (a_2 a_1,\ a_2 b_1 + b_2) \tag{3.14}$$

Toán tử này có tính kết hợp, nên h_t là thành phần thứ hai của tích tiền tố $(a_1, b_1) \circ \cdots \circ (a_t, b_t)$ và tính được bằng quét song song [45]. Mã dùng biến thể Hillis–Steele [46] (Thuật toán 3.1): ở vòng j, mỗi vị trí t kết hợp với vị trí $t - 2^j$; sau $\lceil \log_2 L \rceil$ vòng, mỗi vị trí đã gom đủ các phần tử đứng trước.

Thuật toán 3.1. Quét kết hợp song song (theo hàm `parallel_associative_scan`)

```
Đầu vào : a[1..L], b[1..L]   (xử lý cùng lúc cho mọi kênh d, chiều
trạng thái n, hướng k)
Đầu ra  : h[1..L] với h[t] = a[t]·h[t-1] + b[t], h[0] = 0

1. K ← ⌈log2 L⌉ ;  P ← 2^K
2. Đệm a đến độ dài P bằng giá trị 1, đệm b đến độ dài P bằng giá trị
0
   (cặp (1, 0) là phần tử đơn vị của phép ∘, nên không làm sai kết
quả)
3. step ← 1
4. while step < P:
5.     với mọi t > step, đồng thời:
6.         b[t] ← a[t]·b[t − step] + b[t]
7.         a[t] ← a[t]·a[t − step]
8.     (các vị trí t ≤ step giữ nguyên)
9.     step ← 2·step
10. return b[1..L]
```

Với $L = 400$, mã đệm lên $P = 512$ và chạy $K = 9$ vòng; dòng 6 và 7 tính cùng lúc bằng cắt lát tensor rồi ghép lại (`torch.cat`). Ví dụ từng vòng trên chuỗi $L = 4$ ở Phụ lục F.

Độ phức tạp. Số bước phải chờ nhau giảm từ 399 xuống 9 vòng, nhưng tổng công việc là $O(L \log L)$: mỗi chuỗi cần 4.097 phép kết hợp so với 399 khi tính tuần tự, mỗi ảnh khoảng 67 triệu phép kết hợp. Vì vậy bản cài đặt không có độ phức tạp tuyến tính như Mamba gốc, vốn đạt được nhờ biến thể Blelloch [45] và kernel CUDA gộp phép toán [23]. Bù lại, mã chỉ cần PyTorch, chạy được trên CPU lẫn GPU và cho kết quả tất định. Ở $L = 400$, attention của C2PSA với chi phí $O(L^2)$ vẫn còn rẻ; lợi thế của SSM chỉ thể hiện khi chuỗi rất dài.

Chi phí tầng 10. Ước tính tay (Phụ lục F) cho thấy phần xử lý chính của TSVMamba tốn khoảng 511 triệu MAC, gấp khoảng hai lần PSABlock (khoảng 246 triệu), chủ yếu ở nhánh hình dạng và FFN chứ không ở SS2D; với cả mô hình,

số phép tính chỉ tăng khoảng 1–2%. Độ trễ thì tăng nhiều hơn hẳn (Chương 6), nhiều khả năng do chi phí bộ nhớ của phép quét: mỗi vòng tạo tensor trung gian 256×16×512 cho từng hướng và ghép lại bằng `torch.cat`. Giả thuyết này chưa được kiểm chứng bằng profiler.

3.5. HUẤN LUYỆN VÀ LƯU TRỌNG SỐ

3.5.1. Khởi tạo và học chuyển giao

Cả hai mô hình xuất phát từ `yolo26s-seg.pt` tiền huấn luyện trên COCO [41]; Ultralytics chỉ chép tensor trùng cả tên lẫn kích thước. Ở mô hình đề xuất:

- **Tầng 0–9 và 11–23** nhận trọng số giống hệt mô hình đối chứng (trừ các lớp đầu ra phụ thuộc số lớp, không chép được ở cả hai mô hình).
- **Tầng 10:** 12 trong 58 tensor (cv1, cv2, FFN; 790.016 tham số) trùng tên và kích thước với C2PSA nên được kế thừa; FFN kế thừa chỉ vì trùng tên, không phải tái sử dụng có chủ đích. 1.021.185 tham số còn lại (SS2D, nhánh hình dạng, cổng, fuse, γ) khởi tạo mới: $A = -(1,2,\dots,16)$ theo S4D-Real [44], các lớp khác theo mặc định, $\gamma = 0{,}001$.

Vì γ nhỏ, ở những bước đầu tầng 10 hoạt động gần như $\mathrm{cv2}([a \parallel b])$: mô hình bắt đầu khi tầng 10 chưa có cơ chế tổng hợp ngữ cảnh, rồi học dần đóng góp của khối mới.

3.5.2. Hàm mất mát và chiến lược huấn luyện

Hai mô hình dùng chung hàm mất mát của YOLO26-seg, siêu tham số và danh sách seed, nên khác biệt có hệ thống chỉ quy được cho tầng 10. Tổng mất mát là tổ hợp có trọng số của đầu một-nhiều và đầu một-một [6], [20]:

$$\mathcal{L} = \alpha(e)\,\mathcal{L}_{\mathrm{o2m}} + (1-\alpha(e))\,\mathcal{L}_{\mathrm{o2o}}, \qquad \alpha(e) = 0{,}1 + 0{,}7 \cdot \max\left(1 - \frac{e}{E-1},\ 0\right) \tag{3.15}$$

Với $E = 100$, trọng số đầu một-nhiều giảm từ 0,8 xuống 0,1. Điểm neo dương chọn theo độ đo căn chỉnh của TOOD [37], $t = s^{0{,}5} \cdot u^{6}$ (s là điểm lớp, u là IoU): đầu một-nhiều lấy 10 điểm, đầu một-một chọn 7 ứng viên rồi giữ 1; quy tắc STAL nới cạnh vật nhỏ hơn 8 pixel lên 16 pixel [6].

Bảng 3.3: Các thành phần mất mát của YOLO26-seg (theo đúng nhánh mã đã chạy)

Thành phần	**Nội dung**	**Hệ số**
Hộp (box)	$1 - \mathrm{CIoU}$ [38], có trọng số theo điểm căn chỉnh	7,5
L1	Sai số tuyệt đối trên bốn khoảng cách tới biên hộp. Thay cho DFL vì `reg_max = 1` [6]	1,5
Lớp (cls)	Entropy chéo nhị phân, dùng nhãn mềm từ bộ gán nhãn	0,5
Mặt nạ (seg)	Entropy chéo nhị phân theo pixel giữa mặt nạ dự đoán và mặt nạ thật, chỉ tính trong hộp thật	7,5
Ngữ nghĩa (sem)	0,5·BCE + 0,5·Dice trên nhánh phân đoạn ngữ nghĩa phụ của Proto26. Chỉ chạy khi huấn luyện	7,5

Vì nhánh ngữ nghĩa phụ chỉ chạy khi huấn luyện, cột `val/sem_loss` trong nhật ký luôn bằng 0, đúng hành vi của mã. Bài báo YOLO26 dùng MuSGD [6], còn đề tài dùng AdamW [40] cho cả hai mô hình: tốc độ học 0,001, suy giảm trọng số 0,0005, FP32 hoàn toàn (`amp=False`), 100 epoch, lô 8 ảnh, chế độ tất định với seed cố định (chi tiết ở Chương 5). Quy trình suy luận và huấn luyện giống các bước ở mục 2.4.2, chỉ khác tầng 10 tính theo (3.1) và (3.12); Thuật toán PL.1 (Phụ lục F) viết đầy đủ hai quy trình này. Đề tài không thêm hàm mất mát nào về biên hay tô-pô.

3.5.3. Lưu trọng số ở dạng FP32

Mặc định Ultralytics chuyển mô hình sang FP16 trước khi lưu `best.pt`. Ở seed 5, `running_mean` của lớp BN trong tích chập fuse ở tầng 10 lên tới khoảng 1,5 triệu, vượt giới hạn 65.504 của FP16, nên checkpoint bị tràn số và suy luận sai dù huấn luyện ở FP32 vẫn bình thường. Đề tài sửa mã lưu (hàm lưu mô hình, MultiTrainer, `strip_optimizer`) để giữ FP32 bằng `.float()` thay cho `.half()`, và bỏ đoạn tự thay tensor không hữu hạn trong bản EMA để lỗi được báo ngay. Huấn luyện lại các lượt TSVM bằng mã đã sửa, so sánh `results.csv` trên 20 seed: 44.000 ô số liệu giống hệt, 2.000 ô khác đều thuộc cột thời gian. Như vậy bản sửa chỉ làm checkpoint được lưu đúng; tên "TSVM FP32" chỉ cách lưu này. Nguyên nhân có thể nằm ở thiết kế: SS2D không có lớp chuẩn hóa đầu ra và F_M còn nhân thêm $(1 + G)$ trước fuse, nên biên độ trước lớp BN của fuse không bị chặn. Đây mới là giả thuyết.

3.6. ƯU ĐIỂM VÀ HẠN CHẾ CỦA MÔ HÌNH ĐỀ XUẤT

Ưu điểm:

– **Thay đổi tối thiểu:** chỉ một tầng khác, giao diện giữ nguyên, khác biệt quy được cho khối mới.

– **Ngữ cảnh toàn cục có chọn lọc:** SS2D quét bốn hướng, mức ghi nhớ điều chỉnh theo nội dung qua Δ, B, C.

– **Thông tin hình dạng dẫn dắt đặc trưng toàn cục:** nhánh tích chập có hướng (1×5, 5×1) cùng cổng $(1 + G)$, trong khi C2PSA chỉ có tích chập 3×3 làm mã hóa vị trí.

– **Khởi đầu ổn định:** $\gamma = 0{,}001$ cùng việc kế thừa cv1, cv2, FFN giúp bắt đầu gần mô hình gốc.

– **Dễ chạy:** chỉ cần PyTorch, không biên dịch kernel CUDA, chạy được trên CPU, kết quả tất định.

Hạn chế:

– **Chi phí tăng:** tham số toàn mô hình +7,17%, checkpoint lớn hơn do lưu FP32, độ trễ CPU khoảng 2,1 lần, thời gian huấn luyện mỗi lượt từ khoảng 2,3 giờ lên 4,7 giờ.

– **Phép quét chưa tối ưu:** tổng công việc $O(L\log L)$, nhiều tensor trung gian; ở P5 ($L = 400$) lợi thế của SSM chưa thể hiện.

– **Khác bản VMamba gốc:** thiếu hạng tử D, thiếu LayerNorm đầu ra, khởi tạo Δ mặc định, có thể liên quan đến sự cố số học ở mục 3.5.3.

– **Tên gọi chưa có bằng chứng:** đo lỗi Betti trên 20 seed, TSVM chỉ nhích hơn mô hình gốc, không có ý nghĩa thống kê (Chương 6).

3.7. CÁC CHẾ ĐỘ TÁCH THÀNH PHẦN

Mô-đun TSVMamba có tham số `mode` để bật hoặc tắt từng nhánh (Bảng 3.4). Số tham số tầng 10 của cả bốn chế độ được đếm lại từ checkpoint của các lượt huấn luyện tách thành phần. Mọi lượt chính của TSVM dùng chế độ đầy đủ; ba chế độ còn lại được huấn luyện với seed 0–4 để ghép cặp với mô hình đầy đủ, kết quả ở mục 6.6. Với chỉ 5 seed, đóng góp của từng nhánh mới ở mức thăm dò; ngoài ra, vì các chế độ chênh nhau tới khoảng 740 nghìn tham số, so sánh này chưa tách được phần do cấu trúc với phần do thêm tham số.

Bảng 3.4: Bốn chế độ của khối TSVMamba

Chế độ	**Công thức khối**	**Tham số tầng 10**
`topology_shape_vmamba` (đầy đủ, dùng trong đề tài)	(3.12)	1.811.201
`shape_vmamba_no_guidance`	$b + \gamma \cdot$ FFN(fuse([SS2D(b S])), bỏ cổng	1.745.409
`shape_only`	$b + \gamma \cdot \mathrm{FFN}(S)$, bỏ SS2D	1.331.969
`vmamba_only`	$b + \gamma \cdot \mathrm{FFN}(\mathrm{SS2D}(b))$, bỏ nhánh hình dạng	1.071.873

3.8. KẾT CHƯƠNG

Mô hình đề xuất chỉ thay C2PSA ở tầng 10 bằng C2TSVMamba cùng giao diện 512×20×20, gồm nhánh SS2D bốn hướng, nhánh hình dạng có hướng với cổng $(1 + G)$, và nối tắt có hệ số γ nhỏ. Phép truy hồi được giải bằng quét song song Hillis–Steele (9 vòng, tổng công việc $O(L\log L)$). Khối mới thêm 820.225 tham số; số phép tính toàn mô hình chỉ tăng khoảng 1–2% nhưng độ trễ tăng đáng kể do cách cài đặt phép quét. Chương 6 đánh giá chi phí này đổi lại được bao nhiêu về chất lượng phân đoạn.

CHƯƠNG 4: XÂY DỰNG BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU

Chương này trình bày cách xây dựng bộ dữ liệu BG20 từ hai nguồn ảnh nội soi công khai và các bước đưa ảnh cùng nhãn về dạng mô hình YOLO26-seg sử dụng được. Quá trình gồm giai đoạn chuẩn bị, thực hiện một lần trước khi huấn luyện (chọn ảnh, chia tập, chuyển mặt nạ thành nhãn đa giác), và giai đoạn nạp dữ liệu, diễn ra mỗi khi ảnh được đưa vào mô hình (đổi kích thước, tăng cường, chuẩn hóa).

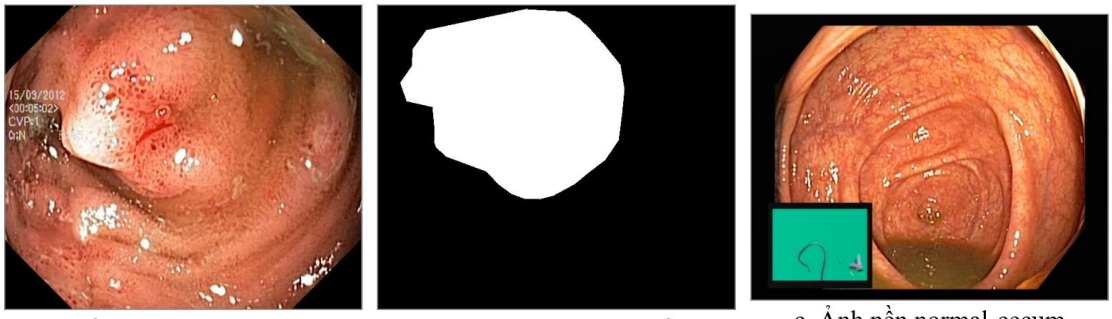
4.1. NGUỒN DỮ LIỆU

4.1.1. Bộ dữ liệu Kvasir-SEG

Kvasir-SEG gồm 1.000 ảnh nội soi đường tiêu hóa có polyp, do Jha và cộng sự công bố năm 2020 [5]. Mỗi ảnh đi kèm một mặt nạ do kỹ sư và bác sĩ khoanh vùng, được bác sĩ chuyên khoa tiêu hóa kiểm tra lại; pixel thuộc polyp màu trắng, còn lại màu đen. Dữ liệu có ba đặc điểm cần xử lý: ảnh có 333 kích thước khác nhau, chiều rộng từ 332 đến 1.920 pixel; kích thước polyp chênh lệch lớn, vùng nhỏ nhất chỉ chiếm 0,15% diện tích ảnh, lớn nhất khoảng 81%, và khoảng 48% số vùng chiếm không quá 10%; ảnh và mặt nạ lưu ở định dạng nén JPEG [5], nên biên polyp có các mức xám trung gian và mặt nạ cần nhị phân hóa lại (mục 4.3).

4.1.2. Ảnh nền normal-cecum

Bộ dữ liệu Kvasir phiên bản 2 gồm 8 lớp ảnh, mỗi lớp 1.000 ảnh [5], [8]. Đề tài dùng lớp normal-cecum (ảnh manh tràng bình thường, 720 × 576 pixel) làm ảnh nền, tức mẫu âm tính không chứa polyp. Nếu chỉ học trên ảnh có polyp, mô hình dễ cho rằng ảnh nào cũng có polyp và khoanh nhầm nếp gấp hay vùng phản quang. Hình 4.1 minh họa hai nguồn dữ liệu.



a. Ảnh nội soi có polyp b. Mặt nạ phân đoạn gốc c. Ảnh nền normal-cecum

Hình 4.1: Ví dụ ảnh có polyp, mặt nạ phân đoạn gốc và ảnh nền trong bộ dữ liệu BG20

Nguồn: ảnh a và b từ bộ dữ liệu Kvasir-SEG [5], ảnh c từ lớp normal-cecum của bộ dữ liệu Kvasir [8], đề tài ghép và đặt chú thích.

4.2. XÂY DỰNG VÀ PHÂN CHIA BỘ DỮ LIỆU BG20

Ảnh polyp được chia theo danh sách có sẵn gồm 880 ảnh huấn luyện và 120 ảnh kiểm định, do nhóm tác giả Kvasir-SEG cung cấp kèm bản phát hành trên Kaggle trong hai tệp train.txt và val.txt [53], giúp kết quả so sánh được với các nghiên cứu dùng cùng cách chia. Với ảnh nền, chương trình sắp xếp 1.000 tên tệp normal-cecum, cố định seed bằng 42 rồi chọn ngẫu nhiên 200 ảnh (160 huấn luyện, 40 kiểm định), nên mỗi lần chạy lại đều cho đúng 200 ảnh như nhau. Tên BG20 thể hiện số ảnh nền bằng 20% số ảnh polyp; tính trên toàn bộ 1.200 ảnh, ảnh nền chiếm 16,67%.

Bảng 4.1: Phân bố ảnh và nhãn trong bộ dữ liệu BG20

Thành phần	**Huấn luyện**	**Kiểm định**	**Tổng**
Ảnh có polyp	880	120	1.000
Ảnh nền normal-cecum	160	40	200
Tổng số ảnh	1.040	160	1.200
Số vùng polyp được gán nhãn	936	127	1.063

Nguồn: đề tài thống kê từ bộ dữ liệu BG20.

Số vùng polyp lớn hơn số ảnh polyp vì một ảnh có thể chứa nhiều polyp tách rời, mỗi polyp một nhãn riêng. Bộ dữ liệu theo định dạng Ultralytics (hai thư mục images và labels, mỗi ảnh một tệp nhãn cùng tên), chỉ có một lớp polyp với chỉ số 0. Ảnh nền đi kèm tệp nhãn rỗng, nên ảnh nền không phải là một lớp thứ hai.

4.3. CHUYỂN MẶT NẠ PHÂN ĐOẠN THÀNH NHÃN ĐA GIÁC

YOLO26-seg yêu cầu nhãn dạng văn bản: mỗi dòng gồm chỉ số lớp và tọa độ các đỉnh đa giác bao quanh đối tượng, chuẩn hóa về đoạn 0–1 [42]. Đề tài chuyển mỗi mặt nạ thành nhãn đa giác qua năm bước bằng thư viện OpenCV; ảnh nội soi giữ nguyên.

Bước 1. Nhị phân hóa bằng phương pháp Otsu. Do nén JPEG, mặt nạ có thêm mức xám trung gian (mặt nạ ở Hình 4.2 có 15 mức xám). Phương pháp Otsu [54] chọn ngưỡng t tự động, giữ ngưỡng làm phương sai giữa nhóm tối và nhóm sáng lớn nhất:

$$\sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2 \tag{4.1}$$

$$t^* = \underset{0 \le t \le 255}{\operatorname{argmax}}\ \sigma_B^2(t) \tag{4.2}$$

Trong đó ω_0, ω_1 là tỷ lệ số pixel và μ_0, μ_1 là mức xám trung bình của nhóm tối và nhóm sáng. Trên 1.000 mặt nạ, t^* chỉ nằm trong khoảng 5 đến 8: mặt nạ gốc gần như đã nhị phân, bước Otsu chủ yếu loại nhiễu do nén.

Bước 2. Làm liền vùng polyp bằng phép đóng hình thái học [52], với A là tập pixel trắng và B là phần tử cấu trúc vuông 3 × 3:

$$A \bullet B = (A \oplus B) \ominus B \tag{4.3}$$

Phép giãn nở $\oplus$ làm vùng trắng phình ra một pixel để lấp lỗ và khe, phép co $\ominus$ thu lại một pixel để biên trở về gần vị trí cũ.

Bước 3. Tìm đường bao và loại vùng nhiễu nhỏ bằng thuật toán dò biên của Suzuki và Abe [55], chỉ lấy đường bao ngoài cùng của mỗi vùng; đường bao có diện tích nhỏ hơn 20 pixel vuông bị loại. Trên 1.000 mặt nạ, chương trình tìm được 1.206 đường bao, loại 143, còn đúng 1.063 vùng polyp như Bảng 4.1.

Bước 4. Đơn giản hóa đường bao bằng thuật toán Douglas–Peucker [56], chỉ giữ những điểm lệch khỏi đoạn thẳng nối hai đầu nhiều hơn ngưỡng

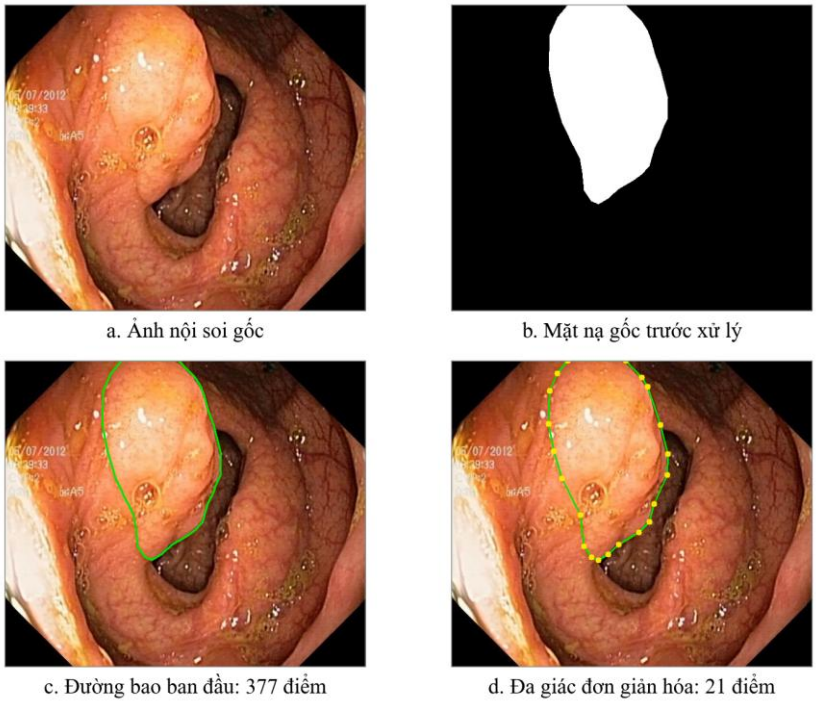
$$\varepsilon = 0{,}002 \cdot L \tag{4.4}$$

với L là chu vi đường bao (pixel); ví dụ chu vi 1.000 pixel cho ε bằng 2 pixel. Số điểm trung bình mỗi đa giác giảm từ 333,2 xuống 24,6 (giảm 92,6%), mỗi đa giác có từ 10 đến 80 đỉnh.

Bước 5. Chuẩn hóa tọa độ và ghi tệp nhãn:

$$\hat{x} = \frac{x}{W}, \qquad \hat{y} = \frac{y}{H} \tag{4.5}$$

Giá trị làm tròn đến 6 chữ số thập phân; mỗi polyp ghi thành một dòng gồm chỉ số lớp và các cặp tọa độ. Nhờ chuẩn hóa, nhãn không phụ thuộc kích thước ảnh. Hình 4.2 minh họa: đường bao ở ảnh c gồm 377 điểm, sau khi đơn giản hóa đa giác ở ảnh d chỉ còn 21 đỉnh nhưng vẫn giữ được hình dạng polyp.



a. Ảnh nội soi gốc

b. Mặt nạ gốc trước xử lý

c. Đường bao ban đầu: 377 điểm

d. Đa giác đơn giản hóa: 21 điểm

Hình 4.2: Trước và sau khi chuyển mặt nạ phân đoạn thành nhãn đa giác

Nguồn: ảnh a và b từ bộ dữ liệu Kvasir-SEG [5], ảnh c và d do đề tài tạo bằng chương trình chuyển đổi mặt nạ của đề tài.

4.4. ĐỔI KÍCH THƯỚC ẢNH

Mô hình dùng kích thước đầu vào 640 pixel. Để polyp không bị méo, ảnh được thu phóng giữ tỷ lệ sao cho cạnh dài nhất bằng 640:

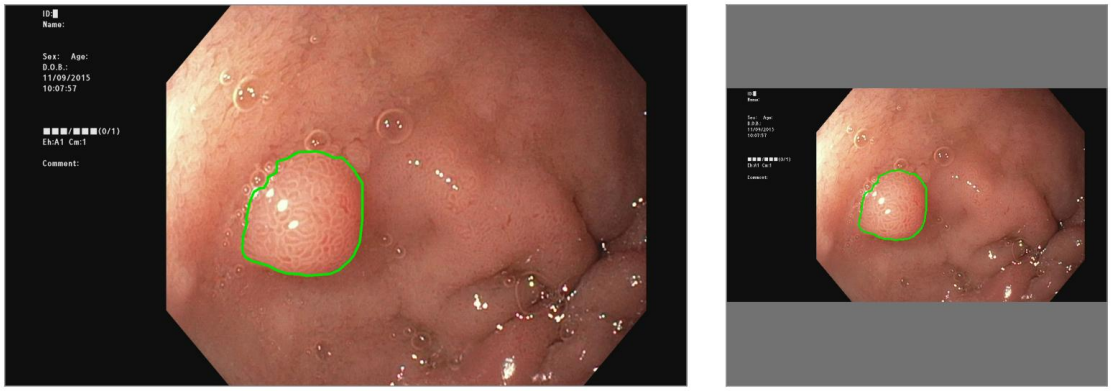
$$r = \frac{640}{\max(W,H)} \tag{4.6}$$

$$W' = \lceil r \cdot W \rceil, \qquad H' = \lceil r \cdot H \rceil \tag{4.7}$$

Phần thiếu của cạnh ngắn được bù bằng LetterBox: ảnh đặt giữa khung đích, phần trống tô xám 114 và chia đều hai phía:

$$p_x = \frac{640 - W'}{2}, \qquad p_y = \frac{640 - H'}{2} \tag{4.8}$$

Tọa độ đa giác cũng được nhân với r và cộng p_x, p_y. Ở nhánh huấn luyện, khung đích là 640 × 640; ở nhánh kiểm định, các ảnh có tỷ lệ khung gần nhau được gom thành lô 16 ảnh, khung mỗi lô làm tròn lên bội số của 32 và cộng thêm nửa bước, nên có cạnh dài 672 pixel. Ví dụ ở Hình 4.3, ảnh gốc 1.920 × 1.072 pixel thu còn 640 × 358 pixel, thêm hai dải xám cao 141 pixel ở trên và dưới.



a. Ảnh gốc 1920 × 1072 pixel

b. Sau LetterBox 640 × 640 pixel

Hình 4.3: Trước và sau khi đổi kích thước ảnh bằng thu phóng giữ tỷ lệ và LetterBox

Nguồn: ảnh a từ bộ dữ liệu Kvasir-SEG [5], ảnh b do đề tài tạo theo cách xử lý LetterBox trong mã nguồn Ultralytics.

4.5. TĂNG CƯỜNG DỮ LIỆU

Tập huấn luyện chỉ có 1.040 ảnh, nên mô hình dễ ghi nhớ từng ảnh. Tăng cường dữ liệu tạo một biến thể ngẫu nhiên khác nhau mỗi lần ảnh được nạp, chỉ áp dụng cho tập huấn luyện. Nhóm phép hình học thay đổi vị trí pixel nên đa giác nhãn biến đổi theo; nhóm phép màu giữ nguyên nhãn (Bảng 4.2).

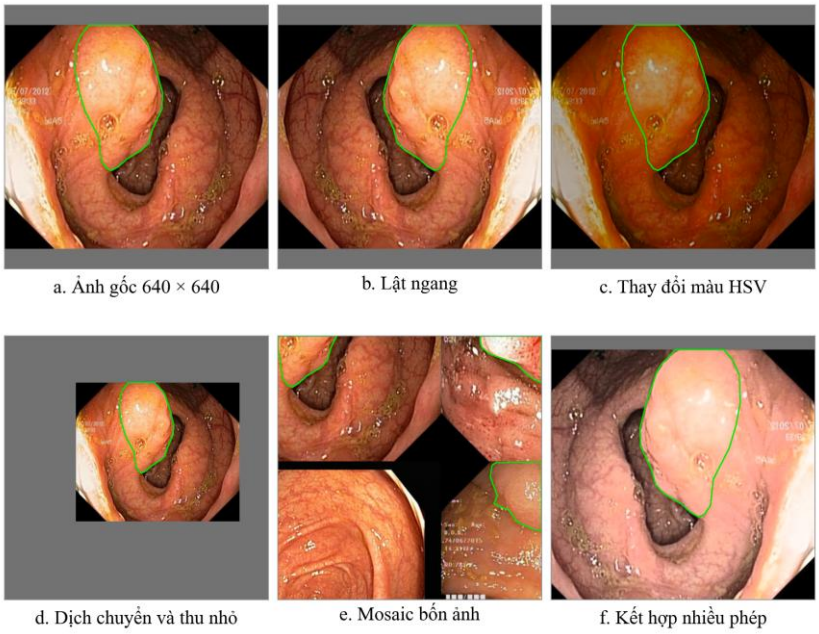
- **Mosaic** [39] ghép bốn ảnh vào bốn góc quanh một tâm ngẫu nhiên rồi cắt về 640 × 640, tạo polyp ở nhiều vị trí và kích thước; vì ảnh ghép không giống ảnh nội soi thật, Mosaic được tắt trong 10 epoch cuối [42].
- **Affine:** dịch tâm lệch tối đa 10% khung, thu phóng hệ số 0,5 đến 1,5; phần trống tô xám 114.
- **Màu HSV:** gần như giữ nguyên sắc độ để màu niêm mạc không phi thực tế, nhưng thay đổi mạnh độ bão hòa và độ sáng để mô phỏng điều kiện chiếu sáng.
- **Lật ngang** với xác suất 0,5. Lật dọc, xoay, cắt xiên và trộn ảnh không được sử dụng.

Bảng 4.2: Thông số tăng cường dữ liệu trong cấu hình huấn luyện

Phép biến đổi	**Tham số**	**Giá trị**	**Ý nghĩa**
Mosaic	mosaic	1,0	Luôn ghép bốn ảnh
Tắt Mosaic	close_mosaic	10	Tắt trong 10 epoch cuối
Dịch chuyển	translate	0,1	Lệch tối đa 10% khung
Thu phóng	scale	0,5	Hệ số từ 0,5 đến 1,5
Sắc độ	hsv_h	0,015	Dịch sắc độ rất nhẹ
Độ bão hòa	hsv_s	0,7	Thay đổi tối đa 70%
Độ sáng	hsv_v	0,4	Thay đổi tối đa 40%
Lật ngang	fliplr	0,5	Xác suất 50%
Lật dọc, xoay, cắt xiên, phối cảnh	flipud, degrees, shear, perspective	0	Không sử dụng
Trộn ảnh	mixup, cutmix, copy_paste	0	Không sử dụng

Nguồn: tệp cấu hình huấn luyện của đề tài.

Hình 4.4 minh họa từng phép tăng cường trên cùng một ảnh huấn luyện (hệ số chọn cố định trong phạm vi Bảng 4.2); ở cả sáu trường hợp, đường bao luôn khớp với polyp.



Hình 4.4: Ảnh huấn luyện trước và sau các phép tăng cường dữ liệu

Nguồn: ảnh gốc từ bộ dữ liệu Kvasir-SEG [5] và Kvasir [8], các phép biến đổi do đề tài tạo minh họa theo thông số trong Bảng 4.2.

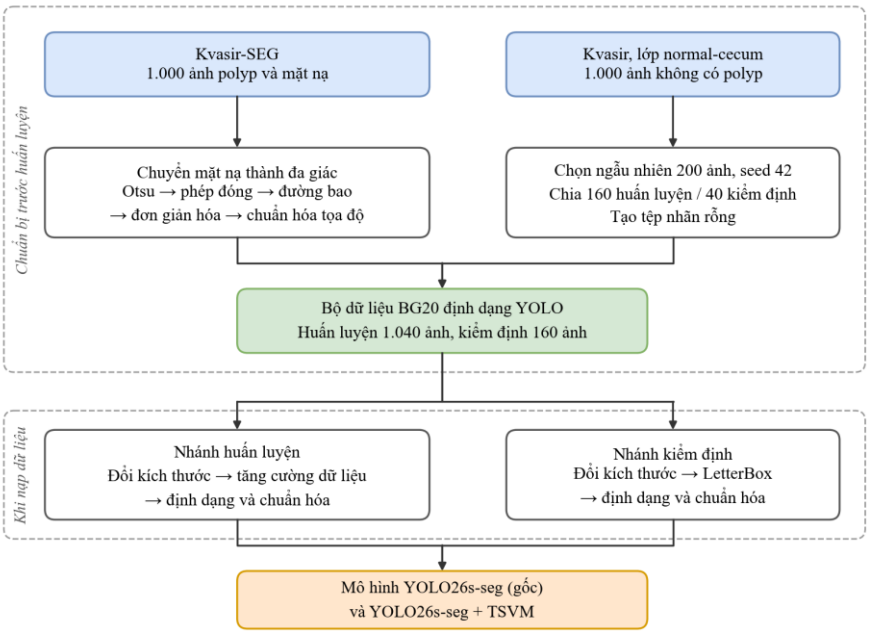
4.6. CHUẨN HÓA, KIỂM TRA TOÀN VẸN VÀ TỔNG HỢP QUY TRÌNH

Chuẩn hóa. Ảnh được đổi kênh màu từ BGR sang RGB, sắp lại chiều thành kênh × cao × rộng, ghép thành lô 8 ảnh và chia giá trị pixel cho 255 (không trừ trung bình hay chia độ lệch chuẩn của ImageNet). Đa giác được tô đầy để tạo lại mặt nạ huấn luyện 160 × 160, nhỏ hơn ảnh 4 lần; với ảnh nền, mặt nạ chỉ gồm giá trị 0.

Kiểm tra toàn vẹn. Chương trình chuyển đổi tự kiểm tra số ảnh và số tệp nhãn của từng tập. Lượt kiểm tra độc lập cho thấy mỗi ảnh có đúng một tệp nhãn, cả 1.063 dòng nhãn hợp lệ, không có ảnh trùng tên hoặc trùng nội dung theo mã băm SHA-256 giữa hai tập. Chạy lại bằng OpenCV 5.0.0, cả 1.000 tệp nhãn trùng khớp từng ký tự, và bước chọn ảnh nền với seed 42 cho đúng danh sách đã lưu. Giới hạn: dữ liệu không có thông tin định danh bệnh nhân nên chưa khẳng định được hai tập không chứa ảnh của cùng bệnh nhân; mã băm chưa phát hiện ảnh gần giống nhau; không có tập kiểm thử độc lập.

Tổng hợp. Hình 4.5 tóm tắt quy trình: phần trên là giai đoạn chuẩn bị (một lần), phần dưới là giai đoạn nạp dữ liệu với nhánh huấn luyện có tăng cường và

nhánh kiểm định chỉ đổi kích thước bằng LetterBox, cả hai kết thúc bằng bước chuẩn hóa.



Hình 4.5: Quy trình xây dựng bộ dữ liệu BG20 và tiền xử lý dữ liệu

Nguồn: đề tài tự xây dựng.

Khối TSVM nằm bên trong mô hình và chỉ xử lý đặc trưng sau khi ảnh đã được chuẩn hóa. Hai mô hình dùng chung quy trình tiền xử lý, nên dữ liệu không phải nguồn tạo ra khác biệt giữa chúng; dao động giữa các lần huấn luyện được kiểm soát bằng 20 seed ghép cặp ở Chương 6.

CHƯƠNG 5: CÀI ĐẶT VÀ ỨNG DỤNG

Ghi chú soạn thảo (xóa khi nộp):

– Bản rút gọn, nén từ `md_bao_cao/06_CHUONG_5_CAI_DAT_UNG_DUNG.md` (tag git `ban_day_du_147`). Bảng API, cấu trúc kết quả suy luận và cấu trúc bản ghi MongoDB chuyển xuống Phụ lục G. Ba bảng môi trường gộp thành Bảng 5.1; bảng chế độ tách thành phần dùng chung Bảng 3.4.

– Bản rút gọn giữ **7 ảnh chụp** (Hình 5.1, 5.3–5.8). Các ảnh tùy chọn của bản đầy đủ (trang chủ, đăng nhập, trang phân tích trước khi chạy, dashboard, quản trị, thông báo lỗi, chế độ tối, kết quả kiểm thử, tài liệu API `/docs`) không đưa vào bản này. Số hình ở đây khác số khung [CHỤP HÌNH] của bản đầy đủ.

– **[CẦN KIỂM TRA]** là điểm cần nhóm xác nhận trước khi nộp.

Chương này trình bày phần cài đặt thực nghiệm (môi trường, cài đặt hai mô hình, huấn luyện trên Kaggle, lưu trữ kết quả) và ứng dụng minh họa PolypVision (kiến trúc, dịch vụ suy luận, lưu trữ, giao diện Web và Mobile).

5.1. MÔI TRƯỜNG VÀ CÔNG CỤ

Huấn luyện chạy trên Kaggle theo đề cương; mỗi tài khoản có hạn mức GPU khoảng 30 giờ mỗi tuần, trong khi một lượt mô hình tích hợp mất khoảng 4,7 giờ, nên các lượt được chia cho nhiều tài khoản chạy song song. Kết quả được tải về máy cá nhân để tính lại số liệu, vẽ hình và đo tốc độ.

Bảng 5.1: Môi trường huấn luyện và đánh giá

Thành phần	**Giá trị**	**Ghi chú**
Huấn luyện	Kaggle Notebook, NVIDIA Tesla T4 15 GB, Python 3.12, PyTorch 2.10.0 + CUDA 12.8 **[CẦN KIỂM TRA]**	Theo nhật ký lượt chạy thử; chưa có nhật ký gắn với từng lượt chính
Ultralytics	8.4.127	Ghi trong mọi tệp `best.pt`
Thời gian một lượt (100 epoch)	Mô hình gốc khoảng 2,3 giờ; mô hình tích hợp khoảng 4,7 giờ	Trung bình 20 seed
Máy đánh giá	AMD Ryzen 7 250 (8 lõi, 16 luồng), RAM 15,3 GiB; NVIDIA GeForce RTX 5050 Laptop; Windows 11	Đo độ trễ CPU, GPU; đánh giá theo điểm ảnh

Thành phần	Giá trị	Ghi chú
Môi trường Python có GPU	PyTorch 2.13.0 + CUDA 13.0	Script đánh giá theo điểm ảnh
Dịch vụ suy luận	Python 3.12, PyTorch 2.6.0 (CPU), Ultralytics 8.4.127 theo `requirements.txt` **[CẦN KIỂM TRA: môi trường ảo hiện cài 8.4.103, nên cài lại cho khớp]**	Ứng dụng PolypVision

Đề cương quy định Python cho mô hình và API, Java cho Web; nhóm bổ sung Dart/Flutter cho di động. Đề tài chỉ dùng PyTorch (không dùng TensorFlow) vì Ultralytics YOLO26 [42] và các bản cài đặt Mamba đều viết trên PyTorch, nhờ đó khối C2TSVMamba ghép thẳng vào mô hình gốc. Các công cụ khác: NumPy, pandas, OpenCV, SciPy, Matplotlib cho dữ liệu, thống kê và vẽ hình; FastAPI, Uvicorn cho dịch vụ suy luận; Java 17, Spring Boot 4.1, Thymeleaf, Spring Security, Bootstrap 5 và MongoDB cho Web; Flutter cho di động; Git để quản lý mã.

5.2. CÀI ĐẶT HAI MÔ HÌNH

5.2.1. Mô hình gốc YOLO26s-seg

Mô hình gốc dùng nguyên bản Ultralytics 8.4.127, khởi tạo từ `yolo26s-seg.pt` tiền huấn luyện trên COCO [41], lớp đầu ra đổi từ 80 lớp sang 1 lớp `polyp` (tệp `data_bg20.yaml` khai báo `nc: 1`, `names: {0: polyp}`). Lệnh huấn luyện chính:

```
model = YOLO("yolo26s-seg.pt")
model.train(data="data_bg20.yaml", epochs=100, imgsz=640, batch=8,
            optimizer="AdamW", lr0=0.001, warmup_epochs=5.0,
            amp=False, deterministic=True, seed=SEED, workers=2,
            patience=100, close_mosaic=10)
```

Bảng 5.2: Siêu tham số huấn luyện (giống nhau ở mọi lượt của cả hai mô hình)

Nhóm	Tham số
Dữ liệu	Ảnh vào 640×640; không lưu đệm ảnh (`cache=False`)
Tối ưu	AdamW [40]; tốc độ học ban đầu 0,001, cuối 0,01 × ban đầu; momentum 0,937; suy giảm trọng số 0,0005; khởi động 5 epoch
Lịch	100 epoch; lô 8 ảnh; không dừng sớm (`patience=100`); tắt Mosaic ở 10 epoch cuối
Tăng cường	Mosaic [39] 1,0; lật ngang 0,5; co giãn 0,5; dịch 0,1; đổi màu HSV (0,015; 0,7; 0,4); không xoay, không lật dọc, không MixUp
Mặt nạ	Tỉ lệ thu nhỏ mặt nạ 4 (mặt nạ 160×160)

Nhóm	Tham số
Độ chính xác số	FP32 hoàn toàn (`amp=False`)
Tất định	`deterministic=True`, 2 luồng nạp dữ liệu, seed cố định cho từng lượt

Khóa tất định. Notebook đặt seed cho `random`, `numpy`, `torch` (cả CUDA), đặt `PYTHONHASHSEED`, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, tắt `cudnn.benchmark` và bật `torch.use_deterministic_algorithms(True)`, nên cùng seed thì cùng thứ tự dữ liệu và cùng khởi tạo.

5.2.2. Mô hình tích hợp VMamba

Khối C2TSVMamba không có trong Ultralytics gốc, nên nhóm tạo bản sao (fork) của Ultralytics 8.4.127:

Bảng 5.3: Các tệp được thêm hoặc sửa trong fork

Tệp	Nội dung thay đổi
`nn/modules/topology_shape_vmamba.py` (mới)	Cài đặt SS2D, ShapeAwareBranch, DirectionalShapeExtractor, TopologyShapeGate, TSVMamba và C2TSVMamba
`nn/modules/__init__.py`, `nn/tasks.py`	Xuất các lớp mới; đăng ký `C2TSVMamba` để trình đọc YAML dựng được mô hình
`cfg/models/26/yolo26-seg-TopologyShapeVMamba.yaml` (mới)	Giống `yolo26-seg.yaml`, chỉ đổi tầng 10: `C2PSA` → `C2TSVMamba`
`engine/trainer.py`, `utils/torch_utils.py`	Bản vá lưu checkpoint ở FP32, mô tả trong `PATCH_FP32.md`
`tests/test_topology_shape_vmamba.py` (mới)	Kiểm thử tự động cho khối mới

Mô hình tích hợp được dựng từ YAML ở cỡ *s* rồi nạp trọng số, cùng tham số huấn luyện ở Bảng 5.2:

```
model = YOLO("yolo26s-seg-TopologyShapeVMamba.yaml").load("yolo26s-seg.pt")
```

Lưu ý: Ultralytics đoán cỡ mô hình từ **tên tệp YAML**; tên phải chứa `yolo26s`, nếu không mô hình bị dựng ở cỡ *n* và không nạp được trọng số. Bản vá FP32 (mục 3.5.3) đổi hàm lưu mô hình, `MultiTrainer` và `strip_optimizer` sang `.float()`, và báo lỗi ngay khi gặp NaN hoặc vô cùng; vì vậy mô hình tích hợp được gọi là **TSVM FP32**. Thí nghiệm tách thành phần dùng cùng notebook, chỉ đổi đối số tầng

10 trong YAML, ví dụ `[-1, 2, C2TSVMamba, [1024, 0.5, 16, 3, "vmamba_only"]]`; ba chế độ ở Bảng 3.4, mỗi chế độ chạy seed 0–4, tổng 15 lượt chia cho 6 tài khoản Kaggle.

5.3. QUY TRÌNH HUẤN LUYỆN VÀ LƯU TRỮ KẾT QUẢ THỰC NGHIỆM

Đề tài dùng ba notebook Kaggle: notebook mô hình gốc (tạo `data_bg20.yaml`, cài `ultralytics==8.4.127`, khóa tất định, huấn luyện, nén kết quả); notebook mô hình tích hợp (thêm bước nạp fork bằng liên kết tượng trưng tên `ultralytics` và tải `yolo26s-seg.pt`); notebook tách thành phần (thêm ô chọn chế độ, seed và ô **kiểm tra mô hình** trước khi huấn luyện, sai số tham số tầng 10 thì dừng ngay để không tốn giờ GPU). Bản notebook mô hình tích hợp còn lưu trong kho mã trỏ tới fork cũ (FP16); khi chạy các lượt chính, đường dẫn đã được sửa sang fork FP32, bằng chứng là mọi `best.pt` của mô hình tích hợp đều lưu FP32 **[NÊN LƯU LẠI BẢN NOTEBOOK ĐÃ SỬA VÀO KHO MÃ]**.

[CHỤP HÌNH 5.1] Notebook Kaggle đang chạy.

- **Màn hình:** notebook tách thành phần trên Kaggle (đang chạy), khung bên phải mở mục *Input* (thấy hai dataset `kvasir-yolo-seg-bg20` và `ultralytics-topology-shape-aware-vmamba-fp32`) và *Settings* (thấy Accelerator là GPU).
- **Cần thấy:** đầu ra của ô kiểm tra mô hình, có dòng "Tham số tầng 10 = … ☑", và vài dòng nhật ký huấn luyện có cột epoch, loss, mAP.
- **Lưu ý:** che tên tài khoản Kaggle nếu không muốn công khai.

Mỗi lượt huấn luyện lưu riêng một thư mục `Kvasir_BG20_<Mô hình>_s<seed>_w2` gồm `args.yaml` (toàn bộ cấu hình), `results.csv` (chỉ số và loss từng epoch) và `weights/best.pt`, đặt trong `03_ket_qua_huan_luyen/` theo từng mô hình (`Baseline_YOLO26s_seg/`, `TSVM_FP32/`, và `Vmamba_only/`, `Shape_only/`, `No_guidance/` cho thí nghiệm tách thành phần). Từ các tệp này, script `tinh_lai_so_lieu.py` tính lại mọi số liệu ra bảng CSV, `danh_gia_pixel.py` suy luận lại để tính Dice và IoU theo điểm ảnh cho bốn mô hình, `ve_hinh.py` vẽ hình. Mọi con số trong Chương 6 truy được về bảng và script tương ứng (Phụ lục C).

5.4. ỨNG DỤNG POLYPVISION: YÊU CẦU VÀ KIẾN TRÚC

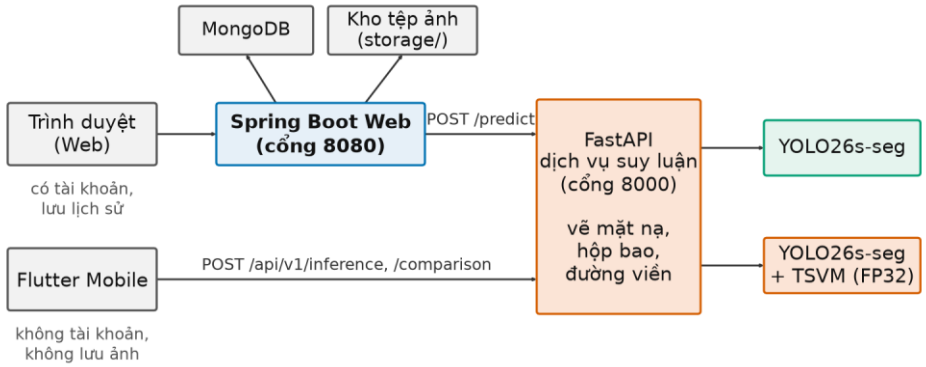
Theo đề cương, ứng dụng cần cho phép tải ảnh nội soi, phân đoạn polyp và hiển thị mặt nạ, đường biên, độ tin cậy, lưu trữ kết quả dự đoán và so sánh mô hình

gốc với mô hình tích hợp VMamba. Nhóm bổ sung quản lý tài khoản, phân quyền, lịch sử, trang quản trị và ứng dụng di động.

Bảng 5.4: Đối chiếu yêu cầu đề cương với chức năng đã cài đặt

Yêu cầu	**Web**	**Mobile**	**Ghi chú**
Tải ảnh nội soi	Có	Có (chọn từ thư viện hoặc chụp)	JPG, JPEG, PNG; tối đa 10 MB
Hiển thị mặt nạ phân đoạn	Có	Có	Ảnh chồng (overlay) mặt nạ lên ảnh gốc
Hiển thị đường biên polyp	Có	Có	Dịch vụ suy luận vẽ thêm đường viền vàng theo đa giác của từng mặt nạ (`masks.xy`) sau khi Ultralytics tô mặt nạ và hộp bao
Hiển thị độ tin cậy	Có (thanh tiến trình, %)	Có	Độ tin cậy cao nhất trong các đối tượng; dịch vụ trả thang 0–1, Website đổi sang 0–100
Lưu trữ kết quả	Có (MongoDB + tệp ảnh)	Không lưu	Mobile cố ý không lưu ảnh để bảo vệ quyền riêng tư
So sánh hai mô hình	Có	Có	Chạy lần lượt hai mô hình trên cùng ảnh

Hệ thống gồm ba thành phần (Hình 5.2). **Dịch vụ suy luận FastAPI** giữ hai mô hình, dùng chung cho nhánh Web (trình duyệt → Spring Boot → FastAPI; Spring Boot quản lý tài khoản, lịch sử, MongoDB, tệp ảnh) và nhánh Mobile (Flutter → FastAPI, không tài khoản, không lưu lịch sử). Tách dịch vụ suy luận giúp nạp mô hình một lần, phần Java không phụ thuộc PyTorch, và có thể chuyển dịch vụ sang máy có GPU mà không đổi giao diện.



Hình 5.2: Kiến trúc hệ thống PolypVision. Website (Spring Boot, cổng 8080) và ứng dụng di động (Flutter) là hai client độc lập của cùng dịch vụ suy luận FastAPI (cổng 8000). Chỉ nhánh Web có MongoDB và kho tệp ảnh.

5.5. DỊCH VỤ SUY LUẬN VÀ TỔ CHỨC LƯU TRỮ

Khi khởi động, dịch vụ nạp hai checkpoint lấy từ **seed 0**: `YOLO26s-seg` (mô hình gốc) và `YOLO26s-TSVM-FP32-seg` (mô hình tích hợp). Mã khối C2TSVMamba được chép vào dịch vụ và đăng ký đúng đường dẫn mô-đun ghi trong checkpoint; mô hình được ép về FP32 và suy luận với `half=False`; tệp `experiment_manifest.json` lưu mã băm SHA-256 của từng checkpoint, dịch vụ trả mã băm của tệp đang nạp ở API `/health` để đối chiếu. Ở seed 0, epoch của `best.pt` trùng epoch có mask mAP50–95 cao nhất (mô hình gốc epoch 89, mô hình tích hợp epoch 88), nhưng riêng seed này mô hình tích hợp có mask mAP50–95 **thấp hơn** mô hình gốc (0,7245 so với 0,7366), nên các hình so sánh trên ứng dụng chỉ có tính minh họa. Ngưỡng tin cậy mặc định 0,25, kích thước ảnh 640. Dịch vụ có API riêng cho Web (`/health`, `/predict`) và cho Mobile (`/api/v1/models`, `/api/v1/inference`, `/api/v1/comparison`…), kiểm tra ảnh đầu vào trước khi suy luận và không lưu ảnh sau khi xử lý; danh sách API và cấu trúc kết quả ở Phụ lục G. Thời gian `processingTimeMs` trả về gồm cả tiền xử lý và hậu xử lý, không phải phép đo hiệu năng khoa học.

Chỉ nhánh Web lưu kết quả: tài khoản (tập `users`, mật khẩu băm bằng BCrypt) và lịch sử phân tích (tập `diagnosis_history`) trong MongoDB, ảnh gốc và ảnh kết quả trong thư mục `storage/original/` và `storage/results/`. Một lần so sánh tạo hai bản ghi có cùng `comparisonGroupId`, nên dùng chung cấu trúc cho phân tích đơn lẻ và so sánh. Người dùng thường chỉ xem và xóa được dữ liệu của mình; tài khoản quản trị xem được thống kê toàn hệ thống.

5.6. GIAO DIỆN VÀ CHỨC NĂNG

Website (Spring Boot, Thymeleaf, Bootstrap 5) có: đăng ký, đăng nhập, phân quyền `USER` và `ADMIN`; phân tích một ảnh bằng một trong hai mô hình; so sánh hai mô hình trên cùng ảnh; lịch sử phân tích và so sánh có tìm kiếm, lọc, phân trang; tải ảnh kết quả, xóa dữ liệu; hồ sơ, đổi mật khẩu; trang quản trị và thống kê. Các cơ chế bảo mật cơ bản gồm phiên đăng nhập, chống giả mạo yêu cầu (CSRF), băm mật khẩu và tiêu đề HTTP bảo mật.

[CHỤP HÌNH 5.3] Kết quả phân tích một ảnh. *(Hình quan trọng nhất của chương)*

– **Thao tác:** đăng nhập bằng **tài khoản thử tạo riêng cho việc chụp hình**, vào trang *Phân tích* (`/diagnose`, **không** dùng trang `/diagnoses/demo` vì đó là kết quả mẫu cố định). Chọn một ảnh tập kiểm định có polyp rõ (gợi ý: một trong

ba ảnh ở Hình q01 Chương 6), mô hình *YOLO26s-TSVM-FP32-seg*, bấm *Phân tích*.

– **Cần thấy cùng lúc:** ảnh gốc và ảnh có mặt nạ chồng lên, đường viền vàng; **ô độ tin cậy dạng %** có thanh tiến trình; số polyp; thời gian xử lý; tên mô hình; dòng cảnh báo y khoa.

[CHỤP HÌNH 5.4] Kết quả so sánh hai mô hình.

– **Thao tác:** vào trang *So sánh* (`/compare`), tải **cùng ảnh** ở Hình 5.3, bấm chạy.

– **Cần thấy:** hai cột kết quả cạnh nhau, mỗi cột có ảnh overlay, độ tin cậy, số mặt nạ và thời gian xử lý. Chú thích chỉ mô tả, **không kết luận** mô hình nào tốt hơn từ một ảnh.

[CHỤP HÌNH 5.5] Lịch sử phân tích.

– **Thao tác:** sau khi đã phân tích vài ảnh (đơn lẻ và so sánh), vào trang *Lịch sử* (`/history`).

– **Cần thấy:** danh sách bản ghi có ảnh thu nhỏ, tên mô hình, độ tin cậy, thời điểm; ô tìm kiếm, bộ lọc, phân trang.

[CHỤP HÌNH 5.6] Dữ liệu lưu trữ. *(Minh chứng "tổ chức lưu trữ kết quả" của đề cương)*

– **Thao tác:** mở MongoDB Compass, tập `diagnosis_history`, mở bản ghi của lần phân tích ở Hình 5.3; mở thêm thư mục `storage/`. Ghép thành một hình hai phần.

– **Cần thấy:** các trường `modelType`, `confidence`, `originalImageUrl`, `resultImageUrl`, `createdAt`… và hai thư mục `original/`, `results/` có tệp ảnh tương ứng.

– **Lưu ý:** không chụp tập `users` (có email và mã băm mật khẩu).

Ứng dụng di động (Flutter) gọi thẳng dịch vụ suy luận, phục vụ trình diễn nhanh: màn hình chính hiển thị trạng thái dịch vụ và hai mô hình; suy luận một mô hình hoặc so sánh hai mô hình; kiểm tra ảnh (JPG, JPEG, PNG, tối đa 10 MB); tiến trình tải lên, hủy, thử lại, báo lỗi; xem ảnh toàn màn hình; tự co giãn theo màn hình, chế độ sáng và tối.

[CHỤP HÌNH 5.7] Ứng dụng di động (một hình ba phần: (a) màn hình chính, (b) kết quả suy luận, (c) so sánh hai mô hình).

– **Thao tác:** chạy dịch vụ suy luận với `--host 0.0.0.0`, chạy ứng dụng trên máy ảo Android (`AI_BASE_URL=http://10.0.2.2:8000`) hoặc điện thoại thật cùng mạng; dùng cùng ảnh ở Hình 5.3, chọn mô hình tích hợp.

– **Cần thấy:** (a) trạng thái dịch vụ "sẵn sàng" và hai mô hình; (b) ảnh overlay, độ tin cậy, số mặt nạ, thời gian xử lý; (c) kết quả hai mô hình trên cùng màn hình.

Kiểm thử. Mỗi thành phần có bộ kiểm thử tự động riêng (Bảng 5.5); ngoài ra nhóm đã kiểm thử thủ công trên điện thoại Android thật các chức năng kiểm tra trạng thái, suy luận, so sánh, phóng to, thử lại khi lỗi, xoay màn hình, chế độ tối.

Bảng 5.5: Kết quả kiểm thử tự động **[CẦN KIỂM TRA: chạy lại các lệnh kiểm thử trước khi nộp và cập nhật số]**

Thành phần	Công cụ	Kết quả ghi nhận
Ứng dụng di động	`flutter test`	498/498 ca đạt
Dịch vụ suy luận	`pytest`	50/50 ca đạt
Website	`mvnw test` (JUnit)	261/261 ca đạt

5.7. TRIỂN KHAI TRÊN DỮ LIỆU THỰC TẾ

Để kiểm tra mô hình trên ảnh **khác nguồn**, nhóm dùng bốn bộ dữ liệu công khai không tham gia huấn luyện: CVC-ClinicDB (612 ảnh), CVC-ColonDB (380 ảnh), ETIS-Larib (196 ảnh) và phần ảnh đơn của PolypGen (1.537 ảnh từ 6 trung tâm). Việc triển khai gồm đánh giá định lượng (chạy checkpoint 20 seed của bốn mô hình trên toàn bộ bốn bộ, tính Dice, IoU, Precision, Recall theo điểm ảnh, kết quả ở mục 6.7) và minh họa trên ứng dụng bằng chức năng so sánh.

[CHỤP HÌNH 5.8] So sánh hai mô hình trên ảnh ngoài (một hình hai phần: (a) ảnh CVC-ClinicDB, (b) ảnh ETIS-Larib).

– **Thao tác:** trang *So sánh* của Website; mỗi bộ chọn một ảnh đại diện cho xu hướng chung ở mục 6.7, không chọn trường hợp hiếm.

– **Cần thấy:** ảnh overlay và độ tin cậy của hai mô hình. Chú thích ghi rõ tên bộ dữ liệu và tên tệp ảnh.

Kết quả trên ứng dụng chỉ có tính minh họa; kết luận về khả năng tổng quát hóa dựa trên đánh giá định lượng ở Chương 6. Bốn bộ dữ liệu là ảnh công khai, không phải dữ liệu lâm sàng thu tại bệnh viện.

5.8. HẠN CHẾ VÀ KẾT CHƯƠNG

Phần cài đặt còn các hạn chế: phép quét SS2D viết bằng PyTorch thuần nên mô hình tích hợp chậm khoảng gấp đôi trên CPU; thông tin GPU và phiên bản PyTorch trên Kaggle chỉ có từ nhật ký lượt chạy thử; ứng dụng di động chưa có tài khoản, lịch sử và chưa chạy ngoại tuyến; chưa có HTTPS và giới hạn tần suất gọi API, ảnh lưu trên đĩa cục bộ; ứng dụng không phải thiết bị y tế, độ tin cậy hiển thị là điểm của mô hình chứ không phải xác suất mắc bệnh.

Chương 5 đã trình bày môi trường, cách cài đặt hai mô hình (fork Ultralytics chỉ khác tầng 10, bản vá lưu FP32), quy trình huấn luyện có khóa tất định với kết quả truy được về tệp gốc, và ứng dụng PolypVision đáp ứng các yêu cầu tải ảnh, hiển thị mặt nạ và độ tin cậy, lưu trữ và so sánh hai mô hình. Chương 6 trình bày kết quả thực nghiệm.

CHƯƠNG 6: THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trả lời câu hỏi: khi thay khối C2PSA ở tầng 10 bằng C2TSVMamba (gọi tắt là **TSVM**), chất lượng phân đoạn thay đổi thế nào và chi phí tính toán tăng bao nhiêu.

6.1. THIẾT LẬP THỰC NGHIỆM

Mô hình. Đề tài đánh giá bốn mô hình, mỗi mô hình 20 lượt huấn luyện (seed 0–19): YOLO26s-seg (mô hình gốc, đối chứng), TSVM (mô hình đề xuất, chỉ khác tầng 10, lưu trọng số FP32), PraNet với xương sống ResNet-50 [9] và U-Net [10]. Hai mô hình YOLO dùng chung mọi thiết lập (BG20, ảnh 640 pixel, 100 epoch, lô 8, AdamW, khởi tạo COCO, cùng tăng cường dữ liệu); lượt của hai mô hình có cùng seed tạo thành một **cặp**, nhờ đó phần dao động do seed được trừ bớt khi so sánh [31]. Một lượt mất trung bình khoảng 2,3 giờ với mô hình gốc và 4,7 giờ với TSVM. PraNet và U-Net được huấn luyện ở 352 × 352 pixel, không tăng cường dữ liệu, chỉ để đặt kết quả vào bối cảnh.

Tập đánh giá. Mục 6.2–6.6 tính trên **tập kiểm định của BG20** (120 ảnh có polyp với 127 vùng polyp, 40 ảnh nền). Không có tập kiểm thử riêng nên kết quả tuyệt đối có thể hơi lạc quan, nhưng hai mô hình chịu cùng thiên lệch. Mục 6.7 bổ sung bốn bộ dữ liệu không tham gia huấn luyện.

Chỉ số chính là **mask mAP50–95** (mục 2.7.3), chọn trước khi xem kết quả vì nó phạt cả khoanh thiếu lẫn biên lệch; các chỉ số khác là chỉ số phụ, mang tính thăm dò. Với mỗi lượt, đề tài lấy **epoch có mask mAP50–95 lớn nhất** và mọi chỉ số khác ở cùng epoch; mục 6.2.3 kiểm tra lại với hai quy tắc khác.

Kiểm định. Với hiệu ghép cặp $d_i = x_i^{\text{TSVM}} - x_i^{\text{gốc}}$, đề tài dùng kiểm định t ghép cặp [48] kèm CI 95% (mục 2.8.2), kiểm định Wilcoxon dấu–hạng [49] (mục 2.8.3), cỡ hiệu ứng d_z [51] và số seed TSVM tốt hơn; mức ý nghĩa $\alpha = 0{,}05$. Đề tài không hiệu chỉnh đa kiểm định, nên chỉ kết luận chắc chắn trên chỉ số chính [50]. Để so sánh độ dao động giữa các seed, đề tài dùng kiểm định Levene dạng trung vị (Brown–Forsythe) [57]; kiểm định này coi hai nhóm là độc lập nên chỉ mang tính thăm dò. Các phép tính dùng SciPy.

Đánh giá theo pixel. PraNet và U-Net không cho ra từng đối tượng kèm điểm tin cậy, nên để so sánh bốn mô hình, đề tài chạy lại suy luận và tính Dice, IoU, Precision, Recall **theo pixel** (mục 2.7.2): nhãn là mặt nạ vẽ lại từ nhãn đa giác

ở độ phân giải gốc; YOLO dùng ảnh 640 pixel, ngưỡng tin cậy 0,25, ngưỡng IoU 0,7, gộp mọi mặt nạ của ảnh; PraNet, U-Net dùng ảnh 352 × 352, ngưỡng 0,5. Chỉ số tính trên **từng ảnh** của 120 ảnh có polyp (ảnh bị bỏ sót hoàn toàn có Dice bằng 0), lấy trung bình mỗi seed rồi qua 20 seed; Dice gộp toàn bộ pixel được ghi kèm. Với 40 ảnh nền, đề tài đếm tỉ lệ ảnh có ít nhất một pixel bị dự đoán là polyp (tỉ lệ **báo nhầm**). Mọi giá trị ghi dưới dạng **trung bình ± độ lệch chuẩn qua 20 seed**.

6.2. SO SÁNH MÔ HÌNH GỐC VÀ MÔ HÌNH ĐỀ XUẤT TRÊN 20 SEED

6.2.1. Kết quả tổng hợp

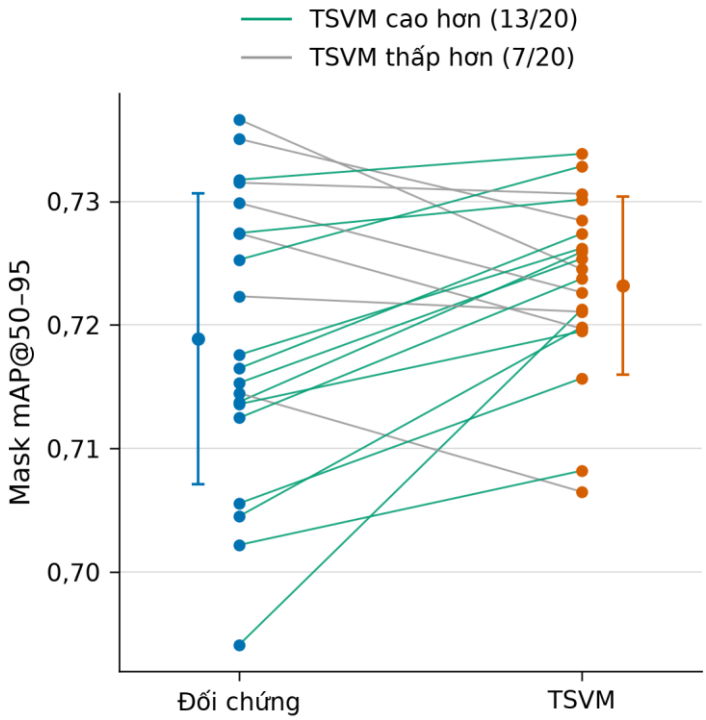
Bảng 6.1: Chỉ số mặt nạ của hai mô hình (20 seed, epoch có mask mAP50–95 lớn nhất)

Chỉ số	YOLO26s-seg	TSVM	Δ (TSVM − gốc)	CI 95% của Δ	p (t)	p (Wilcoxon)	Seed TSVM tốt hơn
Mask mAP50–95	0,7189 ± 0,0118	0,7232 ± 0,0072	**+0,0043**	[−0,0002; 0,0088]	0,061	0,070	13/20
Mask mAP50	0,9069 ± 0,0125	0,9083 ± 0,0086	+0,0014	[−0,0053; 0,0080]	0,668	0,812	10/20
Mask Precision	0,9041 ± 0,0287	0,9145 ± 0,0233	+0,0104	[−0,0086; 0,0293]	0,266	0,498	11/20
Mask Recall	0,8594 ± 0,0241	0,8590 ± 0,0179	−0,0004	[−0,0126; 0,0119]	0,948	0,926	9/20
Mask F1	0,8806 ± 0,0142	0,8856 ± 0,0129	+0,0049		0,259	0,388	12/20
Box mAP50–95	0,7228 ± 0,0159	0,7273 ± 0,0134	+0,0045	[−0,0031; 0,0121]	0,227	0,105	13/20
Box mAP50	0,9006 ± 0,0116	0,9014 ± 0,0083	+0,0008	[−0,0053; 0,0069]	0,778	1,000	11/20

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Trên chỉ số chính, TSVM đạt 0,7232, cao hơn mô hình gốc 0,0043 (khoảng 0,6% tương đối) và cao hơn ở 13/20 seed. Tuy vậy, CI 95% của hiệu số vẫn chứa 0, kiểm định t cho $p = 0{,}061$, Wilcoxon cho $p = 0{,}070$, cỡ hiệu ứng $d_z = 0{,}44$ ở mức nhỏ đến vừa theo quy ước của Cohen (được Lakens [51] trình bày lại). **Ở mức ý nghĩa 0,05, chưa đủ bằng chứng để khẳng định TSVM tốt hơn mô hình gốc**; kết quả chỉ cho thấy xu hướng tăng nhỏ. Ngược lại, $p > 0{,}05$ cũng không chứng minh hai mô hình tương đương: hiệu số thật có thể từ gần 0 đến khoảng +0,009.

Các chỉ số phụ đi cùng chiều nhưng yếu hơn và không chỉ số nào có ý nghĩa thống kê: Precision tăng khoảng 1 điểm phần trăm, Recall gần như không đổi, mAP50 gần bằng nhau (+0,0014). Điều này gợi ý phần chênh lệch nhỏ nằm ở các ngưỡng IoU cao, tức mức khớp biên mặt nạ, chứ không ở khả năng tìm ra polyp.



Hình 6.1: Mask mAP50–95 của từng seed. Mỗi đường nối hai lượt cùng seed của hai mô hình.

Hiệu số dao động khá rộng, từ −0,0121 (seed 0) đến +0,0272 (seed 3). Lượt tốt nhất của mô hình gốc (0,7366, seed 0) còn cao hơn lượt tốt nhất của TSVM (0,7339, seed 8); các hiệu số dương lớn nhất rơi vào những seed mà mô hình gốc thấp, như seed 3 (0,6941 so với 0,7213) và seed 10 (0,7045 so với 0,7198).

6.2.2. Độ ổn định giữa các seed

Bảng 6.2: Mức dao động của mask mAP50–95 qua 20 seed

	YOLO26s-seg	**TSVM**
Độ lệch chuẩn	0,0118	0,0072
Nhỏ nhất	0,6941	0,7065
Lớn nhất	0,7366	0,7339
Khoảng (lớn nhất − nhỏ nhất)	0,0425	0,0274

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Độ lệch chuẩn của TSVM nhỏ hơn khoảng 39%, kiểm định Levene dạng trung vị cho $p = 0{,}026$, kết quả duy nhất trên tập kiểm định có $p < 0{,}05$. Vì kiểm định này bỏ qua ghép cặp và không phải chỉ số chính, đề tài chỉ coi đây là dấu hiệu thăm dò: **TSVM ít phụ thuộc vào seed hơn**. Dấu hiệu này chỉ đúng ở epoch tốt nhất: ở epoch 100, độ lệch chuẩn hai mô hình gần bằng nhau (0,0132 và 0,0130);

trong 100 epoch có 34 epoch mà độ lệch chuẩn của TSVM lớn hơn; và trên dữ liệu ngoài (mục 6.7) nó không còn.

6.2.3. Độ nhạy theo quy tắc chọn epoch

Bảng 6.3: Mask mAP50–95 theo ba quy tắc chọn epoch

Quy tắc	YOLO26s-seg	TSVM	Δ	p (t)	p (Wilcoxon)	Seed TSVM tốt hơn
Mask mAP50–95 lớn nhất (chỉ số chính)	0,7189 ± 0,0118	0,7232 ± 0,0072	+0,0043	0,061	0,070	13/20
Epoch của `best.pt` (chỉ số tổng hợp hộp và mặt nạ)	0,7174 ± 0,0129	0,7222 ± 0,0075	+0,0048	0,062	0,076	14/20
Epoch 100 (cuối huấn luyện)	0,7093 ± 0,0132	0,7119 ± 0,0130	+0,0025	0,456	0,349	12/20

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Hai quy tắc dựa trên epoch tốt nhất cho kết luận gần như nhau; lấy epoch cuối thì khoảng cách thu hẹp còn +0,0025 và không còn dấu hiệu ổn định hơn. Kết luận "xu hướng tăng nhỏ, chưa có ý nghĩa thống kê" không phụ thuộc cách chọn epoch.

6.2.4. Quá trình hội tụ

Theo đường mAP trung bình 20 seed (Phụ lục H): trong khoảng 10 epoch đầu TSVM tăng chậm hơn, phù hợp với cách khởi tạo ở mục 3.5.1 (hơn một triệu tham số mới khởi tạo ngẫu nhiên, $\gamma = 0{,}001$ làm khối mới gần như "tắt" lúc đầu); từ epoch 50 đến 81 hai đường đan xen; từ epoch 82 đến 100 trung bình của TSVM luôn cao hơn. Trung vị epoch tốt nhất là 88 với mô hình gốc và 92 với TSVM. Hàm mất mát phân đoạn trên tập kiểm định của mô hình gốc thấp nhất ở epoch 59 (1,2294) rồi tăng lên 1,2893 ở epoch 100, của TSVM thấp nhất ở epoch 90

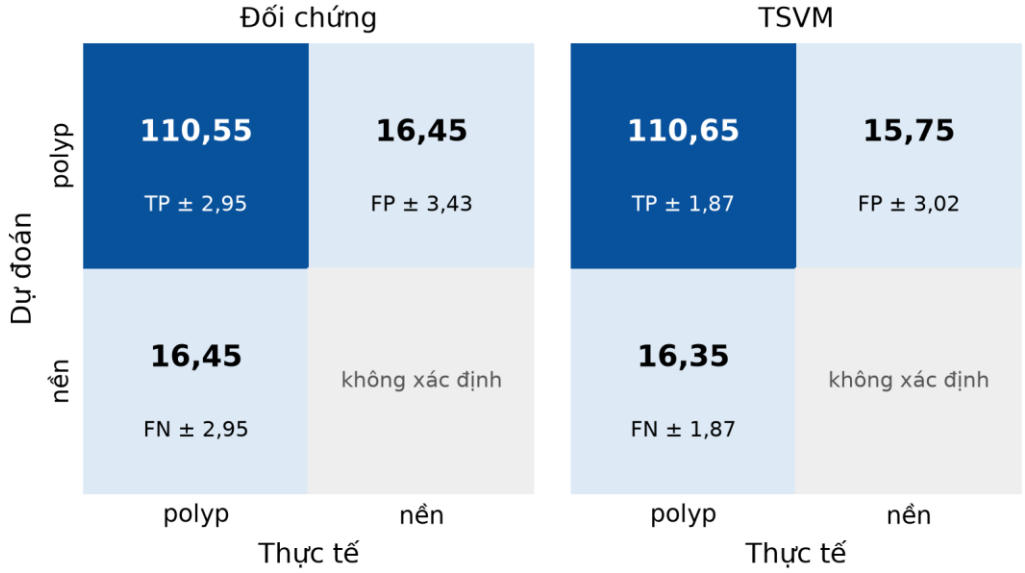
(1,2236): mô hình gốc có dấu hiệu quá khớp sớm hơn trên hàm mất mát, dù mAP vẫn tăng tới cuối. Đây là quan sát mô tả, chưa kiểm định.

6.2.5. Ma trận nhầm lẫn theo đối tượng

Bảng 6.4: Số vùng polyp phát hiện đúng, bỏ sót và dự đoán thừa (trên 127 vùng polyp)

	YOLO26s-seg	TSVM	Δ	p (Wilcoxon)
Phát hiện đúng (TP)	110,55 ± 2,95	110,65 ± 1,87	+0,10	1,000
Bỏ sót (FN)	16,45 ± 2,95	16,35 ± 1,87	−0,10	1,000
Dự đoán thừa (FP)	16,45 ± 3,43	15,75 ± 3,02	−0,70	0,671

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.2: Ma trận nhầm lẫn trung bình 20 seed. Ô nền–nền để trống vì không xác định được ở mức đối tượng.

Hai mô hình tìm được gần như cùng số polyp, khoảng 110,6/127 vùng (87%); TSVM dự đoán thừa ít hơn trung bình 0,7 vùng, không có ý nghĩa thống kê. Ô TN không xác định được ở mức đối tượng, nên đề tài không tính độ đặc hiệu từ bảng này; tỉ lệ báo nhầm trên ảnh nền được đo riêng ở mục 6.3.

6.3. ĐÁNH GIÁ THEO PIXEL VÀ SO SÁNH VỚI PRANET, U-NET

Bảng 6.5: Chỉ số theo pixel của bốn mô hình trên tập kiểm định (20 seed)

Mô hình	Dice	IoU	Precision	Recall	Dice gộp pixel	Ảnh nền báo nhầm (%)
YOLO26s-seg	0,8486 ± 0,0171	0,7927 ± 0,0170	0,8585 ± 0,0170	0,8703 ± 0,0193	0,8659 ± 0,0313	1,2 ± 1,9

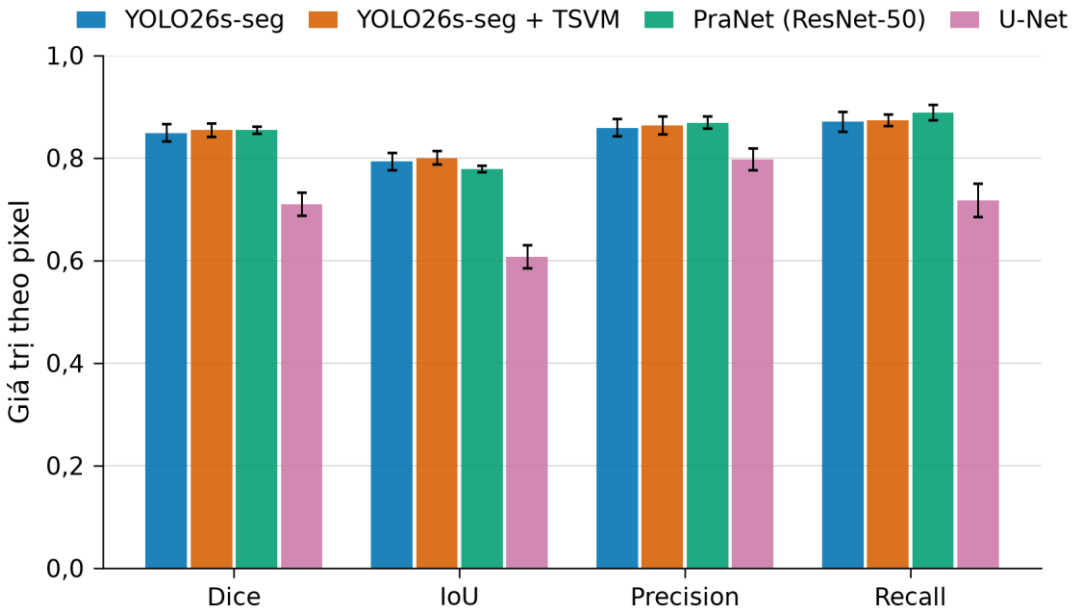
Mô hình	Dice	IoU	Precision	Recall	Dice gộp pixel	Ảnh nền báo nhầm (%)
TSVM	0,8540 ± 0,0129	**0,7998 ± 0,0129**	0,8632 ± 0,0173	0,8730 ± 0,0112	**0,8785 ± 0,0243**	1,5 ± 2,1
PraNet	**0,8541 ± 0,0068**	0,7785 ± 0,0063	**0,8687 ± 0,0117**	**0,8881 ± 0,0145**	0,8715 ± 0,0060	**0,8 ± 1,2**
U-Net	0,7097 ± 0,0221	0,6069 ± 0,0221	0,7964 ± 0,0213	0,7167 ± 0,0327	0,7412 ± 0,0185	20,4 ± 6,4

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Bảng 6.6: So sánh ghép cặp TSVM với từng mô hình theo pixel (cùng seed)

So với	Chỉ số	Δ (TSVM − mô hình kia)	p (t)	p (Wilcoxon)	Seed TSVM cao hơn
YOLO26s-seg	Dice	+0,0054	0,260	0,246	13/20
	IoU	+0,0071	0,130	0,114	14/20
	Ảnh nền báo nhầm	+0,25 điểm %	0,694	0,793	
PraNet	Dice	−0,0001	0,982	0,927	12/20
	IoU	+0,0213	< 0,001	< 0,001	19/20
	Recall	−0,0151	0,002	0,001	5/20
U-Net	Dice	+0,1443	< 0,001	< 0,001	20/20
	Ảnh nền báo nhầm	−18,9 điểm %	< 0,001	< 0,001	

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

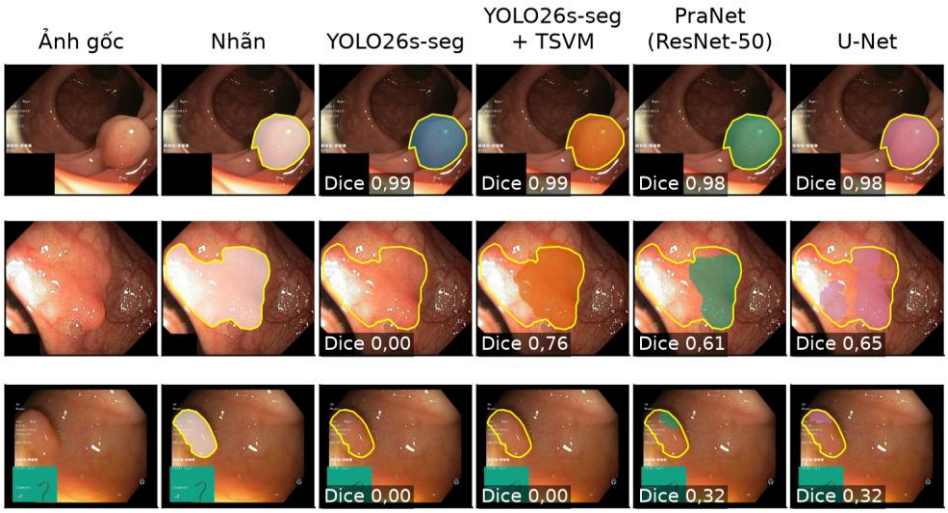


Hình 6.3: Dice, IoU, Precision và Recall theo pixel của bốn mô hình. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed.

So với mô hình gốc, TSVM vẫn có xu hướng tăng nhẹ theo pixel (Dice +0,005, IoU +0,007, cao hơn ở 13–14/20 seed) nhưng không có ý nghĩa thống kê, khớp với kết luận theo mAP.

So với PraNet, Dice trung bình gần như bằng nhau, nhưng hai mô hình mạnh ở hai mặt: TSVM có **IoU cao hơn** ở 19/20 seed ($p < 0{,}001$), tức mặt nạ khớp nhãn sát hơn khi đã khoanh được polyp; PraNet có **Recall cao hơn** ($p = 0{,}002$) và **ổn định hơn nhiều** giữa các seed (độ lệch chuẩn Dice 0,0068 so với 0,0129). Dice trung bình của YOLO bị kéo xuống bởi các ảnh **bỏ sót hoàn toàn**: ở ngưỡng tin cậy 0,25, YOLO không dự đoán pixel nào trên trung bình khoảng 6,5/120 ảnh có polyp mỗi seed (mô hình gốc 6,45, TSVM 6,65), còn PraNet chỉ 0,95 ảnh. Khi gộp pixel cả 120 ảnh, Dice của TSVM là 0,8785, cao hơn PraNet (0,8715).

So với U-Net, ba mô hình còn lại hơn rất xa; U-Net còn báo nhầm khoảng 20% ảnh nền, trong khi YOLO và PraNet chỉ khoảng 1–1,5% (dưới một ảnh mỗi seed). Ảnh nền của tập kiểm định đều lấy từ lớp manh tràng bình thường, khá đồng nhất, nên con số này chưa đại diện cho niêm mạc bình thường nói chung.



Hình 6.4: Ví dụ dự đoán của bốn mô hình (seed 0). Ba ảnh được chọn tự động: ảnh cả bốn mô hình đều tốt, ảnh TSVM hơn mô hình gốc nhiều nhất và ảnh khó nhất.

Hình 6.4 chỉ có tính minh họa; ảnh được chọn theo tiêu chí trong chú thích, không ngẫu nhiên. **Giới hạn của phép so sánh:** YOLO dùng ảnh 640 pixel, có tăng cường và khởi tạo COCO, còn PraNet, U-Net dùng ảnh 352 pixel, không tăng cường; PraNet dùng ResNet-50 thay cho Res2Net-50 của bản gốc; ngưỡng chưa tối ưu riêng. Kết quả phản ánh các cấu hình cụ thể này, không so trực tiếp với số trong bài báo gốc của PraNet.

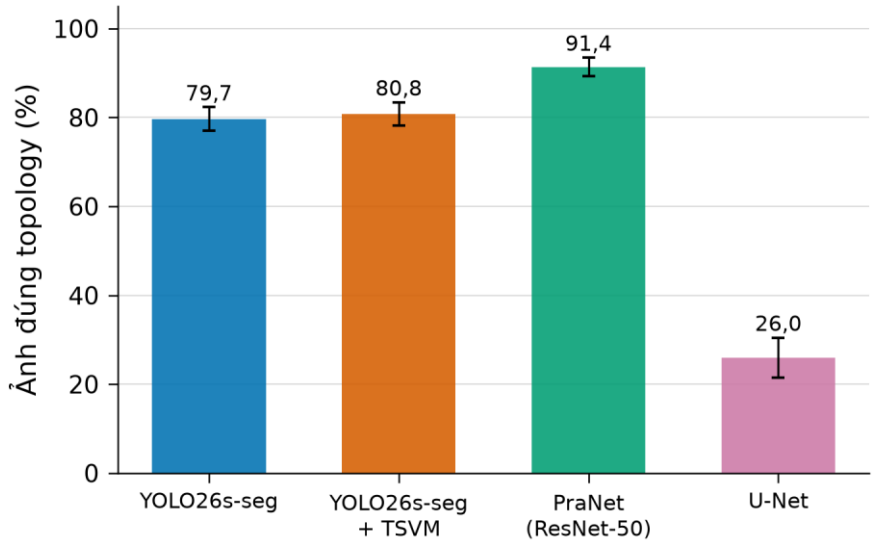
6.4. CHỈ SỐ TOPOLOGY CỦA MẶT NẠ

mAP và Dice gần như không phạt các lỗi hình dạng tổng thể như mặt nạ vỡ nhiều mảnh hay có lỗ nhỏ. Đề tài đo trực tiếp hai số Betti đầu tiên của mặt nạ: b_0 là **số mảnh** (thành phần liên thông 8 hướng) và b_1 là **số lỗ** (vùng nền liên thông 4 hướng nằm hẳn bên trong vùng polyp); mảnh và lỗ dưới 20 pixel không được đếm, trùng ngưỡng loại nhiễu ở Chương 4. Trên 120 ảnh có polyp, nhãn có 113 ảnh một mảnh, 7 ảnh hai mảnh, không ảnh nào có lỗ. Một ảnh **đúng topology** khi cả b_0 và b_1 của dự đoán bằng của nhãn.

Bảng 6.7: Chỉ số topology của bốn mô hình (ngưỡng 20 pixel, 20 seed)

Chỉ số	YOLO26s-seg	TSVM	PraNet	U-Net
Ảnh đúng topology ↑	0,797 ± 0,027	0,808 ± 0,026	**0,914 ± 0,021**	0,260 ± 0,045
Lỗi b_0 trung bình ↓	0,198 ± 0,045	0,191 ± 0,032	**0,086 ± 0,025**	2,745 ± 0,641
Lỗi b_1 trung bình ↓	0,105 ± 0,039	0,094 ± 0,027	**0,004 ± 0,006**	1,215 ± 0,419
Ảnh thừa mảnh ↓	0,082 ± 0,027	0,082 ± 0,018	**0,057 ± 0,021**	0,638 ± 0,046
Ảnh có lỗ ↓	0,073 ± 0,020	0,063 ± 0,014	**0,004 ± 0,005**	0,432 ± 0,069
Ảnh thiếu mảnh ↓	0,071 ± 0,023	0,066 ± 0,016	**0,027 ± 0,008**	0,028 ± 0,016

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.5: Tỉ lệ ảnh có polyp được dự đoán đúng topology (số mảnh và số lỗ khớp nhãn).

So với mô hình gốc, mọi chỉ số topology của TSVM đều nhích theo chiều có lợi (ảnh đúng topology +1,1 điểm phần trăm, ảnh có lỗ −1 điểm phần trăm) nhưng không chỉ số nào có ý nghĩa thống kê (ảnh đúng topology $p = 0{,}139$, ảnh có lỗ $p = 0{,}099$); khi đếm cả mảnh dưới 20 pixel, kết luận không đổi (0,758 so với 0,770; $p = 0{,}086$). **Đề tài không khẳng định khối đề xuất cải thiện topology của mặt nạ.** PraNet có topology tốt nhất rõ rệt: 91% ảnh đúng so với 81% của TSVM, TSVM kém hơn ở cả 20/20 seed ($p < 0{,}001$). Một giải thích hợp lý, chưa kiểm chứng, là PraNet lấy ngưỡng một bản đồ xác suất trơn, còn YOLO ghép mặt nạ từ các mặt nạ mẫu 160 × 160 rồi phóng lên (mục 2.3.3), dễ tạo lỗ nhỏ và mép răng cưa. Ở hai mô hình YOLO, lỗi chia khá đều giữa thừa mảnh, có lỗ và thiếu mảnh, mỗi kiểu khoảng 6–8% số ảnh.

6.5. CHI PHÍ TÍNH TOÁN

Bảng 6.8: Số tham số và số phép tính của hai mô hình

	YOLO26s-seg	**TSVM**	**Chênh lệch**
Tham số toàn mô hình	11.434.463	12.254.688	+820.225 (+7,2%)
Tham số tầng 10	990.976 (C2PSA)	1.811.201 (C2TSVMamba)	+82,8%
GFLOPs (`FlopCounterMode` của PyTorch)	34,29	34,82	+1,5%
GFLOPs (cách đếm của Ultralytics)	37,32	37,86	+1,4%
Tệp `best.pt`	22,27 MiB (FP16)	47,37 MiB (FP32)	

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

`FlopCounterMode` chỉ đếm tích chập và nhân ma trận, không đếm các phép theo từng phần tử trong phép quét SS2D, nên GFLOPs của TSVM là **cận dưới**. Tệp `best.pt` của TSVM lớn gấp khoảng hai lần chủ yếu vì lưu FP32 (mục 3.5.3).

Bảng 6.9: Độ trễ suy luận của hai mô hình (checkpoint seed 0, lô 1 ảnh, ảnh 640 × 640, FP32)

Phần cứng	Cách đo	YOLO26s-seg (ms)	TSVM (ms)	Tỉ lệ
CPU AMD Ryzen 7 250, 4 luồng	Chỉ lan truyền xuôi, trung bình 100 lần	269,56 ± 18,71	577,36 ± 24,40	2,14 lần
	Cả quy trình dự đoán (tiền xử lý, lan truyền xuôi, hậu xử lý), 20 ảnh	286,56 ± 20,09	567,86 ± 24,45	1,98 lần
GPU RTX 5050 Laptop	Chỉ lan truyền xuôi, trung vị 1.000 lần	26,33	39,86	1,51 lần
	Trung bình ± độ lệch chuẩn	26,99 ± 3,20	41,42 ± 6,82	1,53 lần
	Nhỏ nhất	20,79	29,80	1,43 lần

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Trên GPU, mỗi mô hình được đo 5 vòng, mỗi vòng 200 lần sau 50 lần khởi động, xen kẽ hai mô hình và đảo thứ tự mỗi vòng; trước mỗi khối đo, chương trình chờ GPU nguội dưới 55 °C. GPU ở 41–51 °C và chạy ở xung nhịp tối đa trong cả lần đo; tỉ lệ độ trễ giữa hai mô hình ổn định qua các vòng, từ 1,44 đến 1,63 lần. Một lần đo trước đó không kiểm soát nhiệt cho độ trễ tuyệt đối nhỏ hơn khoảng 1,5–2 lần (giá trị nhỏ nhất: mô hình gốc 11,8 ms, TSVM 20,4 ms) nhưng tỉ lệ tương tự, khoảng 1,7 lần; vì vậy **tỉ lệ khoảng 1,5–1,7 lần là kết quả đáng tin hơn** số mili giây tuyệt đối.

Nhận xét. Số phép tính chỉ tăng khoảng 1,5% nhưng độ trễ tăng khoảng 1,5 lần trên GPU và 2,1 lần trên CPU: GFLOPs không phản ánh đúng tốc độ của khối Mamba viết bằng PyTorch thuần. Nguyên nhân chính là phép quét song song (mục 3.4): 9 vòng nối tiếp, mỗi vòng tạo tensor trung gian và nhiều phép theo từng phần tử, bị giới hạn bởi băng thông bộ nhớ và số lần gọi kernel; Mamba gốc tránh được nhờ kernel CUDA riêng [23]. Như vậy TSVM **tốn thêm khoảng 50–110% thời gian suy luận để đổi lấy mức tăng mAP50–95 khoảng 0,004 chưa có ý nghĩa thống kê**. Ở khoảng 40 ms trên GPU, mô hình vẫn xử lý được khoảng 25 ảnh mỗi giây khi chỉ tính lan truyền xuôi, nhưng đây là đo trên một máy, một checkpoint.

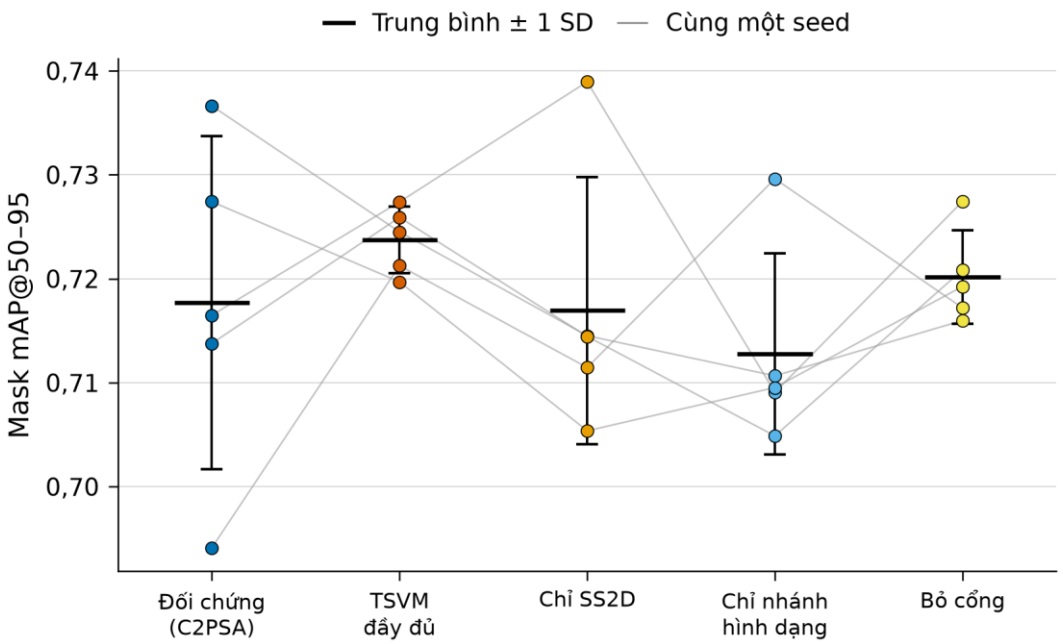
6.6. THÍ NGHIỆM TÁCH THÀNH PHẦN

Để biết thành phần nào của khối TSVMamba đóng góp, đề tài huấn luyện thêm ba biến thể (Bảng 3.4): `vmamba_only` (chỉ giữ SS2D), `shape_only` (chỉ giữ nhánh hình dạng), `shape_vmamba_no_guidance` (giữ hai nhánh, bỏ cổng, hai nhánh chỉ được nối rồi trộn bằng tích chập 1 × 1). Mọi thiết lập khác giữ nguyên; do giới hạn giờ GPU, mỗi biến thể chạy **5 seed (seed 0–4)**, cùng seed với các lượt chính để ghép cặp. Tệp `args.yaml` của 15 lượt đều trùng tham số huấn luyện với lượt TSVM chính cùng seed, chỉ khác tệp cấu hình mô hình (`bang/18_ablation_cau_hinh_args.csv`).

Bảng 6.10: Kết quả thí nghiệm tách thành phần (mask mAP50–95, seed 0–4)

Cấu hình	SS2D	Nhánh hình dạng	Cổng	Tham số tầng 10	Mask mAP50–95	Δ so với gốc	Δ so với đầy đủ	Số seed cao hơn bản đầy đủ
YOLO26s-seg (C2PSA)				990.976	0,7177 ± 0,0160	–	−0,0061	2/5
TSVM đầy đủ	✓	✓	✓	1.811.201	**0,7238 ± 0,0032**	+0,0061	–	–
`vmamba_only`	✓			1.071.873	0,7170 ± 0,0128	−0,0007	−0,0068	1/5
`shape_only`		✓		1.331.969	0,7128 ± 0,0097	−0,0049	−0,0110	1/5
`shape_vmamba_no_guidance`	✓	✓		1.745.409	0,7201 ± 0,0045	+0,0025	−0,0036	1/5

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình 6.6: Mask mAP50–95 từng seed (0–4) của năm cấu hình. Đường xám nối kết quả của cùng một seed; vạch ngang đậm là trung bình, thanh dọc là ±1 độ lệch chuẩn qua 5 seed.

Cả ba biến thể đều có trung bình thấp hơn bản đầy đủ, mỗi biến thể chỉ cao hơn bản đầy đủ ở 1/5 seed:

1. **Bỏ nhánh hình dạng (`vmamba_only`):** giảm 0,0068, gần bằng mô hình gốc (−0,0007), độ lệch chuẩn 0,0128 gấp khoảng bốn lần bản đầy đủ. Seed 1 đạt 0,7390, cao nhất trong 25 lượt, nhưng bốn seed còn lại đều thấp hơn bản đầy đủ. Chỉ thay attention bằng SS2D chưa cho thấy lợi ích.
2. **Bỏ SS2D (`shape_only`):** trung bình thấp nhất, kém bản đầy đủ 0,0110 và kém cả mô hình gốc 0,0049. Nhánh hình dạng dùng riêng không thay được khối ngữ cảnh toàn cục.
3. **Bỏ cổng:** giảm ít nhất (−0,0036), độ lệch chuẩn vẫn nhỏ (0,0045), đứng thứ hai. Phần lớn khác biệt đến từ việc dùng cùng lúc hai nhánh; cổng góp thêm một phần nhỏ, cùng chiều ở 4/5 seed.

Kết quả cho thấy một **xu hướng**: hai nhánh bổ sung cho nhau, bản đầy đủ vừa có trung bình cao nhất vừa dao động ít nhất. Tuy nhiên không biến thể nào khác bản đầy đủ có ý nghĩa thống kê (t ghép cặp $p = 0{,}218$, 0,101 và 0,085; với 5 cặp, Wilcoxon hai phía không thể cho p nhỏ hơn 0,0625). Với lực kiểm định thấp, phần này chỉ dùng để mô tả xu hướng; khoảng cách giữa các cấu hình còn nhỏ hơn độ lệch chuẩn giữa các seed ở mục 6.2 (khoảng 0,007–0,012), và các biến

thể chênh nhau tới khoảng 740 nghìn tham số tầng 10 nên chưa tách được phần do cấu trúc với phần do thêm tham số. Thời gian huấn luyện cũng tách được: `shape_only` mất khoảng 2,4 giờ mỗi lượt, gần bằng mô hình gốc (2,3 giờ), còn `vmamba_only` khoảng 4,6 giờ, bằng bản đầy đủ; gần như toàn bộ thời gian tăng thêm nằm ở SS2D.

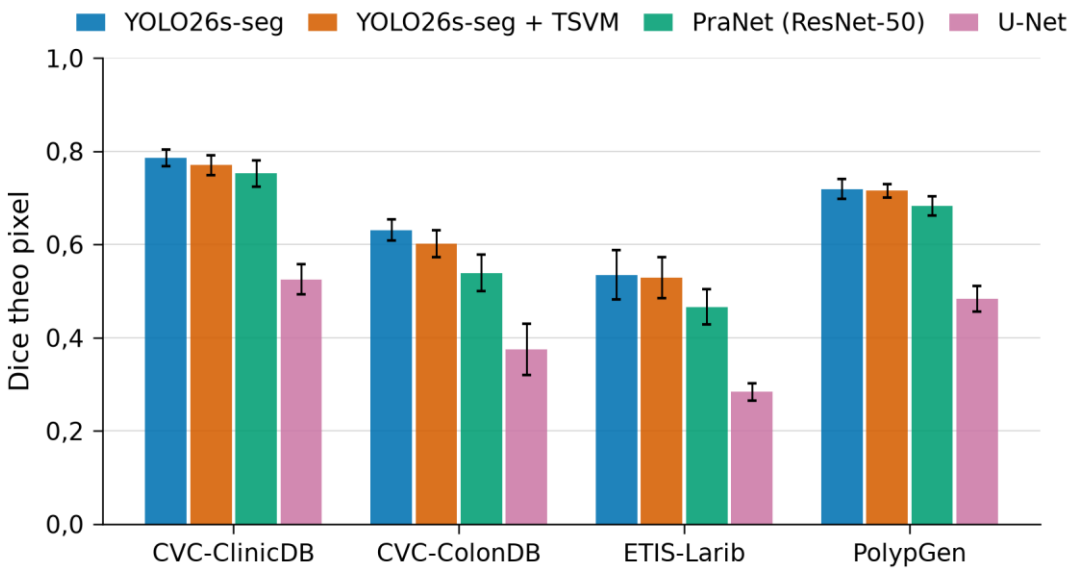
6.7. KẾT QUẢ TRÊN DỮ LIỆU THỰC TẾ

Để đánh giá khả năng tổng quát hóa, đề tài lấy nguyên checkpoint đã huấn luyện trên BG20, chạy trên bốn bộ dữ liệu **khác nguồn** không huấn luyện lại: CVC-ClinicDB [13] (612 ảnh, 384 × 288), CVC-ColonDB [14] (380 ảnh, 574 × 500; bản phổ biến trong các benchmark từ PraNet [9], bài gốc mô tả 300 ảnh), ETIS-Larib [15] (196 ảnh, 1.225 × 966, nhiều polyp nhỏ) và phần ảnh đơn của PolypGen [12] (1.537 ảnh từ 6 trung tâm C1–C6, tới 1.920 × 1.080; 1.412 ảnh có polyp, 125 ảnh mặt nạ rỗng dùng để đo báo nhầm). Giao thức giống mục 6.1, với bốn mô hình và 20 seed. Kiểm tra trùng ảnh giữa PolypGen và Kvasir-SEG bằng MD5, mã băm cảm nhận (pHash, dHash) và tương quan ảnh xám thu nhỏ không thấy cặp nào trùng. Đây là ảnh công khai, không phải dữ liệu lâm sàng do nhóm thu tại bệnh viện.

Bảng 6.11: Dice và IoU theo pixel trên bốn bộ dữ liệu ngoài (20 seed)

Bộ dữ liệu	Chỉ số	YOLO26s-seg	TSVM	PraNet	U-Net	TSVM − gốc	p (t)	p (Wilcoxon)	Seed TSVM cao hơn
CVC-ClinicDB	Dice	**0,786**	0,770	0,752	0,525	−0,016	0,006	0,006	5/20
	IoU	**0,725**	0,712	0,676	0,425	−0,013	0,010	0,015	6/20
CVC-ColonDB	Dice	**0,631**	0,601	0,539	0,375	−0,030	< 0,001	< 0,001	3/20
	IoU	**0,574**	0,549	0,474	0,304	−0,025	0,001	< 0,001	3/20
ETIS-Larib	Dice	**0,535**	0,529	0,466	0,284	−0,005	0,766	0,498	7/20
	IoU	**0,493**	0,490	0,416	0,225	−0,003	0,851	0,571	8/20
PolypGen (gộp)	Dice	**0,719**	0,715	0,683	0,484	−0,004	0,551	0,246	7/20
	IoU	**0,665**	0,663	0,616	0,402	−0,003	0,605	0,312	7/20

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

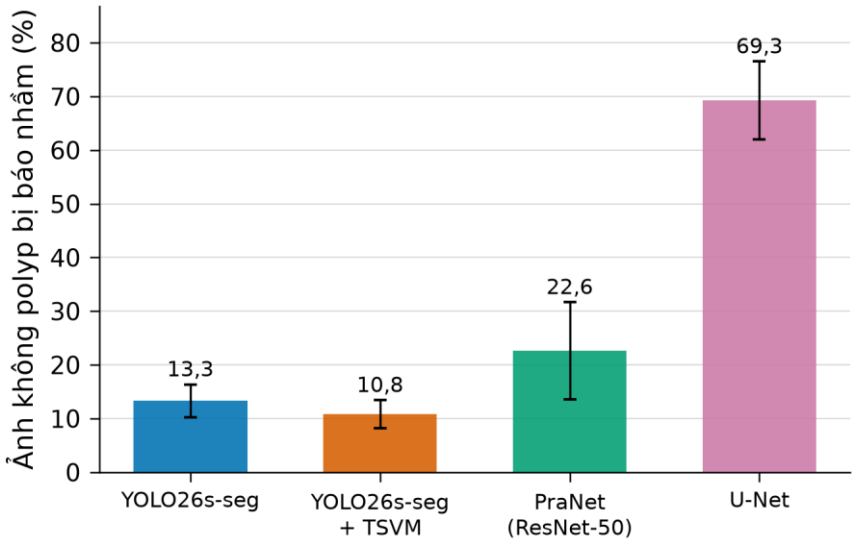


Hình 6.7: Dice theo pixel của bốn mô hình trên bốn bộ dữ liệu ngoài. Thanh lỗi là ±1 độ lệch chuẩn qua 20 seed.

Bốn điểm rút ra:

1. **Mọi mô hình đều giảm mạnh khi gặp dữ liệu khác nguồn.** Dice của TSVM giảm từ 0,854 trên tập kiểm định xuống 0,53–0,77; tỉ lệ ảnh bị bỏ sót hoàn toàn tăng từ khoảng 5% lên 11–39%. ETIS-Larib khó nhất vì nhiều polyp nhỏ.
2. **TSVM không tổng quát hóa tốt hơn mô hình gốc.** Trên CVC-ClinicDB và CVC-ColonDB, TSVM **thấp hơn có ý nghĩa thống kê** (Dice −0,016 và −0,030, cả hai kiểm định $p < 0{,}01$) và bỏ sót trọn ảnh nhiều hơn (+2,1 và +3,8 điểm phần trăm); trên ETIS-Larib và PolypGen, hai mô hình không khác biệt có ý nghĩa.
3. **Hai mô hình YOLO tốt hơn PraNet và U-Net trên dữ liệu ngoài.** TSVM cao hơn PraNet ở cả bốn bộ ($p < 0{,}05$), dù trên tập kiểm định Dice ngang nhau; một phần có thể do YOLO được huấn luyện với tăng cường dữ liệu.
4. **Độ ổn định không còn:** độ lệch chuẩn giữa các seed của TSVM không nhỏ hơn mô hình gốc, thậm chí lớn hơn ở CVC-ClinicDB và CVC-ColonDB.

Trên PolypGen, chất lượng khác nhau rất nhiều giữa các trung tâm (Dice của TSVM từ 0,47 ở C5 đến 0,86 ở C3), cho thấy thiết bị và cách thu ảnh ảnh hưởng mạnh hơn nhiều so với lựa chọn giữa hai mô hình YOLO. Chỉ ở C6 (83 ảnh) TSVM thấp hơn mô hình gốc với cả hai kiểm định, cần thận trọng vì có nhiều phép so sánh.



Hình 6.8: Tỉ lệ ảnh không có polyp của PolypGen (125 ảnh) bị báo nhầm.

Trên 125 ảnh không có polyp của PolypGen, tỉ lệ báo nhầm là 13,3% với mô hình gốc, 10,8% với TSVM, 22,6% với PraNet và 69,3% với U-Net. TSVM báo nhầm ít hơn mô hình gốc 2,5 điểm phần trăm ($p = 0{,}011$; Wilcoxon $p = 0{,}018$), chủ yếu ở C2 và C4; đây là chỉ số phụ nên chỉ là dấu hiệu tích cực cần kiểm chứng thêm. Tỉ lệ này cao hơn nhiều so với khoảng 1% trên 40 ảnh nền của tập kiểm định, cho thấy ảnh nền của BG20 chưa đủ đa dạng.

6.8. THẢO LUẬN

Trả lời câu hỏi nghiên cứu. Trên tập kiểm định, TSVM có mask mAP50–95 cao hơn 0,0043, cao hơn ở 13/20 seed, nhưng chưa có ý nghĩa thống kê ($p = 0{,}061$); đánh giá theo pixel và topology cho cùng kết luận, còn trên dữ liệu ngoài TSVM không tốt hơn và thấp hơn có ý nghĩa ở hai trong bốn bộ. Độ lệch chuẩn giữa các seed nhỏ hơn khoảng 39% (Levene $p = 0{,}026$, thăm dò) nhưng chỉ ở epoch tốt nhất. Trên 5 seed, bỏ bất kỳ thành phần nào cũng làm trung bình thấp hơn bản đầy đủ, bỏ cổng giảm ít nhất (−0,0036), chỉ giữ nhánh hình dạng giảm nhiều nhất (−0,0110), không chênh lệch nào có ý nghĩa thống kê. Tham số tăng 7,2%, GFLOPs tăng 1,5%, nhưng độ trễ tăng khoảng 1,5 lần trên GPU, 2,1 lần trên CPU và thời gian huấn luyện khoảng 2 lần, gần như hoàn toàn do nhánh SS2D.

Vì sao mức cải thiện nhỏ (các giải thích hợp lý nhưng chưa kiểm chứng riêng):

– **Phạm vi thay đổi hẹp:** chỉ tầng 10 ở mức 20 × 20 thay đổi, C2PSA vốn đã có chú ý toàn cục, và ở 400 vị trí lợi thế chi phí tuyến tính của SSM gần như không phát huy.

– **Khởi tạo bất lợi:** khoảng một triệu tham số mới khởi tạo ngẫu nhiên, trong khi C2PSA được tiền huấn luyện trên COCO, và chỉ có 1.040 ảnh huấn luyện; điều này cũng có thể giải thích kết quả trên dữ liệu ngoài, vì khối mới chỉ học từ ảnh Kvasir.
– **Chỉ số gần bão hòa:** mAP50 của cả hai mô hình đã khoảng 0,91, dư địa chủ yếu ở mức khớp biên.

Giới hạn của thực nghiệm: không có tập kiểm thử độc lập (dữ liệu ngoài bù một phần); chỉ một bộ dữ liệu huấn luyện; PraNet, U-Net có thiết lập khác; nhiều phép so sánh không hiệu chỉnh, nên các kết quả $p < 0{,}05$ ở chỉ số phụ chỉ là gợi ý; độ trễ đo trên một máy, một checkpoint; tách thành phần chỉ có 5 seed; topology chỉ đo trên tập kiểm định với nhãn đa giác vốn không có lỗ.

6.9. KẾT CHƯƠNG

Qua 20 cặp lượt huấn luyện, mask mAP50–95 tăng 0,0043 (0,7189 lên 0,7232), cao hơn ở 13/20 seed nhưng **chưa có ý nghĩa thống kê**; độ dao động giữa các seed nhỏ hơn ở epoch tốt nhất (thăm dò). TSVM có Dice ngang PraNet, IoU cao hơn PraNet nhưng topology kém PraNet rõ rệt; độ trễ tăng khoảng 1,5–2,1 lần dù số phép tính gần như không đổi; trên dữ liệu ngoài TSVM không tốt hơn mô hình gốc. Với cách tích hợp và dữ liệu hiện có, khối C2TSVMamba chưa mang lại cải thiện đủ rõ để bù cho chi phí suy luận tăng thêm.

KẾT LUẬN

1. KẾT QUẢ ĐẠT ĐƯỢC

Đề tài đã thực hiện đủ chu trình đặt ra trong đề cương:

- **Khảo sát và lý thuyết:** hệ thống hóa các hướng phân đoạn polyp từ CNN, Transformer, họ YOLO đến Mamba và VMamba; chỉ ra khoảng trống về việc đưa VMamba vào YOLO26-seg cho phân đoạn polyp có đánh giá nhiều seed.
- **Dữ liệu:** xây dựng bộ BG20 gồm 1.200 ảnh (1.000 ảnh polyp Kvasir-SEG theo đúng danh sách chia 880/120, 200 ảnh nền), chuyển mặt nạ sang nhãn đa giác có kiểm tra toàn vẹn.
- **Mô hình:** thiết kế và cài đặt khối C2TSVMamba (SS2D bốn hướng, nhánh hình dạng, cổng điều biến) thay C2PSA ở tầng 10, chỉ dùng PyTorch, có ba chế độ tách thành phần; phát hiện và sửa lỗi tràn số khi lưu checkpoint FP16.
- **Thực nghiệm:** 40 lượt chính (2 mô hình × 20 seed), 15 lượt tách thành phần (3 biến thể × 5 seed), PraNet và U-Net mỗi mô hình 20 seed; đủ các độ đo trong đề cương, thêm chỉ số topology; đánh giá trên bốn bộ dữ liệu ngoài với 2.725 ảnh.
- **Ứng dụng:** PolypVision gồm dịch vụ suy luận FastAPI, website Spring Boot lưu lịch sử trong MongoDB và ứng dụng di động Flutter, cho phép so sánh hai mô hình trên cùng ảnh.

2. ĐÁNH GIÁ KẾT QUẢ

CH1. Mask mAP50–95 trung bình tăng từ 0,7189 lên 0,7232 ($\Delta = +0{,}0043$), mô hình tích hợp cao hơn ở 13/20 seed, nhưng **chưa có ý nghĩa thống kê** (KTC 95% [−0,0002; 0,0088], $p = 0{,}061$). Dice theo pixel (0,854 so với 0,849) và chỉ số topology cho cùng kết luận. Độ lệch chuẩn giữa các seed nhỏ hơn khoảng 39% (Levene $p = 0{,}026$, thăm dò, chỉ đúng ở epoch tốt nhất).

CH2. Trên 5 seed, cả ba biến thể bỏ bớt thành phần đều có trung bình thấp hơn bản đầy đủ: bỏ cổng −0,0036, chỉ giữ SS2D −0,0068, chỉ giữ nhánh hình dạng −0,0110. Theo xu hướng, hai nhánh bổ sung cho nhau và cổng góp thêm một phần nhỏ; không chênh lệch nào có ý nghĩa thống kê.

CH3. Số tham số tăng 7,2%, số phép tính tăng khoảng 1,5%; độ trễ tăng khoảng 1,5 lần trên GPU và 2,1 lần trên CPU, thời gian huấn luyện khoảng gấp đôi, chủ yếu do phép quét SS2D viết bằng PyTorch thuần.

CH4. Cả hai checkpoint chạy được trong PolypVision trên Web và di động. Trên dữ liệu ngoài, mọi mô hình đều giảm chất lượng rõ (Dice của mô hình tích hợp 0,53–0,77); mô hình tích hợp **không tổng quát hóa tốt hơn** mô hình gốc, thấp hơn có ý nghĩa trên CVC-ClinicDB và CVC-ColonDB, chỉ báo nhầm ít hơn trên ảnh không polyp của PolypGen (10,8% so với 13,3%). Hai mô hình YOLO vẫn tốt hơn PraNet và U-Net trên dữ liệu ngoài.

Tổng hợp: đề tài hoàn thành các mục tiêu về quy trình; còn mục tiêu cải thiện chất lượng mặt nạ **chưa đạt ở mức có thể khẳng định**: khối C2TSVMamba chỉ cho xu hướng tăng nhỏ, chưa bù được chi phí suy luận tăng thêm và không giữ được trên dữ liệu khác nguồn. Đề tài báo cáo trung thực kết quả này thay vì chọn một lần huấn luyện có lợi.

3. HẠN CHẾ

- Không có tập kiểm thử riêng; tập kiểm định nhỏ (120 ảnh polyp, 127 đối tượng); dữ liệu huấn luyện từ một nguồn, ảnh nền chưa đa dạng.
- Hai mươi seed chỉ phản ánh dao động do huấn luyện; ước tính cần khoảng 42 cặp để có lực kiểm định 80% với mức chênh lệch quan sát được. Thí nghiệm tách thành phần chỉ có 5 seed, các biến thể khác số tham số. Các phép so sánh phụ không được hiệu chỉnh.
- Chỉ thử một vị trí tích hợp (tầng 10), một cỡ mô hình (*s*), một độ phân giải (640); phần tham số mới khởi tạo ngẫu nhiên trong khi C2PSA được tiền huấn luyện trên COCO; phép quét chậm.
- PraNet và U-Net huấn luyện ở độ phân giải khác, không tăng cường dữ liệu, nên so sánh chỉ mang tính tham khảo.
- Ứng dụng chỉ xử lý ảnh tĩnh, chưa có HTTPS và giới hạn tần suất gọi API, ứng dụng di động chưa có tài khoản; không phải thiết bị y tế.

4. HƯỚNG PHÁT TRIỂN

1. **Đánh giá chặt chẽ hơn:** thêm tập kiểm thử độc lập theo bệnh nhân hoặc trung tâm, tăng số seed hoặc kiểm định chéo, mở rộng ablation lên 20 seed kèm biến thể cùng số tham số.

2. **Tổng quát hóa:** huấn luyện trên nhiều nguồn ảnh, đa dạng hóa ảnh nền, tiền huấn luyện khối VMamba trước khi ghép.
3. **Vị trí tích hợp khác:** đặt khối VMamba ở neck hoặc ở nhiều tầng, như đề cương gợi ý.
4. **Tăng tốc:** dùng kernel selective-scan CUDA, lượng tử hóa để chạy trên thiết bị biên.
5. **Độ đo biên:** Hausdorff 95, Boundary F1, clDice.
6. **Ứng dụng:** xử lý video thời gian thực, bảo mật đường truyền, đồng bộ tài khoản giữa Web và di động, thu thập phản hồi của bác sĩ.

TÀI LIỆU THAM KHẢO

[1] F. Bray, M. Laversanne, H. Sung, J. Ferlay, R. L. Siegel, I. Soerjomataram, and A. Jemal, "Global cancer statistics 2022: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries," *CA Cancer J. Clin.*, vol. 74, no. 3, pp. 229–263, 2024, doi: 10.3322/caac.21834.

[2] A. G. Zauber *et al.*, "Colonoscopic polypectomy and long-term prevention of colorectal-cancer deaths," *N. Engl. J. Med.*, vol. 366, no. 8, pp. 687–696, Feb. 2012, doi: 10.1056/NEJMoa1100370.

[3] S. Zhao *et al.*, "Magnitude, risk factors, and factors associated with adenoma miss rate of tandem colonoscopy: A systematic review and meta-analysis," *Gastroenterology*, vol. 156, no. 6, pp. 1661–1674.e11, 2019, doi: 10.1053/j.gastro.2019.01.260.

[4] D. A. Corley *et al.*, "Adenoma detection rate and risk of colorectal cancer and death," *N. Engl. J. Med.*, vol. 370, no. 14, pp. 1298–1306, 2014, doi: 10.1056/NEJMoa1309086.

[5] D. Jha *et al.*, "Kvasir-SEG: A segmented polyp dataset," in *MultiMedia Modeling (MMM 2020)* (Lecture Notes in Computer Science, vol. 11962). Cham, Switzerland: Springer, 2020, pp. 451–462, doi: 10.1007/978-3-030-37734-2_37.

[6] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," 2026, *arXiv:2606.03748*, doi: 10.48550/arXiv.2606.03748.

[7] Y. Liu *et al.*, "VMamba: Visual state space model," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 103031–103063, doi: 10.52202/079017-3273.

[8] K. Pogorelov *et al.*, "KVASIR: A multi-class image dataset for computer aided gastrointestinal disease detection," in *Proc. 8th ACM Multimedia Syst. Conf. (MMSys)*, Taipei, Taiwan, 2017, pp. 164–169, doi: 10.1145/3083187.3083212.

[9] D.-P. Fan *et al.*, "PraNet: Parallel reverse attention network for polyp segmentation," in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2020* (Lecture Notes in Computer Science,

vol. 12266). Cham, Switzerland: Springer, 2020, pp. 263–273, doi: 10.1007/978-3-030-59725-2_26.

[10] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015* (Lecture Notes in Computer Science, vol. 9351). Cham, Switzerland: Springer, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[11] J. Mei *et al.*, "A survey on deep learning for polyp segmentation: Techniques, challenges and future trends," *Vis. Intell.*, vol. 3, no. 1, Art. no. 1, 2025, doi: 10.1007/s44267-024-00071-w.

[12] S. Ali *et al.*, "A multi-centre polyp detection and segmentation dataset for generalisability assessment," *Sci. Data*, vol. 10, no. 1, Art. no. 75, 2023, doi: 10.1038/s41597-023-01981-y.

[13] J. Bernal, F. J. Sánchez, G. Fernández-Esparrach, D. Gil, C. Rodríguez, and F. Vilariño, "WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation vs. saliency maps from physicians," *Comput. Med. Imaging Graph.*, vol. 43, pp. 99–111, Jul. 2015, doi: 10.1016/j.compmedimag.2015.02.007.

[14] N. Tajbakhsh, S. R. Gurudu, and J. Liang, "Automated polyp detection in colonoscopy videos using shape and context information," *IEEE Trans. Med. Imag.*, vol. 35, no. 2, pp. 630–644, Feb. 2016, doi: 10.1109/TMI.2015.2487997.

[15] J. Silva, A. Histace, O. Romain, X. Dray, and B. Granado, "Toward embedded detection of polyps in WCE images for early diagnosis of colorectal cancer," *Int. J. Comput. Assist. Radiol. Surg.*, vol. 9, no. 2, pp. 283–293, 2014, doi: 10.1007/s11548-013-0926-3.

[16] B. Dong, W. Wang, D.-P. Fan, J. Li, H. Fu, and L. Shao, "Polyp-PVT: Polyp segmentation with pyramid vision transformers," *CAAI Artif. Intell. Res.*, vol. 2, Art. no. 9150015, 2023, doi: 10.26599/AIR.2023.9150015.

[17] N. T. Duc, N. T. Oanh, N. T. Thuy, T. M. Triet, and D. V. Sang, "ColonFormer: An efficient transformer based method for colon polyp segmentation," *IEEE Access*, vol. 10, pp. 80575–80586, 2022, doi: 10.1109/ACCESS.2022.3195241.

[18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[19] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, “YOLACT: Real-time instance segmentation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Seoul, South Korea, 2019, pp. 9156–9165, doi: 10.1109/ICCV.2019.00925.

[20] A. Wang *et al.*, “YOLOv10: Real-time end-to-end object detection,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 107984–108011, doi: 10.52202/079017-3429.

[21] I. Pacal and D. Karaboga, “A robust real-time deep learning based automatic polyp detection system,” *Comput. Biol. Med.*, vol. 134, Art. no. 104519, Jul. 2021, doi: 10.1016/j.compbiomed.2021.104519.

[22] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. [Online]. Available: https://openreview.net/forum?id=uYLFoz1vlAC

[23] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” in *Proc. 1st Conf. Lang. Model. (COLM)*, Philadelphia, PA, USA, 2024. [Online]. Available: https://openreview.net/forum?id=tEYskw1VY2

[24] J. Ma, F. Li, and B. Wang, “U-Mamba: Enhancing long-range dependency for biomedical image segmentation,” 2024, *arXiv:2401.04722*, doi: 10.48550/arXiv.2401.04722.

[25] J. Liu *et al.*, “Swin-UMamba: Mamba-based UNet with ImageNet-based pretraining,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024* (Lecture Notes in Computer Science, vol. 15009). Cham, Switzerland: Springer, 2024, pp. 615–625, doi: 10.1007/978-3-031-72114-4_59.

[26] Z. Xu *et al.*, “Polyp-Mamba: Polyp segmentation with visual Mamba,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024* (Lecture Notes in Computer Science, vol. 15008). Cham, Switzerland: Springer, 2024, pp. 510–521, doi: 10.1007/978-3-031-72111-3_48.

[27] R. Fu, S. Hu, X. Zheng, C. Tang, and X. Liu, “PolyMamba: Spatial-prior guided Mamba for polyp segmentation with high-frequency enhancement,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025* (Lecture Notes in Computer Science, vol. 15970). Cham, Switzerland: Springer, 2025, pp. 455–465, doi: 10.1007/978-3-032-05141-7_44.

[28] D. Adame *et al.*, “Topo-VM-UNetV2: Encoding topology into vision Mamba UNet for polyp segmentation,” in *Proc. IEEE 38th Int. Symp. Comput.-Based Med. Syst. (CBMS)*, 2025, pp. 258–263, doi: 10.1109/CBMS65348.2025.00059.

[29] Z. Wang, C. Li, H. Xu, X. Zhu, and H. Li, “Mamba YOLO: A simple baseline for object detection with state space model,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 8, 2025, pp. 8205–8213, doi: 10.1609/aaai.v39i8.32885.

[30] S.-W. Chan *et al.*, “MambaPSA: A Mamba-based replacement for C2PSA in YOLO26,” 2026, *arXiv:2607.12681*.

[31] X. Bouthillier *et al.*, “Accounting for variance in machine learning benchmarks,” in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html

[32] P. N. Lan, N. S. An, D. V. Hang, D. V. Long, T. Q. Trung, N. T. Thuy, and D. V. Sang, “NeoUNet: Towards accurate colon polyp segmentation and neoplasm detection,” in *Advances in Visual Computing (ISVC 2021)*, Lecture Notes in Computer Science, Cham, Switzerland: Springer, 2021, pp. 15–28, doi: 10.1007/978-3-030-90436-4_2.

[33] H. D. Viet *et al.*, “Validation of YOLOv8 algorithm in detecting colon polyps in endoscopy videos,” *J. Med. Artif. Intell.*, vol. 8, Art. no. 35, 2025, doi: 10.21037/jmai-24-338.

[34] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.322.

[35] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE Conf.*

Comput. Vis. Pattern Recognit. (CVPR), Honolulu, HI, USA, 2017, pp. 936–944, doi: 10.1109/CVPR.2017.106.

[36] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 8759–8768, doi: 10.1109/CVPR.2018.00913.

[37] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, and W. Huang, "TOOD: Task-aligned one-stage object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Montreal, QC, Canada, 2021, pp. 3490–3499, doi: 10.1109/ICCV48922.2021.00349.

[38] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 7, 2020, pp. 12993–13000, doi: 10.1609/aaai.v34i07.6999.

[39] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.

[40] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[41] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014* (Lecture Notes in Computer Science, vol. 8693). Cham, Switzerland: Springer, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[42] G. Jocher, J. Qiu, and A. Chaurasia, *Ultralytics YOLO*, version 8.4.127. Ultralytics, 2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[43] A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[44] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 35971–35983, doi: 10.52202/068431-2607.

[45] G. E. Blelloch, “Prefix sums and their applications,” School Comput. Sci., Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-CS-90-190, Nov. 1990.

[46] W. D. Hillis and G. L. Steele Jr., “Data parallel algorithms,” *Commun. ACM*, vol. 29, no. 12, pp. 1170–1183, Dec. 1986, doi: 10.1145/7902.7903.

[47] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010, doi: 10.1007/s11263-009-0275-4.

[48] Student, “The probable error of a mean,” *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908, doi: 10.2307/2331554.

[49] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[50] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006. [Online]. Available: https://www.jmlr.org/papers/v7/demsar06a.html

[51] D. Lakens, “Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs,” *Front. Psychol.*, vol. 4, Art. no. 863, 2013, doi: 10.3389/fpsyg.2013.00863.

[52] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[53] D. Jha, “Kvasir-SEG data (polyp segmentation & detection),” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/debeshjha1/kvasirseg (accessed Oct. 9, 2026).

[54] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979, doi: 10.1109/TSMC.1979.4310076.

[55] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” *Comput. Vis., Graph., Image Process.*, vol. 30, no. 1, pp. 32–46, 1985, doi: 10.1016/0734-189X(85)90016-7.

[56] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,”

Cartographica, vol. 10, no. 2, pp. 112–122, 1973, doi: 10.3138/FM57-6770-U75U-7727.

[57] M. B. Brown and A. B. Forsythe, “Robust tests for the equality of variances,” *J. Amer. Statist. Assoc.*, vol. 69, no. 346, pp. 364–367, 1974, doi: 10.1080/01621459.1974.10482955.

PHỤ LỤC

PHỤ LỤC A. KẾT QUẢ TỪNG SEED

A.1. Mô hình gốc và mô hình đề xuất (20 seed)

Mỗi lượt lấy tại epoch đầu tiên đạt mask mAP50–95 cao nhất. P, R là Precision và Recall của mặt nạ tại cùng epoch. Cột "epoch" là epoch được chọn.

Bảng PL.1: Chỉ số mặt nạ của từng seed trên tập kiểm định

Seed	Gốc: mAP50–95	Gốc: mAP50	Gốc: P	Gốc: R	Gốc: epoch	TSVM: mAP50–95	TSVM: mAP50	TSVM: P	TSVM: R	TSVM: epoch	Δ mAP50–95
0	0,7366	0,9136	0,9535	0,8189	89	0,7245	0,9040	0,9312	0,8529	88	−0,0121
1	0,7165	0,8952	0,8533	0,8504	98	0,7274	0,9115	0,9437	0,8661	98	+0,0109
2	0,7138	0,9072	0,9139	0,8355	100	0,7259	0,9003	0,9220	0,8377	83	+0,0121
3	0,6941	0,9052	0,8786	0,8545	61	0,7213	0,9164	0,9188	0,8909	89	+0,0272
4	0,7274	0,9062	0,9387	0,8819	99	0,7197	0,9024	0,9018	0,8676	97	−0,0077
5	0,7350	0,9264	0,9318	0,8605	94	0,7285	0,9096	0,9013	0,8819	90	−0,0066
6	0,7153	0,9080	0,8865	0,8611	81	0,7254	0,9006	0,9223	0,8504	95	+0,0101
7	0,7145	0,9154	0,8983	0,8347	71	0,7065	0,8903	0,8595	0,8425	88	−0,0080
8	0,7318	0,9319	0,9114	0,8912	96	0,7339	0,9103	0,8883	0,8768	93	+0,0021
9	0,7253	0,9097	0,8568	0,8949	84	0,7329	0,9166	0,9288	0,8583	96	+0,0076
10	0,7045	0,8811	0,8780	0,8661	82	0,7198	0,9103	0,9542	0,8425	100	+0,0153
11	0,7299	0,9091	0,9253	0,8783	99	0,7226	0,9097	0,9168	0,8661	87	−0,0072
12	0,7056	0,8828	0,9019	0,8347	87	0,7157	0,9031	0,9162	0,8583	89	+0,0101
13	0,7315	0,9109	0,9348	0,8347	97	0,7306	0,9250	0,8926	0,8819	99	−0,0009
14	0,7223	0,9072	0,9014	0,8637	74	0,7211	0,9064	0,9231	0,8500	96	−0,0012
15	0,7176	0,9040	0,9171	0,8713	80	0,7262	0,9100	0,9347	0,8425	84	+0,0086
16	0,7125	0,8965	0,9213	0,8425	96	0,7237	0,9121	0,9027	0,8763	93	+0,0113
17	0,7274	0,9227	0,8615	0,9134	73	0,7301	0,9219	0,9147	0,8740	100	+0,0027
18	0,7022	0,9004	0,8875	0,8583	94	0,7082	0,8935	0,8762	0,8361	47	+0,0060
19	0,7136	0,9045	0,9303	0,8408	71	0,7195	0,9113	0,9403	0,8268	91	+0,0059
TB ± SD	0,7189 ± 0,0118	0,9069	0,9041	0,8594		0,7232 ± 0,0072	0,9083	0,9145	0,8590		+0,0043

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

A.2. Dice theo pixel của bốn mô hình (20 seed)

Bảng PL.2: Dice theo pixel trên 120 ảnh có polyp của tập kiểm định

Seed	YOLO26s-seg	TSVM	PraNet	U-Net
0	0,8727	0,8366	0,8547	0,7182
1	0,8624	0,8659	0,8635	0,7172
2	0,8394	0,8554	0,8619	0,6914
3	0,8232	0,8580	0,8475	0,7092
4	0,8736	0,8529	0,8431	0,6857
5	0,8468	0,8712	0,8503	0,6696

Seed	YOLO26s-seg	TSVM	PraNet	U-Net
6	0,8632	0,8349	0,8534	0,6844
7	0,8223	0,8293	0,8515	0,7313
8	0,8748	0,8552	0,8504	0,7080
9	0,8254	0,8636	0,8549	0,7322
10	0,8540	0,8647	0,8372	0,7255
11	0,8601	0,8715	0,8519	0,7446
12	0,8448	0,8656	0,8560	0,6898
13	0,8525	0,8651	0,8559	0,7413
14	0,8453	0,8421	0,8561	0,7092
15	0,8469	0,8372	0,8677	0,7039
16	0,8259	0,8562	0,8555	0,7367
17	0,8557	0,8636	0,8609	0,6861
18	0,8274	0,8443	0,8547	0,7236
19	0,8553	0,8468	0,8544	0,6852
TB ± SD	0,8486 ± 0,0171	0,8540 ± 0,0129	0,8541 ± 0,0068	0,7097 ± 0,0221

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

A.3. Thí nghiệm tách thành phần (5 seed)

Bảng PL.3: Mask mAP50–95 từng seed của năm cấu hình

Seed	Gốc (C2PSA)	TSVM đầy đủ	`vmamba_only`	`shape_only`	`shape_vmamba_no_guidance`
0	0,7366	0,7245	0,7145	0,7107	0,7160
1	0,7165	0,7274	0,7389	0,7091	0,7275
2	0,7138	0,7259	0,7145	0,7049	0,7209
3	0,6941	0,7213	0,7115	0,7296	0,7172
4	0,7274	0,7197	0,7054	0,7095	0,7193
TB ± SD	0,7177 ± 0,0160	0,7238 ± 0,0032	0,7170 ± 0,0128	0,7128 ± 0,0097	0,7201 ± 0,0045
Giờ huấn luyện TB	2,32	4,61	4,58	2,41	4,62

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

PHỤ LỤC B. CẤU HÌNH HUẤN LUYỆN

Hai mô hình YOLO, cùng ba biến thể tách thành phần, dùng chung toàn bộ siêu tham số dưới đây. Chúng chỉ khác nhau ở tệp cấu hình mô hình (`model`). Mô hình gốc khởi tạo trực tiếp từ `yolo26s-seg.pt`. Mô hình đề xuất và các biến thể được dựng từ tệp YAML rồi nạp trọng số từ cùng tệp này, chỉ ở các tầng trùng tên và trùng kích thước (mục 3.5.1).

Bảng PL.4: Siêu tham số huấn luyện (theo `args.yaml` của 40 lượt chính)

Nhóm	Tham số	Giá trị
Chung	`epochs`, `batch`, `imgsz`	100; 8; 640

Nhóm	Tham số	Giá trị
	workers, device	2; GPU 0
	patience	100 (không dừng sớm)
	amp	False (huấn luyện FP32)
	deterministic	True
	seed	0–19 (lượt chính), 0–4 (tách thành phần)
Tối ưu	optimizer	AdamW
	lr0, lrf	0,001; 0,01
	momentum, weight_decay	0,937; 0,0005
	warmup_epochs, cos_lr	5; False
Tăng cường dữ liệu	mosaic, close_mosaic	1,0; tắt ở 10 epoch cuối
	mixup	0
	fliplr, flipud, degrees	0,5; 0; 0
	scale, translate	0,5; 0,1
	hsv_h, hsv_s, hsv_v	0,015; 0,7; 0,4
Phân đoạn và đánh giá	overlap_mask, mask_ratio	True; 4
	iou, max_det	0,7; 300
	rect, cache	False; False

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

PHỤ LỤC C. TỔ CHỨC MÃ NGUỒN VÀ TỆP KẾT QUẢ

Bảng PL.5: Các thư mục chính của kho mã

Thư mục	Nội dung
01_du_lieu/	Bộ BG20 ở định dạng YOLO, danh sách chia train.txt/val.txt, script tiền xử lý
02_ma_nguon/ultralytics_TSVM_FP32/	Ultralytics 8.4.127 có thêm khối C2TSVMamba (nn/modules/topology_shape_vmamba.py) và bản vá lưu checkpoint FP32
02_ma_nguon/notebook_kaggle/	Notebook huấn luyện trên Kaggle, gồm cả notebook tách thành phần
03_ket_qua_huan_luyen/	40 lượt chính và 15 lượt tách thành phần; mỗi lượt có args.yaml, results.csv, weights/
05_so_sanh_mo_hinh_khac/	Trọng số PraNet và U-Net (20 seed mỗi mô hình)

Thư mục	Nội dung
00_tai_lieu_tong_quan/scripts/	Script tính lại số liệu, đánh giá theo pixel, topology, FLOPs và độ trễ GPU, tách thành phần
00_tai_lieu_tong_quan/bang/	Các bảng CSV kết quả dùng trong báo cáo
08_du_lieu_thuc_te/	Script và kết quả đánh giá trên bốn bộ dữ liệu ngoài
06_bao_cao/hinh/	Hình của báo cáo và script vẽ hình

Bảng PL.6: Script tạo ra số liệu trong báo cáo

Script	Đầu vào	Đầu ra	Dùng ở
tinh_lai_so_lieu.py	results.csv, args.yaml, ảnh dữ liệu	bang/01–09_*.csv	Ch.4, Ch.6 (mục 6.2)
danh_gia_pixel.py	Checkpoint 4 mô hình × 20 seed, 160 ảnh kiểm định	bang/10–12_*.csv	Ch.6 (mục 6.3)
danh_gia_topology.py	Như trên	bang/13–15_*.csv	Ch.6 (mục 6.4)
do_flops_gpu.py	Checkpoint seed 0 của hai mô hình	bang/16*_*.csv	Ch.6 (mục 6.5)
tinh_ablation.py	results.csv, args.yaml, best.pt của 25 lượt	bang/17*_*.csv, bang/18_*.csv	Ch.6 (mục 6.6)
08_du_lieu_thuc_te/scripts/	Checkpoint 4 mô hình × 20 seed, 4 bộ dữ liệu ngoài	08_du_lieu_thuc_te/bang/20seed/*.csv	Ch.6 (mục 6.7)
ve_hinh.py, ve_so_do.py	Các bảng CSV	Hình PNG 300 dpi và PDF	Toàn báo cáo

PHỤ LỤC D. HƯỚNG DẪN CHẠY LẠI

Các lệnh chạy từ thư mục gốc của kho mã. Script chỉ đọc dữ liệu và kết quả huấn luyện, chỉ ghi vào thư mục bang/ và hinh/. Các bước cần GPU dùng môi trường Python có PyTorch hỗ trợ CUDA.

```
# 1. Tính lại số liệu tổng hợp từ 40 lượt chính
python 00_tai_lieu_tong_quan/scripts/tinh_lai_so_lieu.py

# 2. Đánh giá theo pixel và topology của 4 mô hình (cần GPU)
python 00_tai_lieu_tong_quan/scripts/danh_gia_pixel.py
python 00_tai_lieu_tong_quan/scripts/danh_gia_topology.py

# 3. FLOPs và độ trễ GPU (cần GPU)
python 00_tai_lieu_tong_quan/scripts/do_flops_gpu.py
```

```
# 4. Tách thành phần, có đếm tham số từ checkpoint (cần PyTorch)
python 00_tai_lieu_tong_quan/scripts/tinh_ablation.py --torch

# 5. Vẽ lại toàn bộ hình
python 06_bao_cao/hinh/scripts/ve_hinh.py
```

Để huấn luyện lại một lượt, mở notebook trong `02_ma_nguon/notebook_kaggle/` trên Kaggle (GPU), đặt seed và tệp cấu hình mô hình cần chạy, rồi tải thư mục kết quả về `03_ket_qua_huan_luyen/`. Notebook đặt `deterministic=True` và khóa seed cho mọi thư viện. Trong đề tài, lượt `shape_only` seed 3 và seed 4 mỗi lượt có hai bản kết quả với thời gian huấn luyện khác nhau. Hai bản `results.csv` giống hệt nhau ở mọi cột chỉ số qua cả 100 epoch, cho thấy với cùng phiên bản thư viện và cùng loại GPU, kết quả lặp lại được.

PHỤ LỤC E. PHÂN CÔNG THEO HẠNG MỤC CÔNG VIỆC

Bảng PL.7: Phân công theo hạng mục công việc (C: chính, H: hỗ trợ)

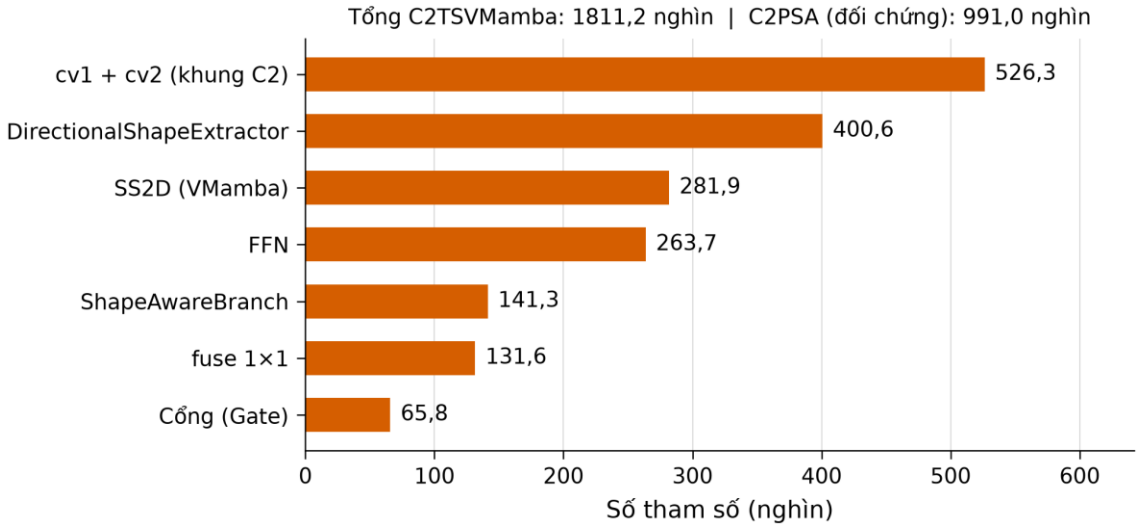
Hạng mục	**Lê Đức Lương**	**Phùng Tuấn Huy**	**Trần Mạnh Toàn**
Khảo sát tài liệu, viết Chương 1	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Cơ sở lý thuyết, viết Chương 2	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Thu thập và tiền xử lý dữ liệu, viết Chương 4	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Thiết kế và cài đặt khối C2TSVMamba, viết Chương 3	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Huấn luyện Baseline và TSVM trên Kaggle	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Huấn luyện PraNet, U-Net	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Thí nghiệm tách thành phần (ablation)	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Phân tích thống kê, vẽ hình, viết Chương 6	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Dịch vụ suy luận FastAPI	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Website Spring Boot, MongoDB	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Ứng dụng di động Flutter	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Viết Chương 5	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Mở đầu, Kết luận, định dạng báo cáo	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]
Slide và chuẩn bị bảo vệ	[ĐIỀN]	[ĐIỀN]	[ĐIỀN]

PHỤ LỤC F. CHI TIẾT KHỐI C2TSVMAMBA

F.1. Truy vết kích thước tensor

Bảng PL.8: Kích thước tensor qua khối C2TSVMamba (một ảnh 640×640)

Bước	**Phép toán**	**Kích thước ra**
Vào tầng 10	x	512×20×20
cv1, chia đôi	Conv 1×1, split	a: 256×20×20; b: 256×20×20
SS2D, bước 1–2	in_proj, DWConv 3×3	U, Z: 256×20×20 mỗi phần
SS2D, bước 3	Trải phẳng	4 chuỗi, mỗi chuỗi 256×400
SS2D, bước 4	x_proj, dt_proj	Δ: 256×400; B, C: 16×400 (mỗi hướng)
SS2D, bước 5	$\bar{A}$, ΔBu, quét	256×16×400 (đệm lên 512 khi quét)
SS2D, bước 5–7	Đọc ra, gộp 4 hướng, cổng Z, out_proj	F_M: 256×20×20
Nhánh hình dạng	(3.7)–(3.9)	S: 256×20×20
Cổng, điều biến	(3.10)	G, F'_M: 256×20×20
fuse, FFN, nối tắt	(3.11)	b': 256×20×20
Ghép, cv2	Concat, Conv 1×1	y: 512×20×20



Hình PL.1: Số tham số của từng thành phần trong khối C2TSVMamba. Tiêu đề hình ghi tổng của C2TSVMamba và của C2PSA để so sánh.

F.2. Ví dụ phép quét song song

Xét chuỗi $L = 4$ với $a = (0{,}5;\ 0{,}8;\ 0{,}5;\ 0{,}9)$ và $b = (1;\ 2;\ 0;\ 1)$. Tính tuần tự theo (3.13): $h_1 = 1;\ h_2 = 0{,}8 \cdot 1 + 2 = 2{,}8;\ h_3 = 0{,}5 \cdot 2{,}8 + 0 = 1{,}4;$

$h_4 = 0{,}9 \cdot 1{,}4 + 1 = 2{,}26$. Thuật toán 3.1 ($K = 2$, không cần đệm) cho kết quả ở Bảng PL.9.

Bảng PL.9: Các vòng của phép quét song song trên ví dụ

Vòng	**Vị trí 1**	**Vị trí 2**	**Vị trí 3**	**Vị trí 4**
Ban đầu (a, b)	(0,5; 1)	(0,8; 2)	(0,5; 0)	(0,9; 1)
step = 1	(0,5; 1)	(0,4; 0,8·1 + 2 = **2,8**)	(0,4; 0,5·2 + 0 = 1)	(0,45; 0,9·0 + 1 = 1)
step = 2	(0,5; **1**)	(0,4; **2,8**)	(0,2; 0,4·1 + 1 = **1,4**)	(0,18; 0,45·2,8 + 1 = **2,26**)

Sau vòng cuối, thành phần b là $(1;\ 2{,}8;\ 1{,}4;\ 2{,}26)$, trùng với kết quả tính tuần tự. Ở vòng step = 2, vị trí 4 kết hợp cặp của vị trí 2 (đã chứa thông tin vị trí 1, 2) với cặp của chính nó (đã chứa vị trí 3, 4), nên chỉ sau hai vòng đã gom đủ bốn phần tử.

Tổng khối lượng tính. Vòng thứ j xử lý $P - 2^j$ vị trí, nên tổng số phép kết hợp là $\sum_{j=0}^{K-1}(P - 2^j) = K \cdot P - (P - 1)$. Với $P = 512$, $K = 9$: mỗi chuỗi 4.097 phép kết hợp (tính tuần tự chỉ 399), mỗi ảnh có 16.384 chuỗi nên khoảng 67 triệu phép kết hợp, mỗi phép gồm hai phép nhân và một phép cộng. Nếu đặt khối ở P4 ($L = 1.600$) hay P3 ($L = 6.400$), số vòng tăng lên 11 và 13.

F.3. Ước tính chi phí tầng 10

Ước tính giải tích số phép nhân–cộng (MAC), tính tay theo kích thước từng lớp cho một ảnh 640×640. Đề tài không báo cáo GFLOPs đo bằng công cụ cho riêng tầng này, vì công cụ đếm tự động không đếm đúng phép quét tự viết.

Bảng PL.10: Ước tính chi phí phần xử lý chính của tầng 10. Không tính cv1, cv2 vì hai khối dùng chung, khoảng 210 triệu MAC.

Khối	**Thành phần**	**Triệu MAC**
PSABlock (C2PSA)	qkv 52,4; $QK^\top$ 20,5; nhân với V 41,0; proj 26,2; mã hóa vị trí 0,9; FFN 104,9	≈ 246
TSVMamba	SS2D: các phép chiếu và tích chập	≈ 112
	Nhánh hình dạng, cổng, fuse, FFN	≈ 399
	Cộng	**≈ 511**

Phép quét song song không tính được bằng MAC: khoảng 67 triệu phép kết hợp, tức khoảng 200 triệu phép toán từng phần tử. Khoản tăng thêm khoảng 0,27 tỉ MAC, trong khi toàn mô hình theo ghi chú của Ultralytics cỡ 37 GFLOPs, nên số phép tính toàn mô hình chỉ tăng khoảng 1–2%. Trên CPU (chỉ forward, 4 luồng, checkpoint seed 0), mỗi ảnh mất khoảng 270 ms với mô hình đối chứng và 577 ms với mô hình đề xuất.

F.4. Quy trình suy luận và huấn luyện của mô hình đề xuất

Ở mô hình đối chứng, quy trình giống hệt; chỉ khác bước 3 dùng C2PSA.

Thuật toán PL.1. Suy luận và huấn luyện YOLO26s-seg + C2TSVMamba

```
SUY LUẬN (một ảnh I)
1. Letterbox: co ảnh để cạnh dài bằng 640, giữ tỉ lệ, thêm viền xám
2. Backbone tầng 0–9: trích đặc trưng, ra 512×20×20 tại tầng 9
3. Tầng 10 – C2TSVMamba: cv1 → chia (a, b) → TSVMamba(b) theo (3.12) →
ghép → cv2
4. Neck tầng 11–22: FPN đi xuống, PAN đi lên → P3, P4, P5
5. Segment26, đầu một-một: với mỗi điểm neo dự đoán hộp, điểm lớp, 32
hệ số mặt nạ;
   Proto26 sinh 32 mặt nạ mẫu 160×160
6. Lấy 300 dự đoán có điểm cao nhất, bỏ các dự đoán dưới ngưỡng tin
cậy (không NMS)
7. Với mỗi dự đoán còn lại: tính Σ c_k·P_k, phóng về kích thước ảnh,
   nhị phân hóa (sigmoid > 0,5), rồi cắt theo hộp
8. Trả về danh sách (hộp, điểm tin cậy, mặt nạ)

MỘT BƯỚC HUẤN LUYỆN (lô ảnh đã tăng cường, epoch e)
1. Lan truyền xuôi như bước 2–5, cho cả hai đầu một-nhiều và một-một
2. Gán nhãn: TAL + STAL, top-10 cho đầu một-nhiều, top-7 rồi giữ 1 cho
đầu một-một
3. Tính mất mát mỗi đầu theo Bảng 3.3; tổng hợp theo (3.15) với α(e)
4. Lan truyền ngược, cập nhật trọng số bằng AdamW; cập nhật bản trung
bình động (EMA) của trọng số
5. Cuối epoch: kiểm định trên tập val bằng trọng số EMA, lưu best.pt
khi chỉ số tốt hơn
```

PHỤ LỤC G. API VÀ CẤU TRÚC LƯU TRỮ CỦA ỨNG DỤNG POLYPVISION

Bảng PL.11: Các API của dịch vụ suy luận

Phương thức	**Đường dẫn**	**Dùng cho**	**Chức năng**
GET	`/health`	Web	Trạng thái dịch vụ, thiết bị, danh sách mô hình

Phương thức	Đường dẫn	Dùng cho	Chức năng
POST	`/predict`	Web	Nhận ảnh (`image`) và tên mô hình (`modelType`), trả kết quả một mô hình
GET	`/api/v1/health`	Mobile	Trạng thái tổng hợp
GET	`/api/v1/models`	Mobile	Danh sách hai mô hình và trạng thái sẵn sàng
POST	`/api/v1/inference`	Mobile	Nhận ảnh (`image`) và khóa mô hình (`modelKey`), suy luận một mô hình
POST	`/api/v1/comparison`	Mobile	Nhận ảnh (`image`), chạy cả hai mô hình trên cùng ảnh

Bảng PL.12: Các trường chính trong kết quả suy luận của API `/predict`

Trường	Kiểu	Ý nghĩa
`detected`	logic	Có phát hiện polyp hay không
`label`	chuỗi	Tên lớp (`polyp`)
`confidence`	số thực	Độ tin cậy cao nhất trong các đối tượng phát hiện
`maskCount`	số nguyên	Số mặt nạ (số polyp) phát hiện được
`processingTimeMs`	số thực	Thời gian gọi mô hình, tính bằng mili giây
`resultImageBase64`	chuỗi	Ảnh JPEG chồng mặt nạ và hộp bao, mã hóa Base64

Ngoài các trường trên còn `success`, `modelType`, `originalFileName` và `message`. Các API cho Mobile dùng cấu trúc riêng: mỗi kết quả có `modelKey`, `modelName`, `modelFamily`, các trường suy luận như trên, ảnh kết quả đặt trong `overlay.data`, kèm `requestId`, kích thước ảnh gốc và một câu cảnh báo nghiên cứu; kết quả so sánh gồm hai phần `baseline`, `vmamba` và `totalProcessingTimeMs`. Khi lỗi, các API `/api/v1/*` trả cấu trúc JSON gồm `requestId`, `status`, `code`, `message`, `fieldErrors`. Nhánh Mobile có giới hạn số yêu cầu suy luận đồng thời và thời gian chờ tối đa; nhánh Web dùng khóa riêng cho từng mô hình. Spring Boot còn cung cấp bộ REST API riêng (`/api/v1/auth`, `/api/v1/diagnoses`, `/api/v1/comparisons`, `/api/v1/dashboard`, `/api/v1/me`).

Bảng PL.13: Tập `users` trong MongoDB

Trường	Ý nghĩa
`id`	Mã tài khoản
`fullName`, `email`	Họ tên và email; email là duy nhất

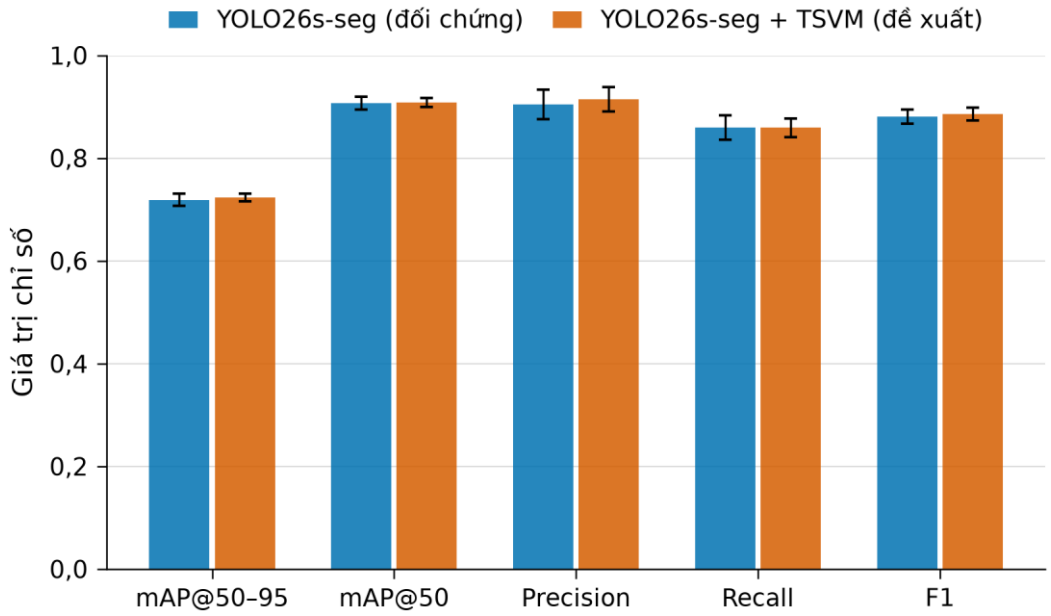
Trường	Ý nghĩa
passwordHash	Mật khẩu đã băm bằng BCrypt, không lưu mật khẩu gốc
role	USER hoặc ADMIN
enabled	Tài khoản đang hoạt động hay bị khóa
createdAt, updatedAt	Thời điểm tạo và cập nhật

Bảng PL.14: Tập diagnosis_history trong MongoDB

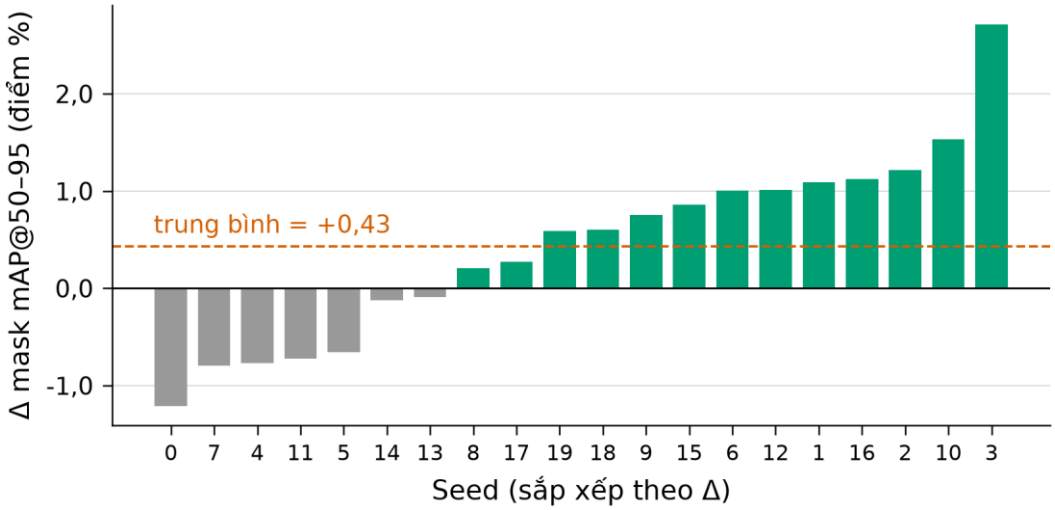
Trường	Ý nghĩa
id, userId	Mã bản ghi, tài khoản sở hữu
fileName	Tên tệp ảnh người dùng tải lên
modelType	Mô hình đã dùng
detected, label, confidence, maskCount	Kết quả suy luận; confidence lưu theo thang 0–100
processingTimeMs	Thời gian xử lý
imageWidth, imageHeight	Kích thước ảnh
originalImageUrl, resultImageUrl	Đường dẫn tới ảnh gốc và ảnh kết quả
createdAt	Thời điểm phân tích
comparisonGroupId, comparisonResult	Gắn hai bản ghi của cùng một lần so sánh với nhau

PHỤ LỤC H. KẾT QUẢ THỰC NGHIỆM BỔ SUNG

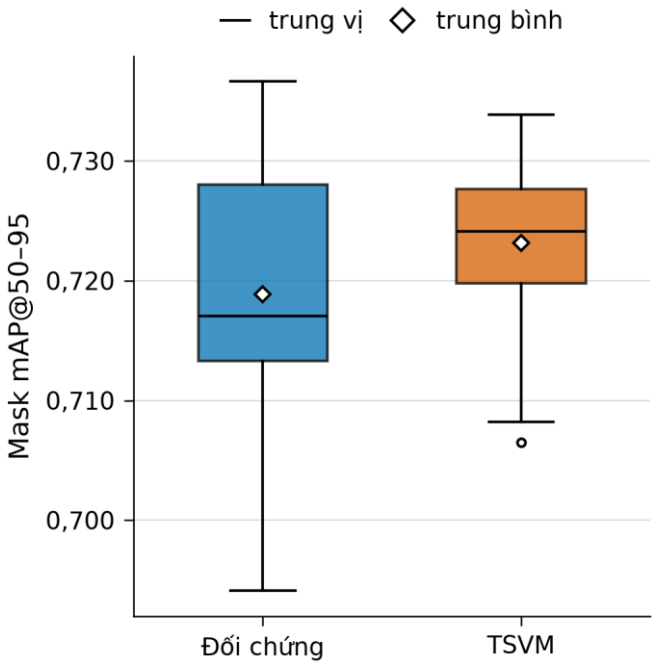
H.1. So sánh hai mô hình YOLO trên tập kiểm định



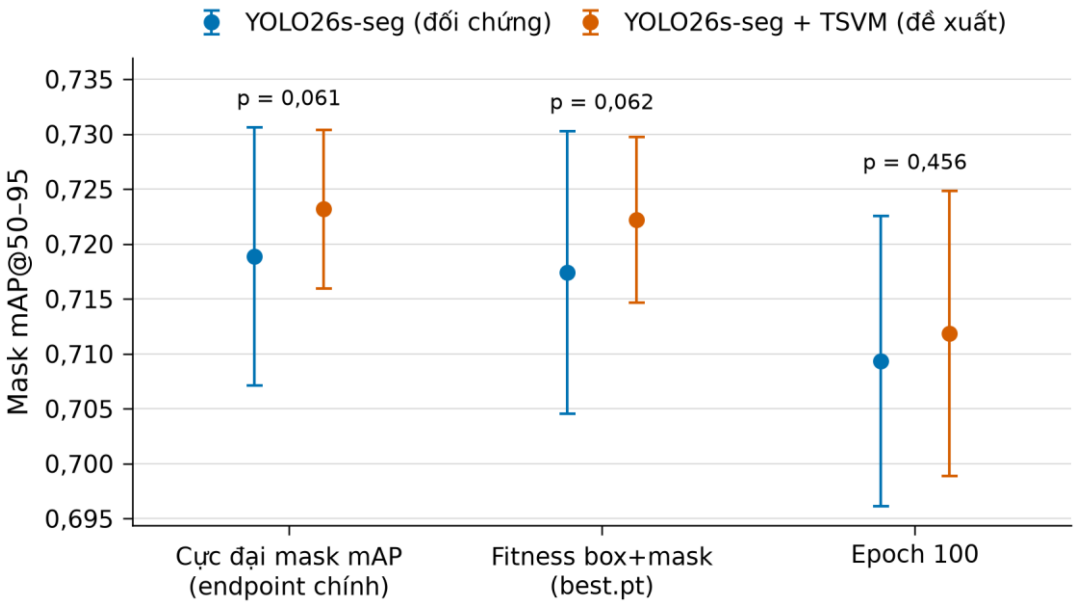
Hình PL.2: Trung bình các chỉ số mặt nạ của hai mô hình trên 20 seed. Thanh lỗi là ±1 độ lệch chuẩn qua các seed.



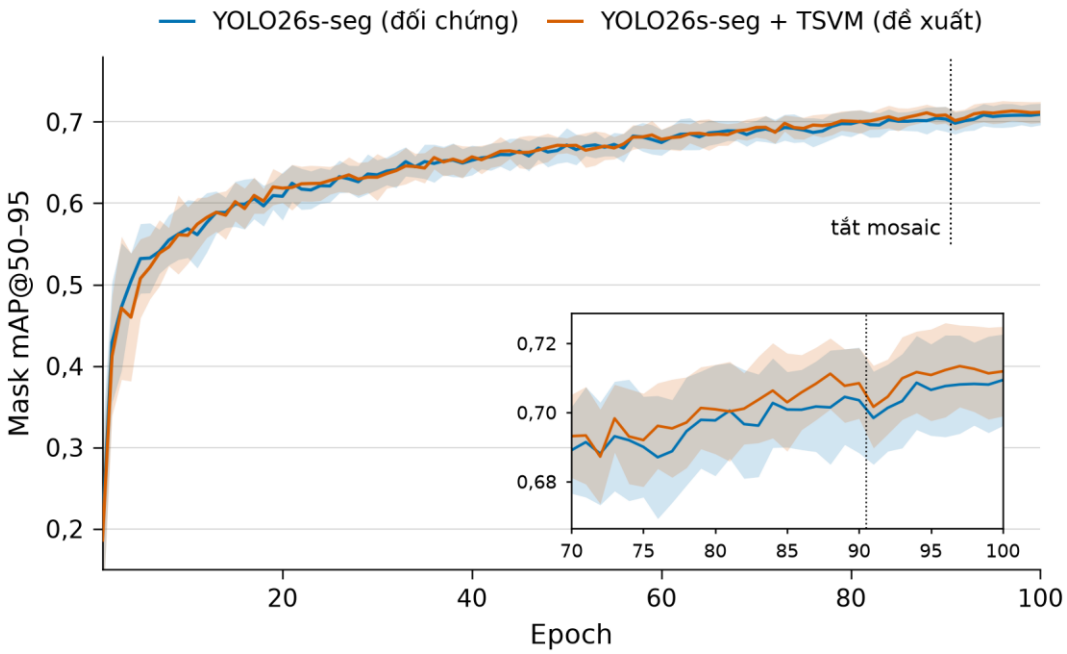
Hình PL.3: Hiệu số mask mAP50–95 (TSVM − gốc) của từng seed, sắp xếp từ nhỏ đến lớn.



Hình PL.4: Phân bố mask mAP50–95 qua 20 seed của hai mô hình.



Hình PL.5: Mask mAP50–95 theo ba quy tắc chọn epoch. Điểm là trung bình, thanh là ±1 độ lệch chuẩn.

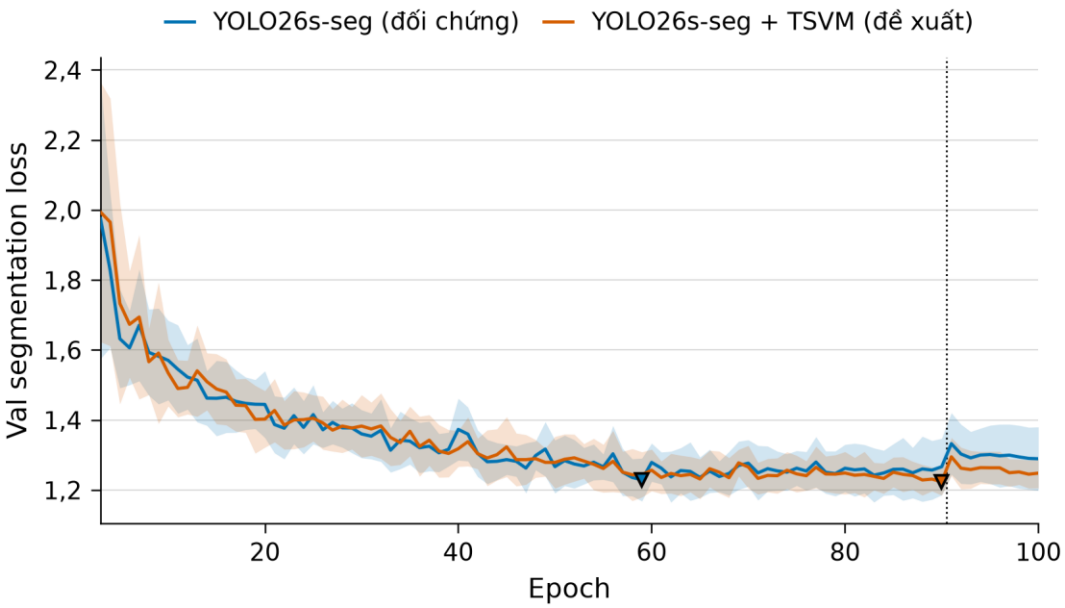


Hình PL.6: Mask mAP50–95 trên tập kiểm định theo epoch. Đường là trung bình 20 seed, miền tô là ±1 độ lệch chuẩn. Ô nhỏ phóng to đoạn epoch 70–100.

Bảng PL.15: Mask mAP50–95 tại một số epoch

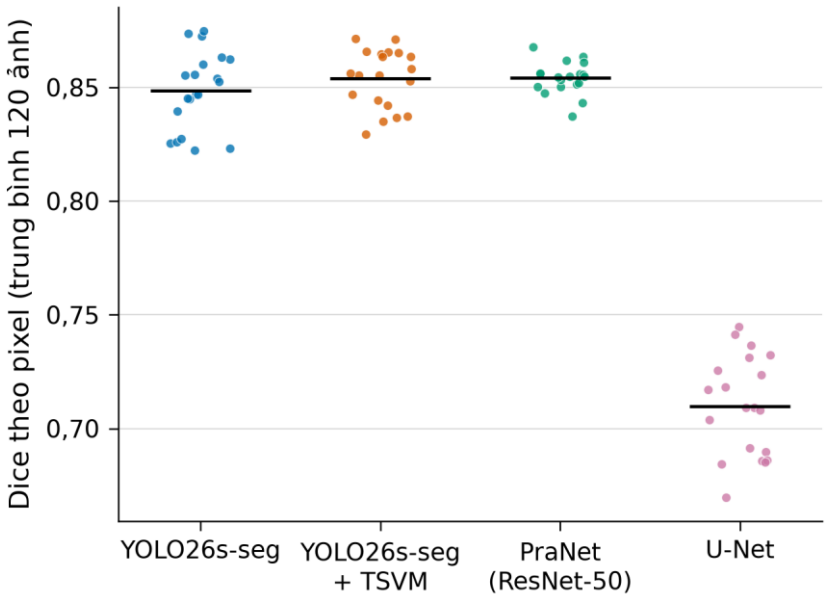
Epoch	YOLO26s-seg	TSVM
5	0,5320 ± 0,0427	0,5076 ± 0,0485
10	0,5687 ± 0,0355	0,5606 ± 0,0347
30	0,6349 ± 0,0183	0,6320 ± 0,0287
50	0,6716 ± 0,0208	0,6709 ± 0,0091
70	0,6890 ± 0,0125	0,6931 ± 0,0120
90	0,7035 ± 0,0152	0,7084 ± 0,0102
100	0,7093 ± 0,0132	0,7119 ± 0,0130

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

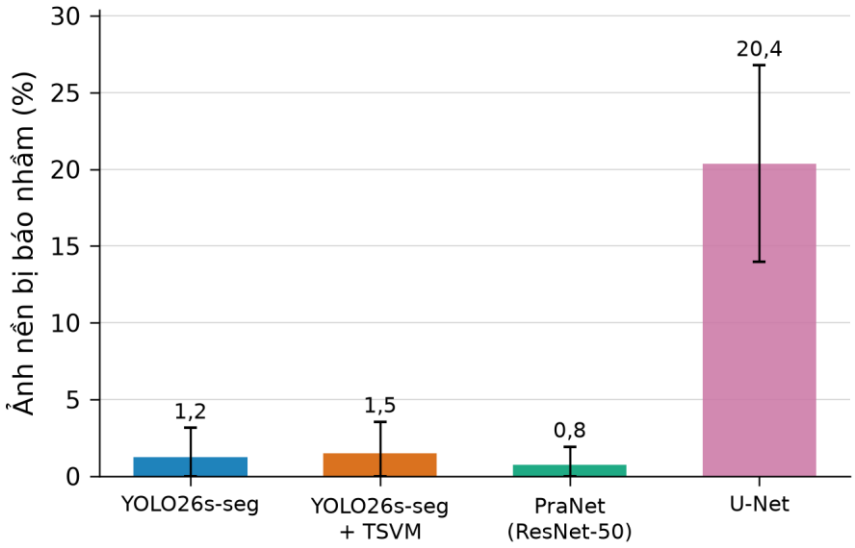


Hình PL.7: Hàm mất mát phân đoạn trên tập kiểm định theo epoch. Dấu ▼ đánh dấu epoch có giá trị trung bình thấp nhất.

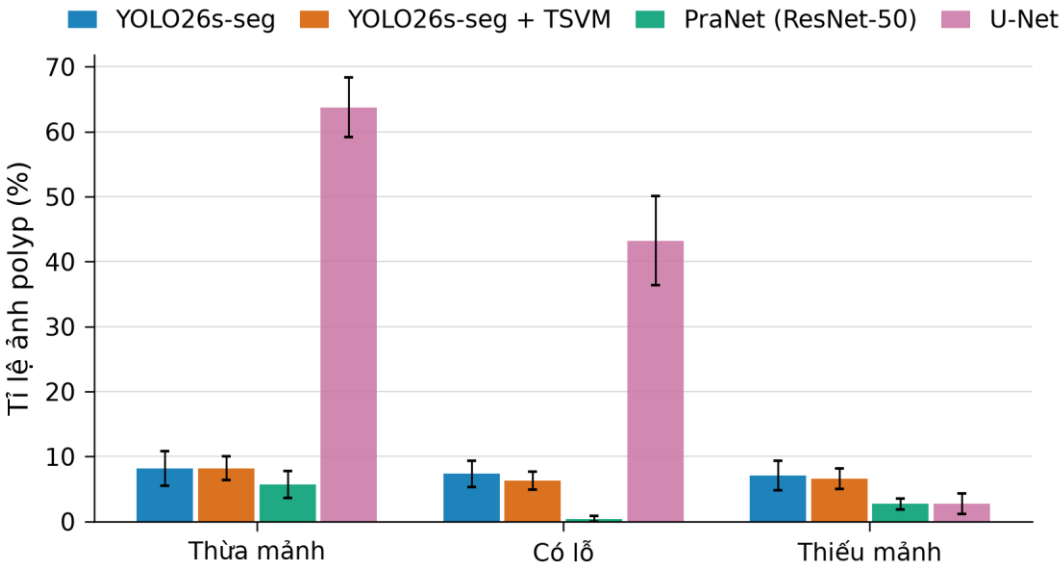
H.2. Đánh giá theo pixel, topology và độ trễ



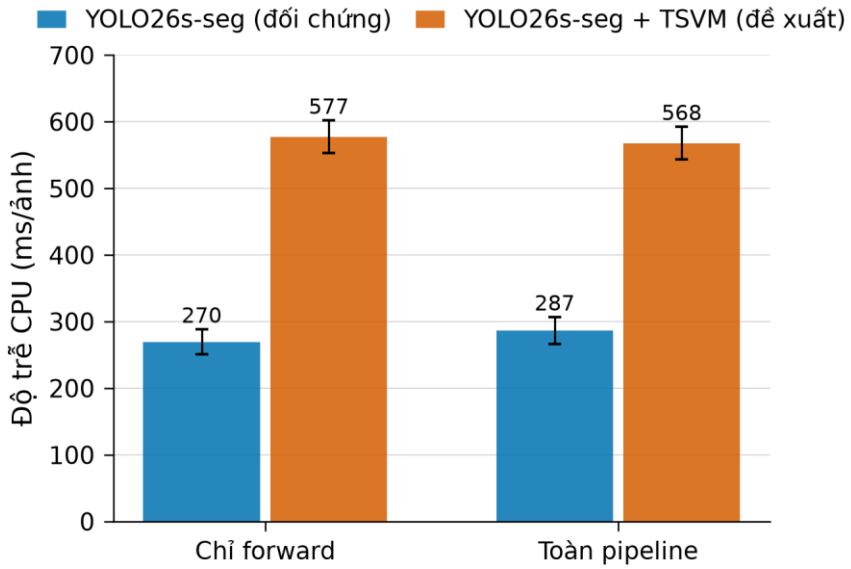
Hình PL.8: Dice theo pixel của từng seed.



Hình PL.9: Tỉ lệ ảnh nền bị báo nhầm (có ít nhất một pixel dự đoán là polyp) trên 40 ảnh nền.



Hình PL.10: Tỉ lệ ảnh mắc từng kiểu lỗi topology: thừa mảnh, có lỗ và thiếu mảnh.



Hình PL.11: Độ trễ trên CPU của hai mô hình ở hai chế độ đo. Thanh lỗi là ±1 độ lệch chuẩn qua 100 lần đo.

H.3. Dữ liệu ngoài

Bảng PL.16: Tỉ lệ ảnh có polyp bị bỏ sót hoàn toàn (mô hình không dự đoán pixel nào)

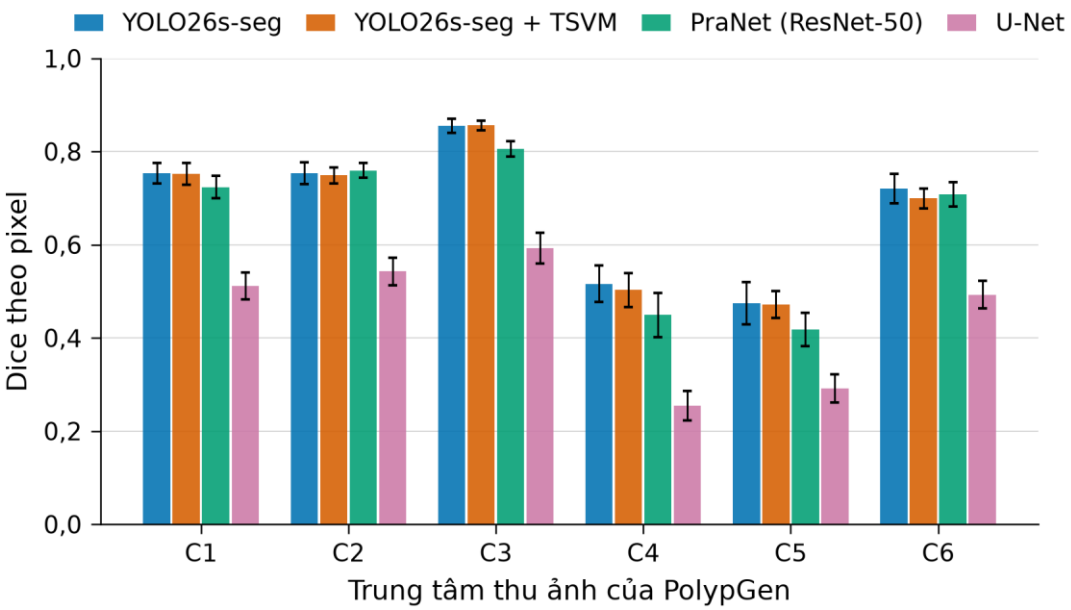
Bộ dữ liệu	YOLO26s-seg	TSVM	PraNet	U-Net	TSVM − gốc	p (t)
CVC-ClinicDB	11,3%	13,4%	10,1%	7,9%	+2,1 điểm %	0,003
CVC-ColonDB	28,2%	32,0%	32,0%	27,0%	+3,8 điểm %	< 0,001
ETIS-Larib	37,3%	38,6%	39,4%	20,6%	+1,3 điểm %	0,580
PolypGen (gộp)	18,1%	18,9%	16,3%	11,5%	+0,8 điểm %	0,296

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.

Bảng PL.17: Dice theo pixel trên từng trung tâm của PolypGen (20 seed)

Trung tâm	Ảnh có polyp	YOLO26s-seg	TSVM	PraNet	U-Net	TSVM − gốc	p (t)	p (Wilcoxon)
C1	251	0,753	0,752	0,724	0,511	−0,001	0,85	0,87
C2	270	0,753	0,749	0,759	0,543	−0,005	0,52	0,50
C3	456	0,855	0,856	0,806	0,593	+0,002	0,69	0,99
C4	146	0,516	0,503	0,449	0,255	−0,013	0,28	0,048
C5	206	0,474	0,472	0,418	0,292	−0,002	0,85	0,45
C6	83	0,720	0,699	0,708	0,492	−0,021	0,032	0,019
Gộp	**1.412**	**0,719**	**0,715**	**0,683**	**0,484**	−0,004	0,55	0,25

Nguồn: nhóm thực hiện tính toán từ kết quả thực nghiệm.



Hình PL.12: Dice theo pixel trên từng trung tâm của PolypGen.